基于线性规划与滚动优化的微网电力调控策略

摘　要

本文针对微网中光伏、储能与外购电协同的日内调度问题，在储能容量 1200–10800 kWh、充放电功率上限 5000 kW、单程效率 0.9、调度间隔 10 min 等约束下，以购电费、合同调整费与紧急补电费之和最小为目标，建立了**线性规划模型**。四问共用同一套母线功率平衡、储能状态递推、分段结算和 5 倍紧急购电口径，差别只在于决策时刻可获得的信息：问题 1 为确定性单日计划，问题 2 引入了负荷与光伏的预测不确定性，问题 3 引入日内发布的光伏预报和合同调整，问题 4 引入逐日波动的价格信息。各方案均由附件数据连续回测，并与对应的可实施基准逐项对照。

针对问题 1：给定单日负荷、光伏与电价，以功率平衡、储能递推、容量与功率边界、日初日末等电量为约束，我们建立了**确定性单日线性规划模型**，用 HiGHS 求解，并借同成本等价变换在不用二元变量的前提下排除同时充放电。两端电量均取 6000 kWh 时，全天购电量为 **59482.70 kWh**，对应电费 **35126.95 元**；不配置储能、按净负荷直接购电要 48052.05 元，储能使单日电费下降 **26.90%**。

针对问题 2：计划期延长为全年。以同星期历史值与近 7 日均值预测净负荷，用近 14 天误差构成 15 个场景，把缺额可能按 5 倍电价补购的代价写入目标函数，形成**两阶段情景线性规划**；情景缺额只作风险代理，实际执行按当前观测量反馈，储电量跨日连续传递。2 月 1 日至 12 月 31 日总费用 **1386.39 万元**，紧急购电 8.55 万 kWh，较点预测基准节省 13.34%，代价是常规购电偏多、未利用供给约 275.35 万 kWh。缺额惩罚由 5 倍提高到 8 倍时紧急电量降至 4.37 万 kWh，总费用反而增加了 21.73 万元。

针对问题 3：在零时合同的基础上引入了 0、6、12、18 时发布的光伏预报，按履行、取消、追加三段结算（取消部分付 50% 违约费、追加部分按 1.5 倍价），建立**滚动优化模型**，并用同一组情景与同一当前状态评价候选安排是否值得采纳。在零时预报相同的条件下，四次更新使全年费用由 1401.04 万元降至 **1364.29 万元**，节省了 36.75 万元（**2.62%**），紧急电量由 7.73 万降至 4.33 万 kWh，11 个月费用均有所下降。若取消部分改按“原计划照付、另计 50%”理解，更新仍节省 0.54%，方向不变；再计入费用门控后，1002 次调整机会采纳 612 次，另省约 5629 元。

针对问题 4：在逐日波动价格下重新解决前两问，并区分价格在决策时刻是否可见。主设定以历史同星期价格预测排计划、按实际价格结算，日前与滚动方案分别为 **1462.14 万元**与 **1440.37 万元**；如果当天价格在零时完全已知，则为 1451.68 万元与 1428.33 万元，只低了 0.72% 与 0.84%，储能等效循环由 591、597 次升到 622、627 次，可见价格信息本身的收益有限。

我们的创新点有三：一是以**共同名义储能轨迹加情景缺额**构成线性风险代理，带不确定性的全年回测因此仍可用稀疏 LP 求解；二是以同成本等价变换在连续变量框架内消除同时充放电；三是以同信息基准把信息价值、算法收益与结算口径分开计量。我们未把电池寿命损耗与日内价格预测误差计入模型，各项节省不可叠加；结论只适用于给定设备参数与交易机制下的费用比较。

关键词：线性规划；历史误差场景；滚动优化；储能调度；信息可得性

一、问题重述

1.1 问题背景

微网由光伏发电、储能和外部购电共同满足小区负荷。储能的用处是把低价时段或光伏富余时段的电量挪到供电紧张时段，但它既装不下无限的电量，转移过程也有损耗。图1标出我们采用的计量边界：功率在交流母线侧平衡，电池内部电量按充、放电效率递推。

我们讨论的调控目标是，在负荷得到满足且储能安全运行的条件下，使常规交易、紧急补电以及日内调整的总费用尽可能低。

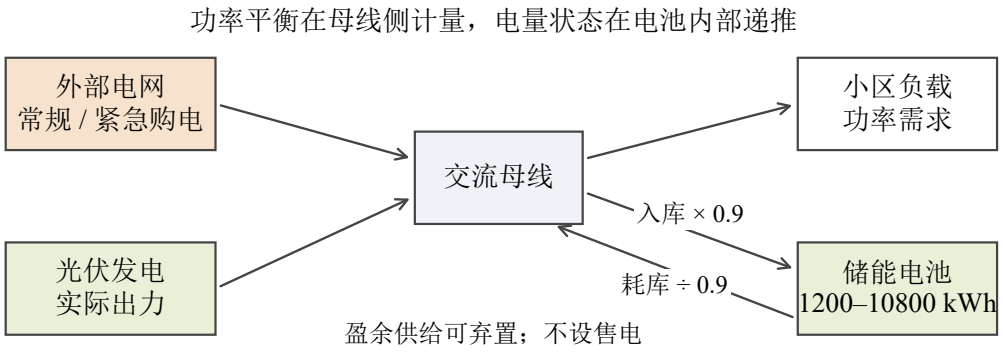


图 1　微网结构与充放电效率的计量边界

1.2 问题任务

题目以 10 分钟为调度间隔，每日共 144 个时段。问题 1 至 3 重复采用同一条日内电价曲线，问题 4 改用附件 4 的逐日实际价格。四问分别要求：

问题 1：依据附件 1 给定的负荷、光伏预测与电价，制定单日购电和储能计划。日初日末电量均为 6000 kWh，输出六个指定时段的购电量及 4 小时分组储能结果。

问题 2：将日前计划扩展为全年连续运行。微网每天零时确定计划，实际缺额按当时交易电价的 5 倍补购；报告 2 月 1 日至 12 月 31 日的结果及四个指定日期。

问题 3：使用 0、6、12、18 时发布的光伏预报调整未来执行安排，计入取消和追加购电的费用，并判断增加日内预报是否具有经济价值。

问题 4：在附件 4 的波动电价下重新求解问题 2、3，分别保存结果。未来价格何时可得会影响计划形成，我们把价格预测主方案与当天价格已知对照分开报告。

五份规定工作簿对应既定主模型；物理和因果性诊断用于核验，先知参照用于衡量信息与策略差距，门控、终端价值、预报融合及费用选型作为独立改进实验。

二、问题分析

四问使用同一物理基础与交易规则：功率在交流母线侧平衡，储能状态按单程效率递推，容量与功率受给定边界限制，常规购电按合同结算，缺额部分按 5 倍价格紧急补购。四问共用同一套母线功率平衡、储能递推与结算规则，差别只在信息条件：确定性输入、预测不确定性、日内预报更新与价格信息，我们按“先保证物理与结算可逐时核对、再逐个放宽信息条件”的顺序推进，每一步只作必要改动，使费用差异能够归因到明确来源。四问的继承关系、核心难点与所用方法汇总于表1，建模与评价流程见图2。

表 1　四个问题的递进关系、核心难点与建模求解方法

问题	新增条件	本质与核心难点	建模与求解方法	主要输出
问题 1	单日负荷、光伏与电价全部已知	本质是含储能的多时段能量调度；难点在功率与电量两组口径的衔接、同时充放电的排除，以及盈余供给的去向	确定性线性规划；HiGHS 求解；用同成本等价变换代替 0-1 变量	指定时段购电量与 4 小时分组充放电量
问题 2	负荷与光伏在计划时不可知；缺额按 5 倍价补购	本质是预测不确定性下的日前决策与跨日状态传递；难点是预测只能使用历史信息、风险必须提前计入目标	同星期与近 7 日均值预测，15 个历史误差情景的两阶段线性规划，观测量反馈执行	2—12 月总费用、紧急电量与事件清单
问题 3	零时计划为结算基准；0、6、12、18 时发布光伏预报	本质是分段结算下的滚动调整与采纳判断；难点是原计划不可改、每个时段只能结算一次	偏差变量的分段线性结算；滚动优化；同一情景下的费用门控	调整后购电量、调整费与紧急事件
问题 4	附件 4 逐日波动价格，发布时点未声明	本质是价格信息可得性对计划的影响；难点是决策与结算必须使用不同价格设定	沿用前三问模型替换价格系数；价格预测主设定与当天价格已知对照	两套全年工作簿与价格信息价值度量

问题 1 的分析：问题 1 的输入曲线完全给定，本质是含储能的多时段能量调度：在储能动态、容量与功率边界、日初日末等电量约束下，在低价时段或光伏富余时段充电以替代高价时段购电，可归结为线性规划。问题有三个难点：量纲与口径衔接，功率以 kW、电量以 kWh 计量，须统一到 10 分钟步长，且附件的时间标记存在 10 分钟错位，必须先固定离散化约定再建模；储能“充电—放电—存量”由效率耦合，若不加处理，最优解可能在同一时段同时充电与放电，虚增损耗而费用不变；盈余电量必须有明确去向，否则模型可借充放抵消消耗任意多余电能。针对上述难点，我们引入可弃置供给变量给出盈余的物理去向，并用保持储能状态与费用不变的等价变换证明主模型参数下总存在互斥最优解，从而无需引入 0-1 变量。所用理论工具为线性规划的等价变换。

问题 2 的分析：问题 2 把计划期延长为全年，核心变化在信息条件：附件 2 给出的是实际负荷与光伏，属于当天零时的未来量，直接作为已知输入即构成前视偏差。由此产生了两个难点。首先是预测：只能在历史信息下构造净负荷预测，并按时间顺序验证，避免用未来数据训练。其次是决策与风险的耦合：少购 1 kWh 省下的常规电费，可能不足以支付同量缺额的 5 倍紧急电费，所以应把未来缺额的可能代价提前计入目标。我们以历史误差轨迹构造情景，把这一权衡写成两阶段模型，其确定性等价形式仍为线性规划。第三个难点是跨日状态传递：日末电量必须来自实际执行结果，每日重置为 6000 kWh 等价于凭空注入或抽走能量。

问题 3 的分析：问题 3 在问题 2 之上增加两项条件：零时计划成为结算基准，日内只能调整未执行安排；0、6、12、18 时各发布一次光伏预报，用于重排剩余时段。难点在于信息与合同的相互作用。一方面，新预报既可能降低缺额，也可能使原计划显得保守，是否调整取决于偏差费用与紧急风险的大小，须在同一费用口径下比较；另一方面，原计划一经签订即为基准，多个时点反复优化只能改变尚未执行的安排，且每个时段最终只结算一次，否则会重复计费。我们把三段结算写成关于偏差变量的分段线性函数，使调整问题不引入整数变量仍为线性规划；求解上采用“更新状态、优化未来、只实施前一段”的滚动优化，并设置费用门控，只有预计更便宜的候选安排才会被采纳。所用理论工具为模型预测控制的滚动时域原理。

问题 4 的分析：问题 4 把电价由重复曲线改为附件 4 的逐日波动价格。难点不在价格进入目标（它只是目标系数的变化），而在价格信息的时点：附件 4 只给出价格数值，并未声明何时发布。若按实际价格排计划，等价于假设零时即可预知当天价格，与

问题 2 的因果性要求冲突；若按未知处理，又需要价格预测器，而题目未提供价格预报数据。我们因此把两种设定分开报告：主设定以历史同星期价格预测决策、按实际价格结算，与问题 2 保持同一信息约束；对照设定把当天价格视为零时已知，只放宽价格信息而保留负荷与光伏未知，用于量化价格信息的价值。低价充电、高价放电虽有经济动机，但容量边界、净负荷水平和价格预测误差会共同限制收益，该问题的结论因而必须与储能循环强度同时报告。

四问的递进关系体现为模型的可扩展性：问题 2 在问题 1 的物理约束上加入历史误差情景与执行反馈，问题 3 在问题 2 的跨日框架内加入合同偏差变量与日内更新，问题 4 只替换价格系数并区分信息设定。每后一问完整继承前一问的储能动态、边界条件与结算规则，新增部分均以独立基准对照检验。各项节省分别记账、不作叠加，也不以报告期费用反选主模型。

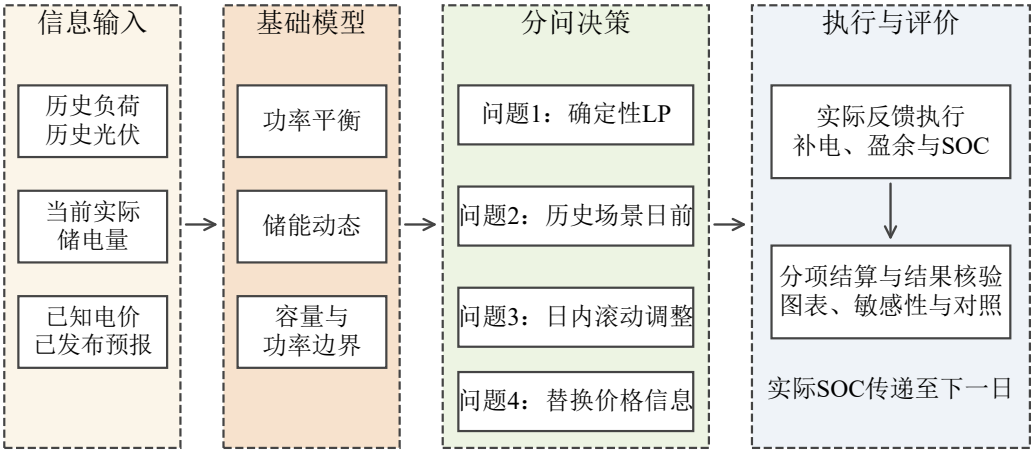


图 2　四个问题的建模联系与计划—执行—评价流程

三、模型假设

我们只保留进入模型或结果解释所必需的简化，列出以下九条假设，并说明每条的用途与依据。

(1) **时段内功率恒定：**每个 10 分钟时段内负荷、光伏、购电与充放电功率均视为常数，$\Delta t = 1/6$ h。(题目以 10 分钟为调度间隔，离散化后电量可由功率与步长相乘直接得到。)

(2) **损耗只计充放电环节：**充电与放电效率均取常数 0.9，线路损耗、自放电、温度变化和辅助耗电暂不计入。(题面只给出一个 90% 的数值，我们取常数以与本模型的时段长度相容；第 5.3.3 节对效率的其他解释另作对照。)

(3) **两侧计量口径分离：**充、放电功率按交流母线侧计量，储电量按电池内部计量；安全电量范围为 1200–10800 kWh，最大充、放电功率均为 5000 kW。(与题目给定边界一致，也避免把功率上限与容量上限混用。)

(4) **同一时段不同时充放电：**执行阶段的反馈规则天然互斥，计划阶段则通过等价变换取互斥最优解。(同时充放只增加损耗、不改变费用，排除后既符合运行实际，又无需引入 0-1 变量。)

(5) **外网只购不售：**不引入题面未给出的售电收益、并网功率上限或需量费；光伏富余与无法利用的已购电量允许弃置，已承诺电量仍按交易规则结算。（自行添加售电价格或接入容量会改变各方案的费用比较口径。）

(6) **只使用决策时刻已可得的信息：**计划不得使用未来实际量；1 月份用于初始化与方法选择，2 月至 12 月用于报告，不用报告期实际费用反选主模型参数。（这是四问费用差异可归因于信息条件的前提。）

(7) **储能状态跨日连续：**问题 1 两端电量均固定为 6000 kWh；问题 2 至 4 仅在 1 月 1 日零时初始化一次，次日接续上一日的实际电量。（每日重置会凭空注入或抽走能量，与全年连续运行的要求不符。）

(8) **合同偏差分段结算：**以零时合同为基准，履行量按原价、取消量按原价 50% 付违约费、新增量按 1.5 倍原价购入，最终偏差只结算一次；原费不退且另罚的解释单列对照。（对应题面的三段措辞，两种文字解释在第 9.2 节分别重算。）

(9) **执行优先保证负荷：**实际执行先满足负荷，再在容量与功率边界内决定充放电，不为追踪名义充电功率额外购买紧急电；供电不足部分按实际交易价格的 5 倍结算。（5 倍价对应题面规定，该规则只依赖当前观测量，是可实施的因果策略。）

效率在第 (2) 项中按单程解释，故回路效率为 0.81。例如母线侧输入 100 kWh，内部只增加 90 kWh；母线侧输出 100 kWh，内部减少约 111.1 kWh。我们把这一区别只放在状态动态中处理，输出电量不再重复扣除效率。

四、符号说明

令日期索引 $d = 1, \ldots, 365$，报告期为 $\mathcal{D} = \{32, \ldots, 365\}$；时段索引 $t = 1, \ldots, 144$，$t = 1$ 对应 0:00–0:10，状态 $E_{d,t}$ 为第 t 个时段结束时的电量，$E_{d,0}$ 为零时电量。发布时刻 $k \in \{0, 6, 12, 18\}$，误差场景索引为 s。单日推导省略日期下标。我们按集合与索引、输入与预测、设备参数、决策变量、费用与评价五类分组列出符号，同类量采用统一的字母约定：P 表示功率、E 表示电量、λ 表示电价、C 表示费用。

符号	说明	单位
(一) 集合与索引		
$\mathcal{D}$	报告期日期集合，即第 32 日至第 365 日	–
d, t, k, s	日期、时段、预报发布时刻、误差场景索引	–
$\mathcal{I}_{d,0}$	第 d 日零时可用的信息集合	–
$p^0, g^{\mathrm{keep}}, g^*$	原计划、当前执行安排、候选安排的功率向量	kW
(二) 输入与实际量		
$P_{\mathrm{load},d,t}, P_{\mathrm{pv},d,t}$	实际负载功率、实际光伏功率	kW
$\widehat{P}_{\mathrm{load},d,t}$	当日零时形成的负载预测	kW
$\widehat{P}^{(k)}_{\mathrm{pv},d,t}$	发布时刻 k 对应的光伏预测	kW
$n_{d,t}, \widehat{n}^{(k)}_{d,t}$	实际净负荷、预测净负荷，均为负载减光伏	kW
$n^{(k)}_{d,t,s}$	场景净负荷	kW
$\lambda_{d,t}, \widehat{\lambda}_{d,t}$	实际交易电价、日前预测电价	元/kWh
(三) 设备与时间参数		
η_c, η_d	充电、放电效率，均为 0.9	–
Δt	时间步长，等于 1/6	h
$E_{\min}, E_{\max}$	允许储电量下限 1200、上限 10800	kWh
$P_{\max}$	最大充电、放电功率，均为 5000	kW

符号	说明	单位
（四）决策与派生变量		
$P^{0}_{\mathrm{grid},d,t}$	零时原始计划购电功率	kW
$P_{\mathrm{grid},d,t}$	最终生效的常规购电功率，不含紧急补电	kW
$P_{\mathrm{ch},d,t}, P_{\mathrm{dis},d,t}$	母线侧充电输入、放电输出功率	kW
$E_{d,t}$	实际或模型中明确指定的边界储电量	kWh
$P_{\mathrm{em},d,t}$	实际紧急购电功率	kW
$P_{\mathrm{sp},d,t}$	弃光或未利用的已付费供给功率	kW
$u_{d,t,s}$	计划模型中的场景缺额补救变量	kW
$y^{+}_{d,t}, y^{-}_{d,t}$	相对零时计划的上调、下调功率	kW
（五）费用与评价指标		
C_{total}	报告期实际总费用	元
v_d, γ	电池内部单位库存价值、无量纲调节系数	元/kWh，–
$\widehat{V}_k(g)$	更新时刻对剩余安排的预计反馈费用	元

计划状态与实际状态：优化阶段的充放电变量与状态描述名义储能计划；实际执行时，以反馈规则重新计算充放电与状态。必要时我们分别加上标 pl 和 act。场景缺额 u 只是优化目标中的风险代理量，不与实际紧急功率 P_{em} 混用。

费用口径：我们定义原计划费用 C_0、上调费用 C_+、下调净费用 C_- 与实际紧急费用 C_{em}，总费用为

$$C_{\mathrm{total}} = C_0 + C_+ + C_- + C_{\mathrm{em}}. \tag{1}$$

主口径中 $C_- \le 0$，表示先扣回取消电量的全额原电费，再收取 50% 违约费后的净减少额。违约费用本身仍非负。保留原费的对照口径中 $C_- \ge 0$。

储能利用指标：为统一表征储能使用强度，定义基于 9600 kWh 可用容量的能量吞吐等效循环次数

$$N_{\mathrm{eq}} = \frac{\sum_{d\in\mathcal{D},t}\left(\eta_c P_{\mathrm{ch},d,t} + P_{\mathrm{dis},d,t}/\eta_d\right)\Delta t}{2(E_{\max} - E_{\min})}. \tag{2}$$

它只刻画运行强度，并不等同于寿命模型里的电池循环计数，我们也没有据此假设折旧成本。

五、问题 1 的模型建立与求解

5.1 模型准备

5.1.1 附件读取与时段对齐

四份附件的规模为：附件 1 共 144 个时段，附件 2 有 365 天负荷与实际光伏，附件 3 有 1460 条整点预报，附件 4 有 365 组日内电价。读取时先核对日期、时段与预报发布顺序，再检查维度、缺失值和非负性，四份附件全部通过。

附件 1 电价介于 0.3713–1.3952 元/kWh，附件 4 介于 0.0076–1.7936 元/kWh。

原始输入的时间字段从 0:10 到次日 0:00，共 144 个值；我们按区间末端标记解释，即 0:10 对应 0:00–0:10 的平均功率。附件 5 的部分模板却从 0:10–0:20 开始，末行延伸到次日 0:10，与完整自然日边界存在 10 分钟错位。输出保留原有工作表名称及字段结构，将 144 个时段标签统一为 0:00–0:10 至 23:50–24:00，不平移数值，这是明确的时间离散化约定；若另有权威时标解释，应据此重新映射并重新计算。

对功率 P_t，时段电量为 $q_t = P_t\Delta t$。例如 4 小时分组包含 24 个时段，其充电量等于 24 个充电功率之和乘以 1/6。输出工作簿以 kWh 计量，填表前统一做这一步换算。

5.2 模型建立

5.2.1 决策变量与目标函数

单日模型的主要变量为 $P_{\mathrm{grid},t}$、$P_{\mathrm{ch},t}$、$P_{\mathrm{dis},t}$ 与 E_t，并引入非负盈余处理量 $P_{\mathrm{sp},t}$。购电只发生在外网方向，目标函数为

$$\min C_1 = \sum_{t=1}^{144} \lambda_t P_{\mathrm{grid},t}\Delta t. \tag{3}$$

价格与时长都是已知系数，目标对购电功率线性。题目未给定的寿命费、售电收益与峰值需量费不进入本次计算。

5.2.2 功率平衡与储能边界

母线功率平衡及储能动态分别为

$$P_{\mathrm{grid},t} + P_{\mathrm{pv},t} + P_{\mathrm{dis},t} = P_{\mathrm{load},t} + P_{\mathrm{ch},t} + P_{\mathrm{sp},t}, \tag{4}$$

$$E_t = E_{t-1} + \eta_c P_{\mathrm{ch},t}\Delta t - \frac{P_{\mathrm{dis},t}\Delta t}{\eta_d}. \tag{5}$$

式(4)使富余供给具有明确去向；消去 $P_{\mathrm{sp},t} \ge 0$ 后，可等价写为供给不低于负荷和充电需求的不等式。式(5)中的充电量需乘效率，放电输出需除以效率，且始终使用 E_{t-1} 与 E_t 连接相邻边界。

$$1200 \le E_t \le 10800, \qquad t = 0, \dots, 144, \tag{6}$$

$$0 \le P_{\mathrm{ch},t} \le 5000, \qquad 0 \le P_{\mathrm{dis},t} \le 5000, \tag{7}$$

$$P_{\mathrm{grid},t} \ge 0, \qquad P_{\mathrm{sp},t} \ge 0, \tag{8}$$

$$E_0 = E_{144} = 6000. \tag{9}$$

上述目标、等式和不等式均为线性表达式，故在暂不添加乘积互斥约束的情况下，可构成线性规划 [1]。将式(3)与约束(4)–(9)汇总，问题 1 的模型为

$$\begin{cases} \min \quad C_1 = \displaystyle\sum_{t=1}^{144} \lambda_t P_{\mathrm{grid},t}\Delta t & \\ \text{s.t.} \quad P_{\mathrm{grid},t} + P_{\mathrm{pv},t} + P_{\mathrm{dis},t} = P_{\mathrm{load},t} + P_{\mathrm{ch},t} + P_{\mathrm{sp},t}, & \\ \qquad E_t = E_{t-1} + \eta_c P_{\mathrm{ch},t}\Delta t - P_{\mathrm{dis},t}\Delta t/\eta_d, & \\ \qquad E_{\min} \le E_t \le E_{\max}, & \\ \qquad 0 \le P_{\mathrm{ch},t} \le P_{\max}, \qquad 0 \le P_{\mathrm{dis},t} \le P_{\max}, & \\ \qquad P_{\mathrm{grid},t} \ge 0, \qquad P_{\mathrm{sp},t} \ge 0, & \\ \qquad E_0 = E_{144} = 6000, & \end{cases} \qquad t = 1, \dots, 144 \tag{10}$$

其中购电功率、充放电功率与储电量为决策变量，负荷、光伏与电价为已知系数，模型规模为 $4 \times 144 = 576$ 个连续变量。

5.2.3 线性性质与同时充放电处理

电价为正并不能自动保证 LP 最优解不同时充放电；我们在允许零成本弃置、效率为常数的条件下，给出一个保持状态与费用的显式消除方法。设某时段 $c = P_{\mathrm{ch},t} > 0$、$p = P_{\mathrm{dis},t} > 0$，取

$$\delta = \min\left(c, \frac{p}{\eta_c \eta_d}\right), \qquad c' = c - \delta, \quad p' = p - \eta_c \eta_d \delta. \tag{11}$$

按上式调整后，至少有一个新功率为零，储能状态增量保持不变。购电功率保持不变，多出来的净供给 $(1 - \eta_c \eta_d)\delta$ 交给弃电变量吸收即可：费用不变，各功率上界也照旧满足。因此，至少存在一个不同时充放电、费用相同的最优解。程序按式(11)处理求解结果，并核验所有时段确实互斥。

若将来引入弃电上限、售电收益、负价格或额外运行机制，上述等价性需重新判断；必要时可引入二元变量 z_t，设置 $P_{\mathrm{ch},t} \le 5000 z_t$、$P_{\mathrm{dis},t} \le 5000(1 - z_t)$，形成混合整数线性规划。我们的主模型不需要二元变量。

5.3 模型求解与结果分析

5.3.1 稀疏线性规划求解

输入数据读取后，按变量顺序组成稀疏约束矩阵，调用 `scipy.optimize.linprog` 并设置 `method='highs'`[2, 3]，检查求解成功标志、等式残差和不等式余量，再输出逐时功率、时段电量与分组汇总。固定初始状态后，消去盈余变量和固定为零的紧急变量，核心问题具有 $4 \times 144 = 576$ 个连续变量。

5.3.2 单日购电与储能调度

单日最优购电量为 **59482.70 kWh**，购电费用为 **35126.95 元**。指定结果见表3和表4。

表 3　问题 1 指定时段常规购电量（电量单位：kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天常规购电量：59482.70 kWh				全天总费用：35126.95 元	

表 4　问题 1 储能充放电量与边界电量（单位：kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量：6000.00				24:00 储电量：6000.00	

购电表列出六个指定 10 分钟时段，储能表按 4 小时汇总。值得核对的是 12:00–12:10：该段仍购入约 480 kWh，而 10:00、14:00 和 20:00 的指定时段无需购电；日间购电与储能补充需求及后续价差有关，并非异常。

全天充电约 2.074 万 kWh、放电约 1.680 万 kWh，差额约 3941 kWh。日初日末电量相等时，$Q_{\mathrm{dis}} = \eta_c \eta_d Q_{\mathrm{ch}}$，本次计算满足该关系，差额可归为充放电损耗。

4 小时分组里同时出现充电和放电，只是说明组内不同 10 分钟时段发生了两类操作，并非同一时段两者并行。

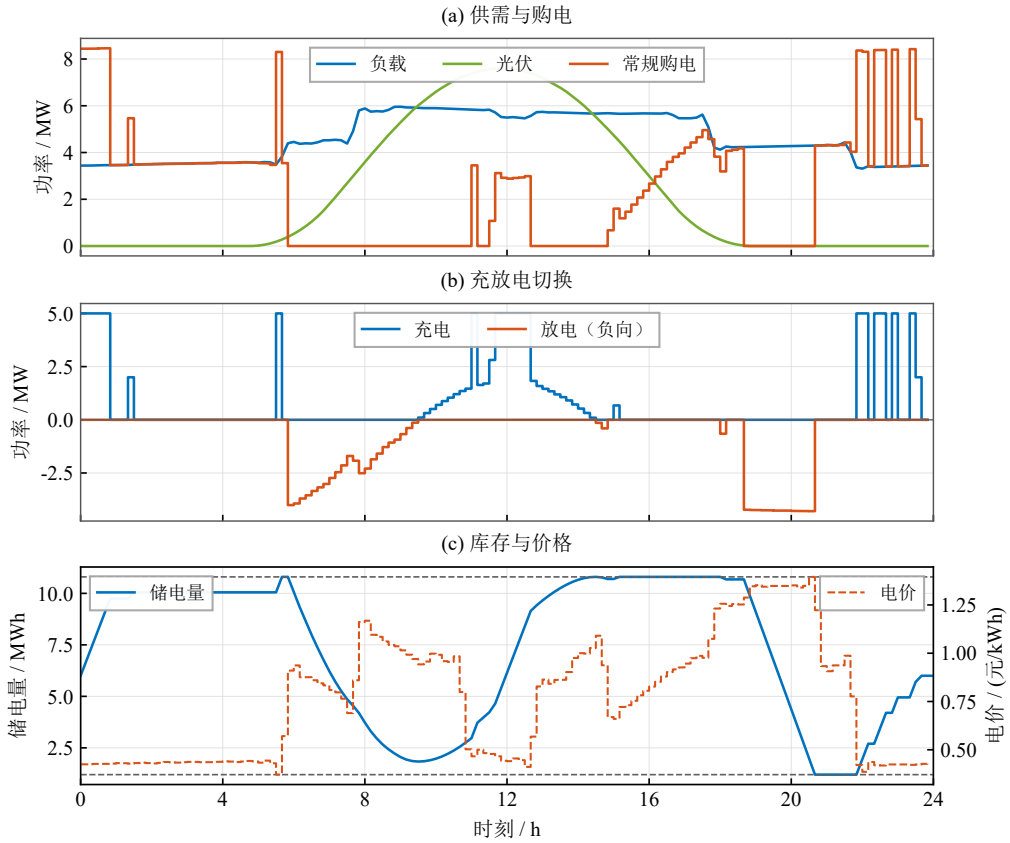


图 3　附件 1 算例中，储能在容量与功率边界内转移购电需求

图3中储电量多次贴近边界，可见低价购电与高价放电同时受 9600 kWh 可用容量和 5000 kW 功率上限的约束。同样输入下若不配储能、直接购买正净负荷，费用为 48052.05 元，本方案节省 12925.10 元、约 26.9%。该降幅只对应附件 1 单日算例，全年节省率见问题 2。

5.3.3 效率解释的敏感性

我们把“充放电效率 90%”解释为两侧单程各 0.9。为检验该解释的影响，以同一附件 1 独立构造 LP，分别改变效率分配，其余约束与两端电量保持一致。若把 0.9 视为回路效率并对称分配，费用约为 33801.50 元，比主设定低 3.77%。仅在充电或放电侧计损耗时，费用分别约 33416.53 元和 34168.61 元。精确结果见附录 D 表68。

相同的回路效率仍可能算出不同费用，因为两侧效率还决定了给定内部容量与母线功率下实际可充、可放的电量。图4显示效率口径对结果有实质影响，主计算维持两侧各 0.9。

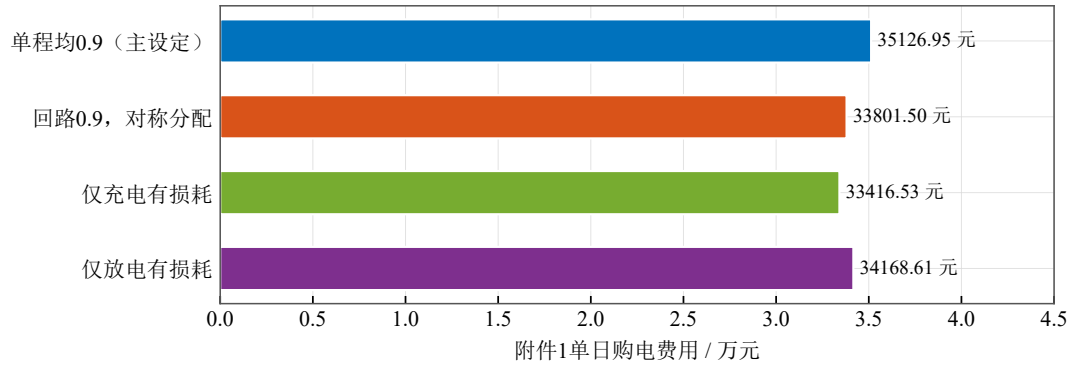


图 4　效率解释改变单日费用，全部比较保持容量与两端电量一致

六、问题 2 的模型建立与求解

6.1 数据特征与历史预测

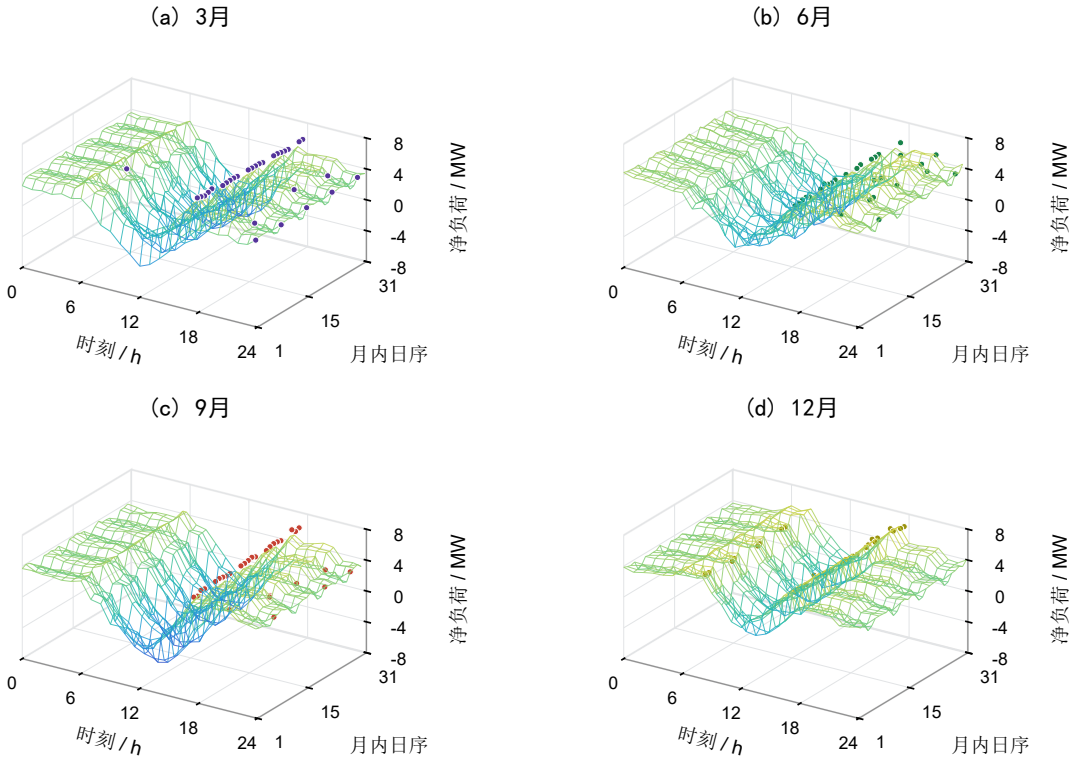


图 5　四个代表月份的实际净负荷曲面，散点标出每日峰值（统一纵轴尺度）

图5以日序、时刻和净负荷展示 3、6、9、12 月的实际变化。净负荷为负荷减光伏；细网格连接逐小时抽取的 10 分钟观测并保留末时段，散点取每日全部 144 时段的最大值。四图共用纵轴范围，网格不代表回归拟合。图示仅作事后描述，不作为零时已知输入；月内平均曲线另存于插图文件夹。季节差异仍需通过历史信息逐日跟踪。

验证按时间顺序进行，不随机打乱训练与验证数据 [4]。候选方法有前一日持久性、3 日和 7 日移动平均、14 日指数加权平均，以及过去最多四个同星期日期的均值，指数权重取 $\exp(-j/5)$，j 为距最新一天的天数。我们用 1 月 15 日至 31 日的逐日历史预测 MAE 选定方法，2 月 1 日起固定方法、滚动更新历史输入；1 月运行的热启动统一用持久性方法，1 月 1 日历史不足时以附件 1 曲线作初始先验，保证 1 月的运行不依赖事后选型。

对任一待预测序列 $x_{d,t}$，选出的同星期预测与 7 日均值分别为

$$\widehat{x}^{\text{week}}_{d,t} = \frac{1}{|\mathcal{L}_d|}\sum_{\ell\in\mathcal{L}_d} x_{d-\ell,t}, \quad \mathcal{L}_d = \{7,14,21,28\}\cap[1,d-1], \tag{12}$$

$$\widehat{x}^{\text{mean7}}_{d,t} = \frac{1}{m_d}\sum_{\ell=1}^{m_d} x_{d-\ell,t}, \quad m_d = \min(7,d-1). \tag{13}$$

历史不足一周时，同星期方法退化为最近七日的平均。各方法评价见表5；WAPE 用绝对误差总和除以实际值总和，避免夜间光伏零值让 MAPE 失去定义。

表 5　历史预测误差（1 月验证，2–12 月测试）

变量	选定方法	验证 MAE	测试 MAE	测试 RMSE	测试 WAPE
负载/kW	同星期均值	117.84	188.88	262.06	4.09%
光伏/kW	7 日移动均值	96.18	153.40	303.86	6.45%
电价/(元/kWh)	同星期均值	0.03	0.0489	0.0696	6.45%

1 月验证选中了负载、价格的同星期均值和光伏的 7 日均值，与“负荷和电价按星期重复”的日常经验相符，但仅一个月的验证不足以证明该规律全年稳定。表5中的误差大小还要放到费用模型里检验：电量紧张时，几个连续时段的偏差可能比孤立的同量偏差更昂贵。

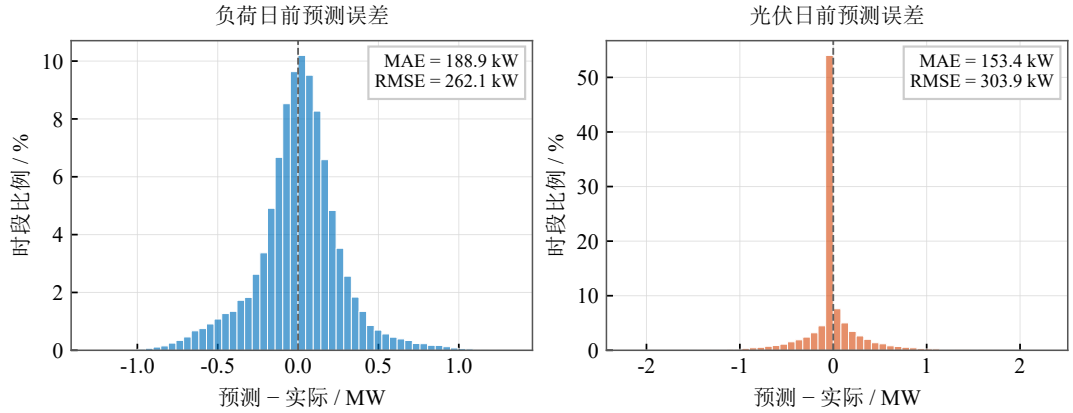


图 6　报告期日前预测误差分布：误差较小的主体之外仍有偏离较大的时段

图6统计报告期全部 48096 个时段，误差定义为预测减实际，纵轴是各分组的时段占比。零误差附近的高度集中主要来自夜间——光伏此时接近零，白天时段的误差仍分布在更宽的范围，不能由该图推断白天预测同样准确。缺额风险还需要在费用目标中单独处理。

6.2 模型建立

6.2.1 历史误差场景与费用目标

令 $\varepsilon^{(k)}_{j,t} = n_{j,t} - \widehat{n}^{(k)}_{j,t}$ 为过去日期 j 相同发布时刻的净负荷预测误差。主模型采用过去 $H = 14$ 天误差轨迹，加上一个零误差场景，得到

$$n^{(k)}_{d,t,s} = \widehat{n}^{(k)}_{d,t} + \varepsilon^{(k)}_{j_s,t}, \qquad \varepsilon^{(k)}_{j_0,t} = 0. \tag{14}$$

场景保留历史日内误差轨迹的形状并等权使用，成熟阶段场景数 $S = 15$；这是近期误差经验分布的近似，不宣称覆盖全部未来误差，也不额外假定正态分布，其窗口敏感性见第 9 节。

一次少购电所省下的常规电费，可能不足以支付同量缺额的 5 倍紧急电费。日前模型因而同时选择共同购电计划、共同名义充放电轨迹及各场景缺额变量 $u_{d,t,s}$，目标为

$$\min J_d = \sum_t \lambda_t P^0_{\mathrm{grid},d,t}\Delta t + \frac{1}{S}\sum_s\sum_t 5\lambda_t u_{d,t,s}\Delta t. \tag{15}$$

对所有 t, s 设置

$$P^0_{\mathrm{grid},d,t} + P^{\mathrm{pl}}_{\mathrm{dis},d,t} - P^{\mathrm{pl}}_{\mathrm{ch},d,t} + u_{d,t,s} \geq n^{(0)}_{d,t,s}, \tag{16}$$

$$u_{d,t,s} \geq 0. \tag{17}$$

名义储能满足式(5)–(7)，以实际 $E^{\mathrm{act}}_{d,0}$ 起步，日末电量自由。共同计划是第一阶段变量，场景缺额是第二阶段变量；各场景没有分别递推自己的电池状态。整理后，问题 2 的日前模型为

$$\begin{cases} \min \quad J_d = \displaystyle\sum_t \lambda_t P^0_{\mathrm{grid},d,t}\Delta t + \frac{1}{S}\sum_s\sum_t 5\lambda_t u_{d,t,s}\Delta t \\ \text{s.t.} \quad P^0_{\mathrm{grid},d,t} + P^{\mathrm{pl}}_{\mathrm{dis},d,t} - P^{\mathrm{pl}}_{\mathrm{ch},d,t} + u_{d,t,s} \geq n^{(0)}_{d,t,s}, \quad \forall t, s, \\ \qquad \text{式(5)–(7)（共同名义储能与功率边界），} \\ \qquad E^{\mathrm{pl}}_{d,0} = E^{\mathrm{act}}_{d,0}, \qquad u_{d,t,s} \geq 0, \\ \qquad P^0_{\mathrm{grid},d,t} = \pi_t(\mathcal{I}_{d,0}, E^{\mathrm{act}}_{d,0}), \end{cases} \qquad t = 1, \ldots, 144,\ s = 1, \ldots, S \tag{18}$$

最后一式表示计划只能由零时已有信息与当前电量决定，是模型的可实施性约束[5]。

仅保留一个零误差场景时，模型退化为点预测基准，供给通常由较便宜的常规购电满足。

需要指出，J_d 只是计划阶段的风险代理而非实际费用：运行时电池按当期供需反馈，实际紧急量未必等于 LP 中的 u。因此最终费用由连续反馈仿真单独结算，第 9 节再用时间配对诊断、窗口敏感性和反馈对照核查这项近似的量级。

6.2.2 实际反馈与跨日状态传递

在执行时段 t，以生效常规功率和实际净负荷计算

$$R_{d,t} = P_{\mathrm{grid},d,t} + P_{\mathrm{pv},d,t} - P_{\mathrm{load},d,t}. \tag{19}$$

若 $R_{d,t} \geq 0$，取

$$P_{\mathrm{ch},d,t} = \min\left\{R_{d,t}, P_{\max}, \frac{E_{\max} - E_{d,t-1}}{\eta_c \Delta t}\right\}, \quad P_{\mathrm{dis},d,t} = P_{\mathrm{em},d,t} = 0, \tag{20}$$

并令 $P_{\mathrm{sp},d,t} = R_{d,t} - P_{\mathrm{ch},d,t}$。若 $R_{d,t} < 0$，则

$$P_{\mathrm{dis},d,t} = \min\left\{-R_{d,t}, P_{\max}, \frac{\eta_d(E_{d,t-1} - E_{\min})}{\Delta t}\right\}, \tag{21}$$

$$P_{\mathrm{em},d,t} = -R_{d,t} - P_{\mathrm{dis},d,t}, \quad P_{\mathrm{ch},d,t} = P_{\mathrm{sp},d,t} = 0. \tag{22}$$

再依据式(5)更新实际状态。该反馈规则不为维持原定充电功率而紧急购电，而是优先满足实际负荷；其计算只依赖当前已观测量。紧急电量和实际费用为

$$Q_{\mathrm{em},d} = \sum_t P_{\mathrm{em},d,t}\Delta t, \tag{23}$$

$$C_d = \sum_t \lambda_t P^0_{\mathrm{grid},d,t}\Delta t + \sum_t 5\lambda_t P_{\mathrm{em},d,t}\Delta t. \tag{24}$$

次日初值为 $E^{\mathrm{act}}_{d+1,0} = E^{\mathrm{act}}_{d,144}$。主方案从 1 月 1 日连续运行，2 月 1 日零时实际电量约 9341 kWh，年末约 1553 kWh。

6000 kWh 只在 1 月 1 日零时使用一次；若每天恢复到该值，相当于每天凭空补入或移走一部分能量。

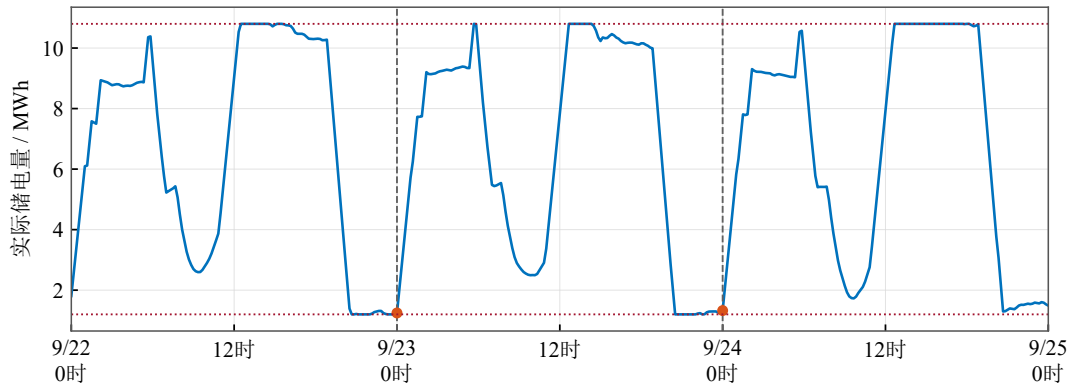


图 7　9 月 22—24 日实际储电量连续衔接，日初不重置为 6000 kWh

6.2.3 终端库存价值的独立修正

不强制日末恢复初值，可能使短时域模型过早耗用库存。为此在日前目标中加入线性终端价值项：

$$\min J^{\mathrm{term}}_d = J_d - v_d E^{\mathrm{pl}}_{d,144}, \qquad v_d = \gamma\eta_d\bar{\bar{\lambda}}_{d+1}. \tag{25}$$

其中 v_d 单位为元/电池内部 kWh；$\bar{\bar{\lambda}}_{d+1}$ 是零时可得的次日平均价估计。重复电价直接采用已知曲线均价；波动电价使用次日对应的过去同星期价格，**不使用当天尚未观测的价格**；该线性项仅改变目标系数，不增加整数变量，也不改变跨日实际状态传递。

γ 从 $\{0, 0.5, 1, 1.5\}$ 里选，办法很直接：每个候选都从 1 月 15 日的实际电量出发模拟到 1 月 31 日，比谁的“现金费用减去期末库存价值”更低，库存价值按附件 1 零至 6 时平均电价乘 η_d 计算。2 月起冻结选出的 γ，接着 1 月末的实际电量继续往下跑，1 月的历史不回头改。各候选参数的 1 月份库存修正费用见附录 D 表62，冻结参数后的报告期结果如下。

表 6　所选终端价值仅在波动电价日前方案中降低报告期费用

方案	γ	总费用/元	紧急电量/kWh	期末 SOC/kWh
重复电价：原模型	0.00	13863865.66	85480.79	1552.80
重复电价：终端价值	0.00	13863865.66	85480.79	1552.80
波动电价：原模型	0.00	14621415.17	83563.75	1548.85
波动电价：终端价值	0.50	14608967.43	83551.31	1545.92

重复电价选中 $\gamma = 0$，即验证期不支持额外的终端奖励。波动电价选中 $\gamma = 0.5$，报告期少支出约 1.24 万元。这说明库存价值值得继续考察，但单个系数能否普遍适用，证据还不够。

终端价值只是对次日价值的线性近似，并未真正求解未来 48 小时，因此不作为收入从电费中扣减。

6.3 模型求解与结果分析

6.3.1 可实施策略与未来已知参照

定义零时信息集 $\mathcal{I}_{d,0}$ 包含截至前一日完成的观测及当时已知的交易规则，要求 $P^{0}_{\mathrm{grid},d,t} = \pi_t(\mathcal{I}_{d,0}, E^{\mathrm{act}}_{d,0})$。附件 2 的当日实际曲线不属于该集合。可实施的主策略按日期向前推进；当日先知参照则刻意放宽信息限制，在零时直接输入当天实际负荷、光伏和相应价格，逐日求解确定性 LP，但仍只优化当天且连续传递实际 SOC。

另设一条对照：把 2–12 月的全部实际曲线一次性交给一个全期 LP，期初、期末电量分别钉在对应主策略的水平，得到全期先知的成本下界。物理约束相同，先知可以用便宜的常规电替代紧急电，所以这个下界不会高于主策略费用；但它需要整个报告期的未来信息，优化时域也不同，不是可执行的方案。

表 7　先知参照费用更低，比较仍须区分信息与库存边界（费用/元，电量/kWh）

信息与策略	总费用	紧急电量	期初 SOC	期末 SOC
重复价：可实施日前	13863865.66	85480.79	9340.95	1552.80
重复价：当日先知	12250938.36	0.00	9340.95	1200.00
重复价：全期先知	12226776.72	0.00	9340.95	1552.80
波动价：可实施日前	14621415.17	83563.75	9218.19	1548.85
波动价：当日先知	12838166.23	0.00	9218.19	1200.00
波动价：全期先知	12779898.60	0.00	9218.19	1548.85

重复电价下，全期先知费用约 1222.68 万元，比主策略低约 163.71 万元。这笔差额里混着预测信息、风险代理与反馈策略三方面的影响，只能当作参考上界，而不是严格的完全信息期望价值。当日先知只优化当天，期末库存与主策略仍有差异，两者不能直接比较。

波动电价的全期参照还放宽了价格信息；第 8.2.1 节的对照只放宽价格信息，两者作用不同。

6.3.2 四个指定日期的结果

四个指定日期的完整指标见表8。指定时段购电量、4 小时分组储能量及紧急事件完整列于附录 C；问题 2 无日内调整，原计划量与最终常规购电量相同，全天总费用含实际紧急补电费。

表 8　问题 2 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	68991.98	68991.98	0.00	41918.85
06-21	36523.04	36523.04	0.00	21502.49
09-23	68842.58	68842.58	1094.87	49439.26
12-21	98355.39	98355.39	0.00	63456.61

指定日期中，6 月 21 日常规购电约 3.65 万 kWh，12 月 21 日约 9.84 万 kWh，外购需求差异较大。9 月 23 日约 1095 kWh 紧急电量中，约 1018 kWh 集中在 20:20–21:30，发生在光伏退出后的晚间负荷阶段；紧急费约 6905 元，占该日总费近 14%。

其余三个指定日期没有发生紧急事件；作为典型日，它们不代表所在月份整月都没有缺额。

6.3.3 报告期费用与紧急购电

报告期覆盖 334 天、48096 个时段，总费约 1386.39 万元，其中计划费约 1337.62 万元、紧急费约 48.77 万元。紧急购电共约 8.55 万 kWh，分布在 105 天的 733 个时段。

表 9　计入缺额风险后，常规购电增加而总费用下降（本算例）

方案	计划电量/万 kWh	紧急电量/kWh	紧急天数	总费用/万元
点预测基准	1999.72	674346.09	272	1599.85
问题 2	2179.10	85480.79	105	1386.39

与点预测基准比较，风险代理方案多购约 179 万 kWh 常规电，少购约 59 万 kWh 紧急电，净节省约 213 万元。提前计入缺额代价，在本算例中比运行后才统计紧急费更有效。

风险预留也有代价。报告期常规购电 2179.10 万 kWh，加上紧急电后共 2187.65 万 kWh，较实际净负荷 1793.92 万 kWh 多 21.95%。这部分差额可由能量守恒核对：未利用供给约 275.35 万 kWh、储能损耗约 119.16 万 kWh，再扣除消耗的期初库存约 0.779 万 kWh。预测偏差通过购电和弃电轨迹发挥作用，不应再作为第三种能量损耗重复相加；我们按这一恒等式核对全年能量，未见缺口。

334 天全部存在未利用供给，累计量约为紧急电量的 32.2 倍。母线侧的电能没有可追踪的独立标签，来源只能靠约定划分：在光伏先供负荷之后，若优先用外购电充电，未利用的外购电为 105.25 万 kWh；若优先用剩余光伏充电，则为 126.29 万 kWh，按对应时段购电价折合约 64.65 万至 74.43 万元。图8给出两种分配的结果，该区间反映的是归属约定的差异，不是概率误差带；这笔电费已包含在原计划费中，不再单列。

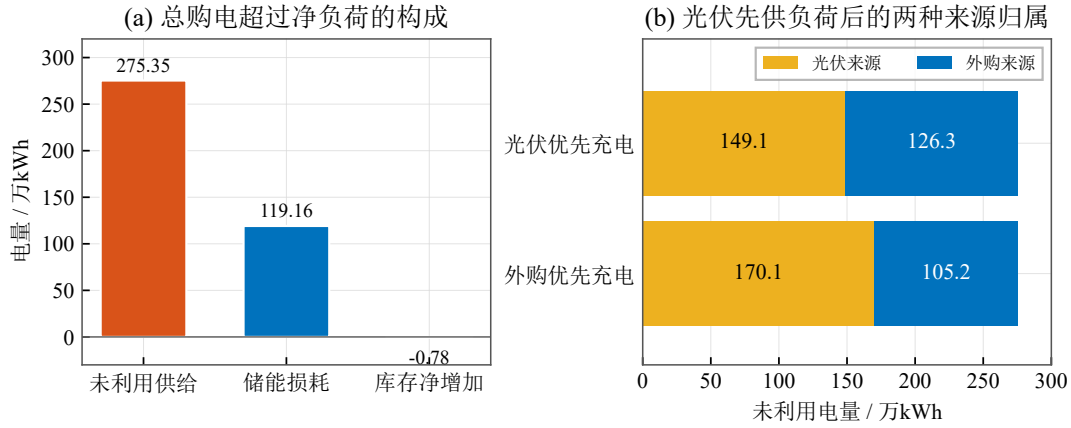


图 8　问题 2 多购电量由弃电、损耗和库存变化解释，弃电来源取决于分配约定

问题 2 的 6 月紧急购电约 24.9 MWh，为各月最高；总费用最高的却是 12 月，约 171 万元。图9把这两个指标分开后可以看出，紧急量较少的月份仍可能因常规购电需求较大而更昂贵。

图10把报告期的紧急购电按月、小时汇总，面板中同时标出报告期总量：问题 2 为 85.48 MWh，问题 3 为 43.27 MWh。两问的紧急补电都集中在少数月份的晚间时段，其

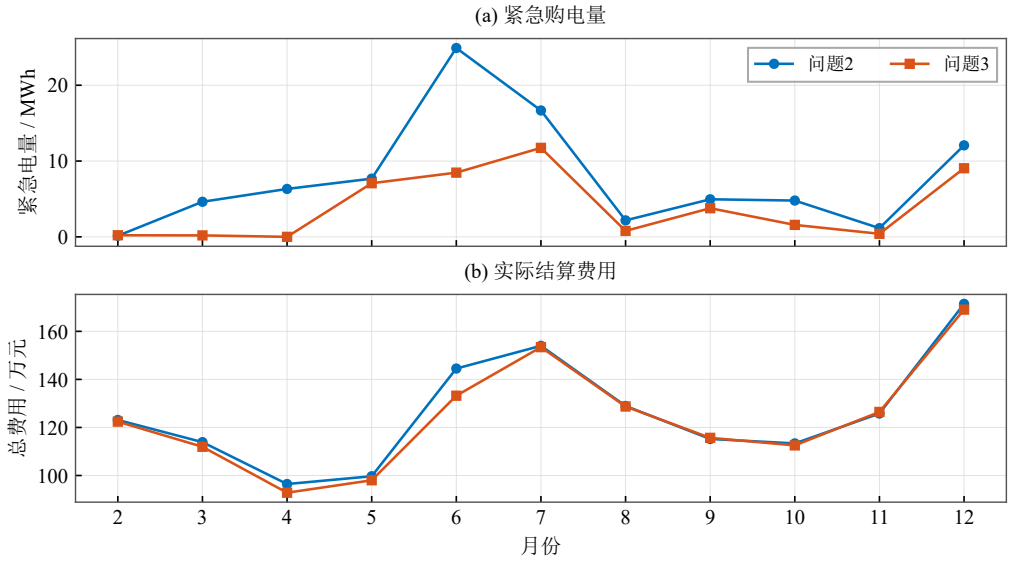


图 9　问题 2 的紧急电量在 6 月最高，总费用峰值却出现在 12 月

(a) 问题2 | 总量 85.48 MWh　　(b) 问题3 | 总量 43.27 MWh

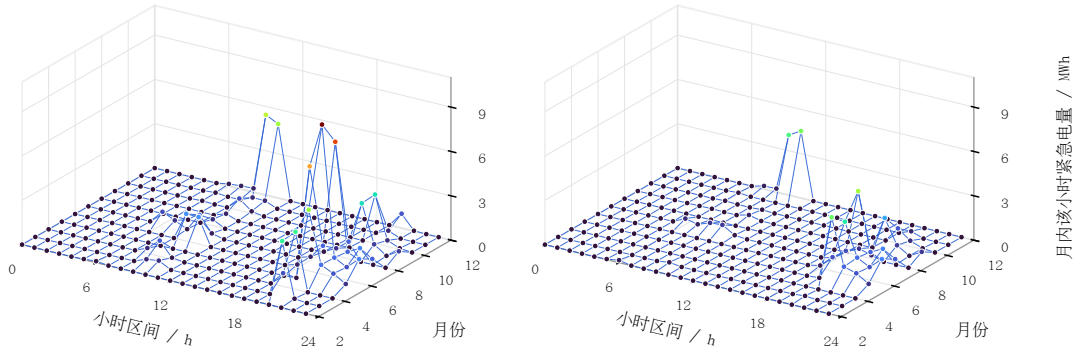


图 10　问题 2 与问题 3 紧急购电的月内小时分布（三维网格，竖轴为当月该小时合计电量，两面板共用同一竖轴尺度）

余时段接近零；问题 3 的峰值低于问题 2，但月度之内仍有局部缺额，年度紧急量下降不代表每天每个时段都改善。两套方案在零时信息和交易规则上都有差别，更新频率的净作用留到问题 3 的对照里单独衡量。

6.3.4 计划保守程度与实际费用的权衡

把式(15)中场景缺额的计划权重暂记为 ρ，仅在独立试验中取 2、3、5、8；实际紧急购电仍按题目要求的 5 倍电价结算。四组试验共享问题 2 在 2 月 1 日的实际初值及历史预测，逐日连续运行。$\rho = 5$ 的重算轨迹与主结果逐项一致。

图11中，权重提高使紧急量下降，未利用供给上升：由 5 提高到 8 时，紧急电量由 8.55 万降至 4.37 万 kWh，但总费用增加约 21.73 万元；降到 2 时总费约 1481.29 万元，同样更高。四个离散试验中原权重 5 费用最低，我们据此不声称它在所有权重上都最优。完整数据见附录 D 表67。

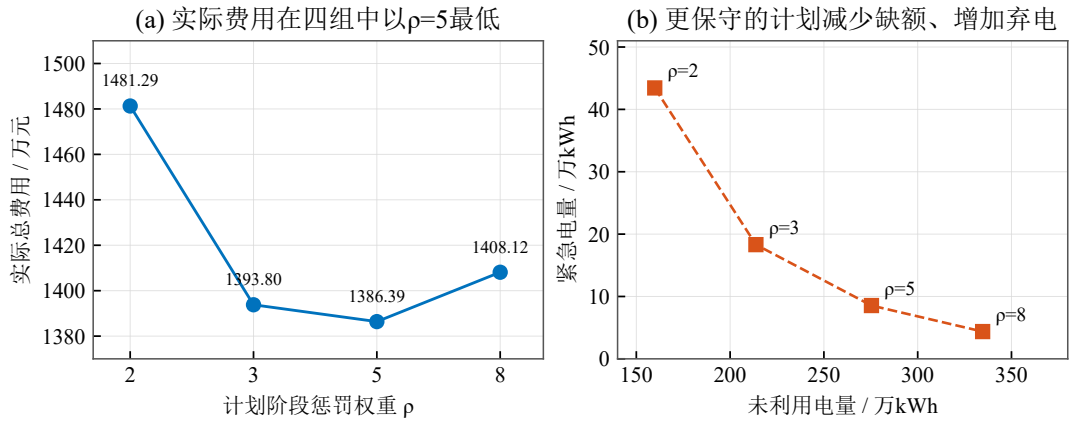


图 11　实际结算倍率固定为 5：更强风险预留减少紧急电，却未持续降低总费

七、问题 3 的模型建立与求解

7.1 模型建立

7.1.1 合同基准与分段结算

令 $P^{0}_{\mathrm{grid},d,t}$ 为零时计划，$P^{(k)}_{\mathrm{grid},d,t}$ 为发布时刻 k 得到的调整方案，引入

$$P^{(k)}_{\mathrm{grid},d,t} - P^{0}_{\mathrm{grid},d,t} = y^{+}_{d,t} - y^{-}_{d,t}, \quad y^{+}_{d,t}, y^{-}_{d,t} \geq 0. \tag{26}$$

主口径 B 分别核算原计划内的履行量、取消量和新增量：履行量按原价，取消量按原价的 50% 付违约费，新增量按 1.5 倍原价购入。原计划电量记为 q_0，正式调整后的常规电量记为 q，则

$$c(q;q_0) = \begin{cases} \lambda_t q + 0.5\lambda_t(q_0 - q), & 0 \leq q \leq q_0, \\ \lambda_t q_0 + 1.5\lambda_t(q - q_0), & q > q_0. \end{cases} \tag{27}$$

取消部分的 50% 电价由第一行直接体现。用功率偏差变量整理，可得

$$C^{B}_{d} = \sum_{t} \lambda_t \left(P^{0}_{\mathrm{grid},d,t} + 1.5y^{+}_{d,t} - 0.5y^{-}_{d,t} + 5P_{\mathrm{em},d,t} \right) \Delta t. \tag{28}$$

下调项的负号来自“退回 100% 原费、再收 50% 违约费”两步合并，并非负的违约单价。例如单价 1 元/kWh 时，原计划 5 kWh 正式下调至 4 kWh，应付 $4 + 0.5 \times 1 = 4.5$ 元；上调至 6 kWh，应付 $5 + 1.5 \times 1 = 6.5$ 元。若只是实际需求减少而未正式调整合同，则不产生退费。

作为原文字面解释的对照，口径 A 保留原计划全额电费，再收取下调违约费：

$$C^{A}_{d} = \sum_{t} \lambda_t \left(P^{0}_{\mathrm{grid},d,t} + 1.5y^{+}_{d,t} + 0.5y^{-}_{d,t} + 5P_{\mathrm{em},d,t} \right) \Delta t. \tag{29}$$

按口径 A，5 kWh 下调到 4 kWh 需付 5.5 元。允许免费处理盈余时，不下调而保留原购电始终不更差，下调不会改善目标。我们更倾向于口径 B：它让取消部分的 50% 价格直接发挥作用，也保留了双向调整的经济意义；口径 A 仍列为解释敏感性对照。

7.1.2 预报更新与滚动目标

问题 3 在发布时刻 k 读取相应预报行：发布时刻的观测记为第 0 个节点，其后 24 个整点预报为第 1 至 24 个节点，剩余 10 分钟区间按末端线性插值。各发布时刻分别

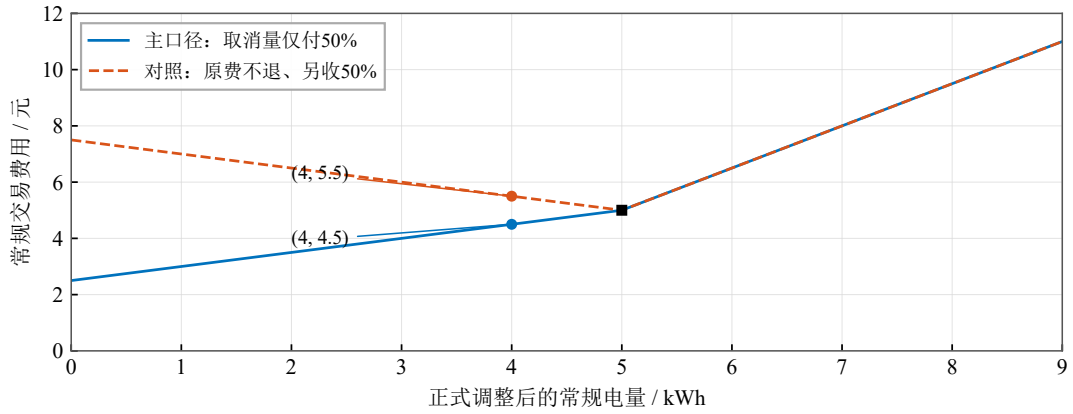


图 12　同一原计划下两种结算解释的费用曲线（5 kWh，1 元/kWh，不含紧急费）

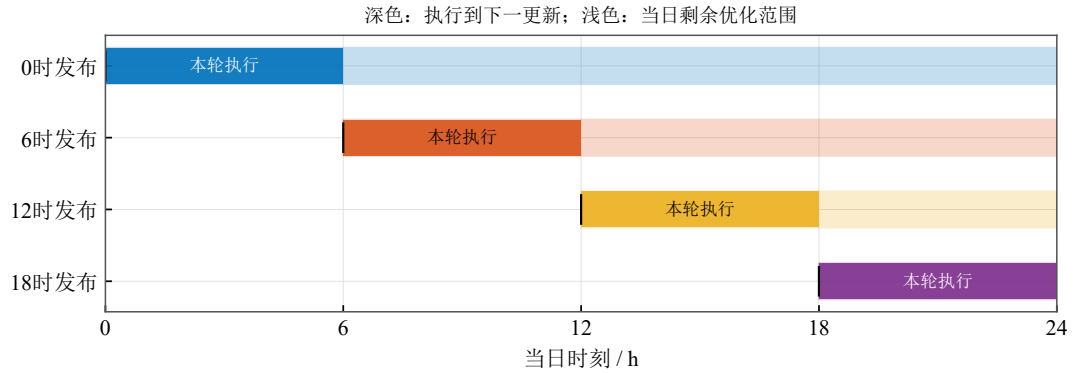


图 13　预报发布时间、剩余优化范围与本轮执行区间

构造误差历史——18 时发布的数据绝不会出现在 12 时的决策里。负载预测仍在零时生成，日内不使用未来实际负载。

零时用对应光伏预报与历史负载预测求解日前模型。对 $k = 6, 12, 18$，令剩余时段集合为 $\mathcal{T}_k = \{6k+1, \ldots, 144\}$。零时原计划一经形成就固定为结算基准，日内只更新尚未执行的安排，已执行时段不回改；原计划费在当前优化中是常数，可从目标中略去。主口径的剩余目标为

$$\min J_{d,k} = \sum_{t \in \mathcal{T}_k} \lambda_t (1.5 y^{+}_{d,t} - 0.5 y^{-}_{d,t}) \Delta t + \frac{1}{S} \sum_{s} \sum_{t \in \mathcal{T}_k} 5 \lambda_t u_{d,t,s} \Delta t. \tag{30}$$

约束包括式(26)、更新场景下的式(16)、储能动态及功率容量边界，初始状态为发布时刻的实际电量。对照口径 A 把目标中的 $-0.5y^{-}$ 换为 $+0.5y^{-}$。对于主口径，同时减少等量 y^{+} 与 y^{-} 可降低目标且保持净偏差不变，因此正电价下最优解不会同时取正，无需额外二元变量，该分段线性费用可直接通过偏差变量构成 LP。

本策略是有限日内剩余时域的滚动优化，遵循“更新状态、优化未来、只实施前一段”的控制思想 [6]，每轮按以下步骤执行：

算法 1　光伏预报到达后的滚动调整与费用门控

输入：当前实际储电量 $E_{d,0}^{\mathrm{act}}$，零时计划 p^0，现行安排 g^{keep}，最新发布的光伏预报，历史误差轨迹，价格估计 $\widehat{\lambda}_{d,t}$

输出：本时段执行的购电与充放电功率、更新后的安排 g^{next}、合同偏差结算

Step 1　零时获取当前 SOC、历史负载预测与零时光伏预报，形成原始全天购电计划 p^0，按反馈规则执行至下一发布时刻；

Step 2　读取刚发布的预报并对当日剩余时段插值，按同一发布时刻的历史误差重建场景 $n_{d,t,s}^{(k)}$；

Step 3　保持 p^0 不变，求解式(30)得到剩余时段的候选安排 g^*；

Step 4　用同一组情景、同一当前 SOC 与同一价格估计，按式(31)比较 $\widehat{V}_k(g^{\mathrm{keep}})$ 与 $\widehat{V}_k(g^*)$，按式(32)决定采纳或保留；

Step 5　只实施到下一发布时刻，按反馈规则更新实际充放电、紧急量与储电量，返回 Step 2；

Step 6　日末按最终生效购电量相对 p^0 的偏差结算一次费用，并传递实际 SOC。

附件预报覆盖未来 24 小时，本滚动基准只优化当日剩余时段，跨日部分未进入目标；终端价值在日前模型中单独检验。

7.1.3 计入调整费的采纳规则

预报更新并不必然触发购电调整。我们保存零时原计划 p^0 作为整日合同基准，另保存当前仍有效的执行安排 g^{keep}（可能来自上一轮已采纳的调整）。新预报到达后，式(30)生成候选 g^*；对这两条安排用同一组最新误差场景、同一当前实际 SOC 和同一价格估计，按式(20)–(22)逐时模拟实际反馈，定义

$$\widehat{V}_k(g)=\sum_{t\in\mathcal{T}_k} c(g_t\Delta t;p_t^0\Delta t)+\frac{1}{S}\sum_{s}\sum_{t\in\mathcal{T}_k}5\widehat{\lambda}_{d,t}P_{\mathrm{em},t,s}^{\mathrm{fb}}(g)\Delta t. \tag{31}$$

c 按式(27)及当时可用价格计算。紧急量 $P_{\mathrm{em}}^{\mathrm{fb}}$ 由场景内连续反馈产生，与 LP 的共同储能轨迹不同，评分因此会受到误差发生先后次序的影响。采纳规则为

$$g^{\mathrm{next}}=\begin{cases} g^*, & \widehat{V}_k(g^{\mathrm{keep}})-\widehat{V}_k(g^*)>\epsilon,\\ g^{\mathrm{keep}}, & \text{其他情形,}\end{cases}\qquad \epsilon=10^{-6}\text{元}. \tag{32}$$

ϵ 只是数值容差，不是额外的固定调整费；候选评分本身已含剩余偏差费用，不会重复计费。未通过筛选时保留现行安排，原始合同 p^0 不变。

评分增加 $O(Sm)$ 计算量，无需新增整数变量。它判断的是预测场景中的得失，实际运行仍可能出现相反结果。

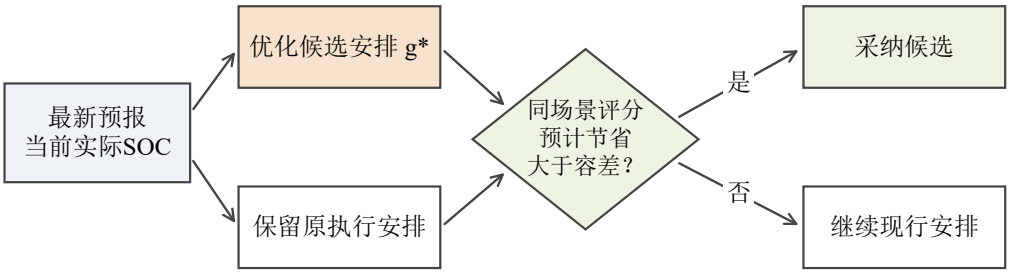
7.2 模型求解与结果分析

7.2.1 典型日的购电与操作区间

典型日汇总见表10，完整指定格式表与紧急事件见附录 C。其中“调整后”表示最终生效的常规购电总量，并非相对原计划的差值。

表 10　问题 3 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	71993.11	66853.81	113.04	43037.70
06-21	35054.77	34929.74	0.00	20690.52
09-23	72432.47	67541.88	464.02	46227.25
12-21	99503.29	97280.92	0.00	63388.89



原始零时合同不改；评分已含调整费；仅更新未来执行段

图 14　以同一信息比较候选与现行安排的日内调整采纳流程

表 11　问题 3 2025 年 3 月 20 日指定时段计划与调整对照（kWh）

时段	零时计划	最终生效	差额
10:00–10:10	0.00	0.00	0.00
12:00–12:10	674.37	558.17	-116.21
14:00–14:10	0.00	0.00	0.00
16:00–16:10	680.86	537.66	-143.20
18:00–18:10	701.05	701.05	0.00
20:00–20:10	0.00	0.00	0.00

3 月 20 日约 7.20 万 kWh 原计划经调整后变为约 6.69 万 kWh。减少量主要来自下调：上调仅约 13 kWh，下调约 5153 kWh；相应上调费约 15 元、下调净退费约 1772 元。该日仍有约 113 kWh 紧急补电。

上、下调量先分时段计算再汇总费用，全天净减少量不直接套用下调费率。

9 月 23 日，问题 3 紧急购电约 464 kWh、总费约 4.62 万元；问题 2 约 1095 kWh、4.94 万元。不过两者的零时预报来源和交易规则都不一样，这笔差额不能全记在预报次数上，下节另设相同零时预报的频率对照来单独识别。

7.2.2 预报更新频率的经济价值

为了单独识别日内更新的作用，使用相同零时预报、相同历史误差窗口和同一结算口径，比较仅零时决策、增加 12 时更新、使用全部四次预报三种方案。各方案均连续运行全年，实际 SOC 允许随其执行结果变化。

表 12　相同零时预报下，四次更新的报告期费用最低

预报时刻	上调费/元	下调净费/元	紧急电量/kWh	总费用/元
仅 0 时	0.00	0.00	77341.11	14010355.00
0、12 时	90237.60	-240050.46	60126.17	13739488.19
0、6、12、18 时	164969.36	-310478.47	43270.28	13642862.05

四次更新相对仅零时决策节省约 36.75 万元（2.62%），紧急电量由约 7.73 万 kWh 降至 4.33 万 kWh。相对只在 12 时增加一次更新，使用 6 时与 18 时预报还少支出约 9.66 万元。

在附件预报不另收费、结算规则取我们主口径时，这组结果支持日内更新。若预报服务或交易另有费用，需从实际节省中扣除，不能直接沿用本次收益判断。

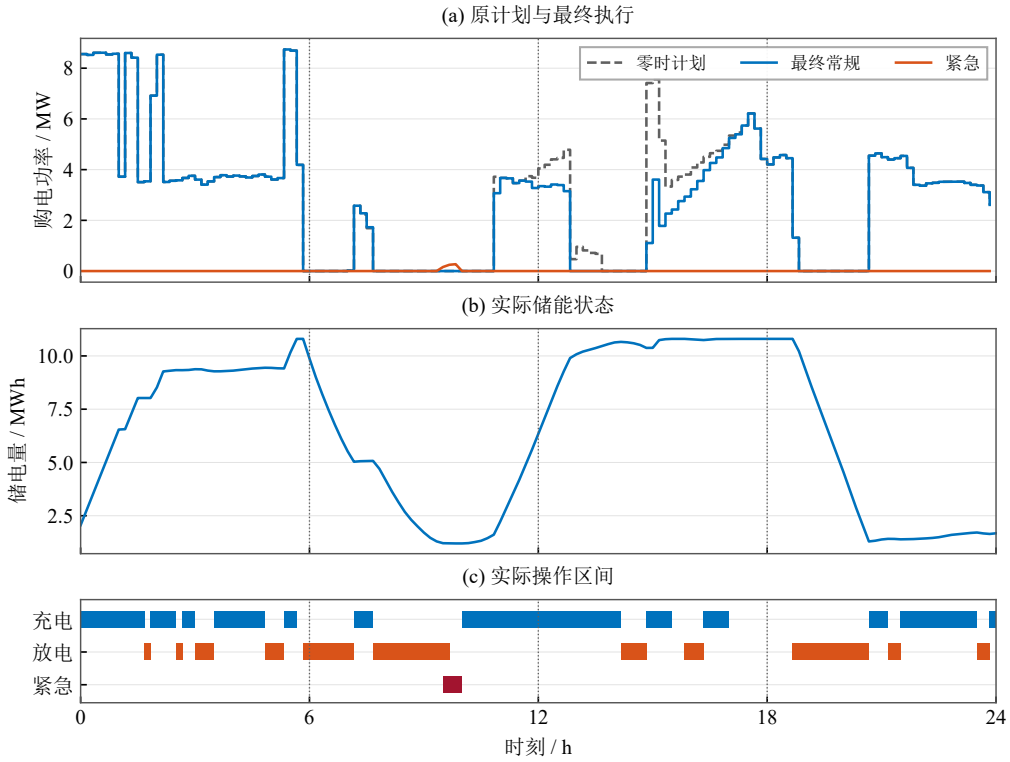


图 15　3 月 20 日多时段下调购电后仍需紧急补电；虚线标出预报更新时刻

问题 3 总费约 1364.29 万元，比问题 2 低约 22.10 万元。但这一差额混入了零时预报来源与结算口径的变化，并非全是日内更新的收益：换用“原计划照付、另计 50%”的口径后，问题 3 为 1393.45 万元，反而比问题 2 高约 7.06 万元。日内更新的净贡献应以下节同一零时预报下的频率对照为准。费用分解方面，原计划费约 1356.44 万元，上调费约 16.50 万元，下调净退费约 31.05 万元，紧急费约 22.40 万元；下调净退费为负只是“退全费再罚一半”合并后的结果，该项已含在总费内。

附件 3 的零时预报并非在每个指标上都更好。其全天 MAE 约 197 kW，高于历史预测的 153 kW；但 6 时、12 时更新对各自后续白天时段的 MAE 约为 185 kW 和 140 kW，低于对应的零时预报误差（图16；精确值见附录 D 表60）。这提供了一种解释：新增信息可能改善较关键的白天时段，即使全天误差并不占优。是否节省费用仍须看实际结算，我们因此把它交给同一零时预报下的频率对照来判断。

11 个月的费用差均为正，但 334 天中仍有 67 天更新后更贵，261 天更便宜，6 天基本相同。5 月的紧急电量也有增加。

按月度费用差计算，月均节省 3.34 ± 1.69 万元，范围 0.75–7.22 万元。其中 $\pm$ 取 11 个月的样本标准差（分母为 10），只刻画月间离散程度；各月长短、季节负荷与跨日库存都不同，该区间不能当作未来节省的置信区间。

逐月费用差与紧急电量差列于附录 D 表61。

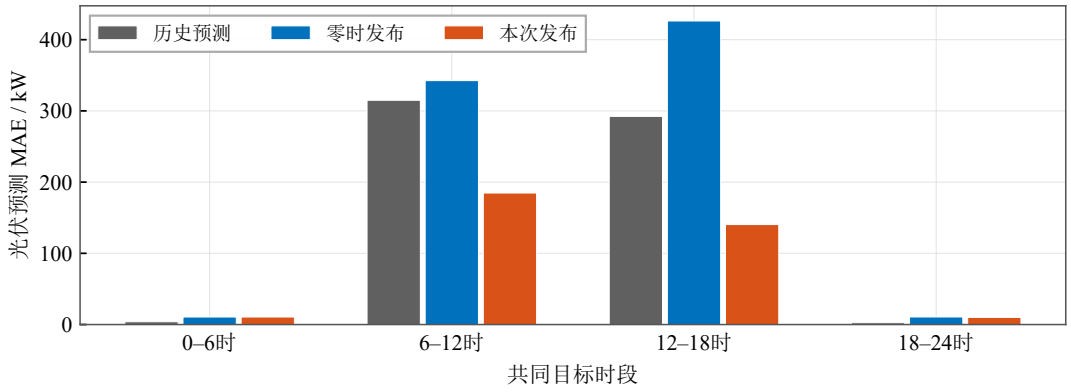


图 16　相同目标区间内，6 时和 12 时新预报降低了白天光伏预测 MAE

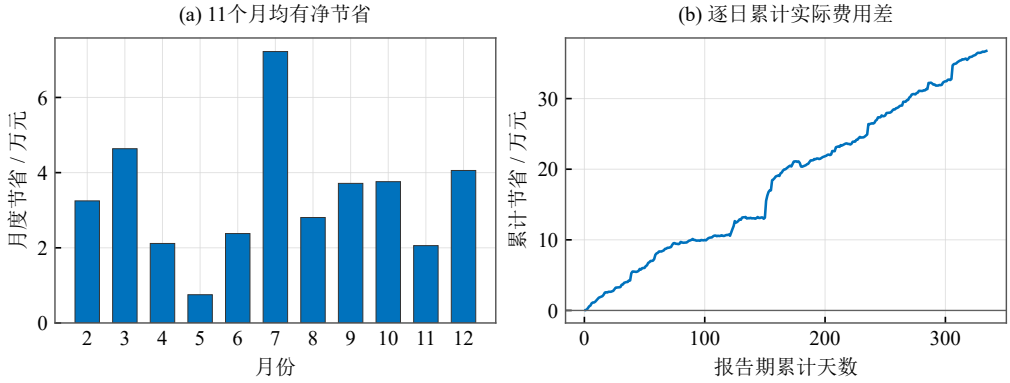


图 17　四次更新在 11 个月均节省费用，月度幅度差异较大

7.2.3 调整采纳规则的实际收益

表 13　计费后筛选调整小幅节省总费，紧急电量略增（采纳、保留以次计）

方案	总费用/元	紧急电量/kWh	采纳	保留
重复价：直接滚动	13642862.05	43270.28	1002	0
重复价：经济门控	13637232.96	44275.40	612	390
波动价：直接滚动	14403672.82	43101.90	1002	0
波动价：经济门控	14397071.03	43799.31	609	393

1002 次日内机会中，重复电价方案采纳 612 次，另有 119 次候选预计更贵、271 次预计费用基本相同。与直接滚动相比，最终节省约 5629 元，仅约 0.04%。波动电价下节省约 6602 元。收益并不大。

两种门控方案的紧急电量反而略有增加，净收益来自各项费用的共同变化；表中“直接滚动采纳 1002 次”指每次都采用优化解，不代表每次的购电量都真的变了。

门控与直接滚动的轨迹分别保存。五个规定工作簿仍采用直接滚动等既定主方案，便于逐项复核每个改动的效果。

八、问题 4 的模型建立与求解

附件 4 记录实际交易价格，却未说明何时发布。我们把未来价格未知作为主设定：历史价格预测参与计划，实际价格参与结算；当天完整价格已知的情形单列。两者都沿用前问的储能与功率约束。

8.1 模型建立

8.1.1 未来价格未知的主设定

价格未知时，把计划里用到的价格全部换成预测价 $\widehat{\lambda}_{d,t}$；到了结算，再全部换回附件 4 的实际价 $\lambda_{d,t}$。以问题 4 的滚动目标为例，替换后为

$$\min \sum_{t\in\mathcal{T}_k} \widehat{\lambda}_{d,t}(1.5y^{+}_{d,t} - 0.5y^{-}_{d,t})\Delta t + \frac{1}{S}\sum_{s}\sum_{t\in\mathcal{T}_k} 5\widehat{\lambda}_{d,t}u_{d,t,s}\Delta t. \tag{33}$$

物理约束和偏差定义沿用前三问。价格按同星期均值预测，6、12、18 时更新光伏时不更新价格预测。这里尚未构建日内价格预测器，即使更新时刻已经观测到当天此前价格，也不重估剩余价格，这一简化可能限制日内调节效果。两个问题 4 工作簿均使用该设定；把当天实际价格作为已公布曲线输入的情况仅出现在对照模型中。

8.1.2 当天价格已知的对照设定

对照模型在每天零时获知当天附件 4 价格曲线，令 $\widehat{\lambda}_{d,t} = \lambda_{d,t}$，但未来负载与光伏实际量仍然未知。设置该方案是为了分离价格信息的影响；它不是全信息完美预测，其费用也不构成全年最优下界。

8.2 模型求解与结果分析

8.2.1 价格信息与储能循环

表 14　提前获知价格降低回测费用，并增加储能循环强度

模型与信息	计划费/万元	紧急电量/kWh	总费用/万元	等效循环
日前，价格预测	1410.86	83563.75	1462.14	591.46
日前，价格已知	1398.32	86659.63	1451.68	622.10
滚动，价格预测	1431.33	43101.90	1440.37	597.33
滚动，价格已知	1418.38	43877.11	1428.33	627.39

提前已知当天价格时，日前、滚动方案总费分别为 1451.68 万元、1428.33 万元；相对价格预测主方案分别少支出约 10.46 万元、12.04 万元，约为 0.72%、0.84%。两套方案的储能等效循环分别由约 591、597 次增至 622、627 次。

更准确的价差信息增加了调节活动。我们把结论限定在交易费用层面：没有计入循环寿命成本，长期经济性尚不能据此确定。

价格预测设定继续对应 `result4-2.xlsx` 和 `result4-3.xlsx`。价格已知的两份逐时结果以“补充 _ 价格已知”前缀单独提供。前者强调决策时的信息限制，后者对应把附件 4 直接作为已知价格参数的解读；我们把两套结果分别保存，不混填为一份答案。

8.2.2 典型日及费用构成

两套可实施方案的四个指定日期汇总见表15和表16，对应紧急事件见附录 C 表50和表59。完整的指定时段购电表和 4 小时充放电表列于附录 C，全部 334 天结果写入所要求的两个工作簿。

表 15　问题 4-2 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	69026.66	69026.66	0.00	43411.08
06-21	36837.19	36837.19	0.00	18609.89
09-23	68830.77	68830.77	1076.53	51152.10
12-21	98222.77	98222.77	0.00	74692.77

表 16　问题 4-3 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	71871.84	66695.93	114.06	44535.29
06-21	35397.64	35241.80	0.00	18088.21
09-23	72586.44	67662.34	466.14	48346.91
12-21	99343.17	97208.96	0.00	74630.49

价格波动并未在每个日期都提高费用：6 月 21 日，日前方案由重复电价下约 2.15 万元降至 1.86 万元，12 月 21 日却由约 6.35 万元升至 7.47 万元。价格水平和价差出现的时段，与当时净负荷是否匹配，比“波动更大”这个概括更能解释差异。

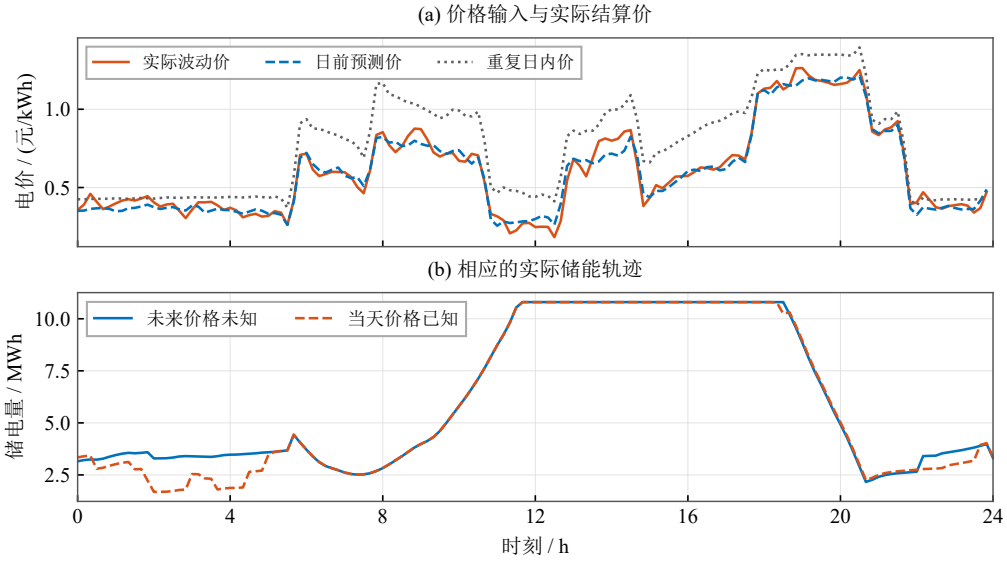


图 18　价格信息差异改变储能调度轨迹（6 月 21 日）

表 17　本样本中，波动电价下两套主方案的费用均高于重复电价基准

模型	计划费	上调费	下调净费	紧急费	总费用
问题 2	1337.62	0.00	0.00	48.77	1386.39
问题 3	1356.44	16.50	-31.05	22.40	1364.29
问题 4-2	1410.86	0.00	0.00	51.28	1462.14
问题 4-3	1431.33	17.81	-33.01	24.24	1440.37

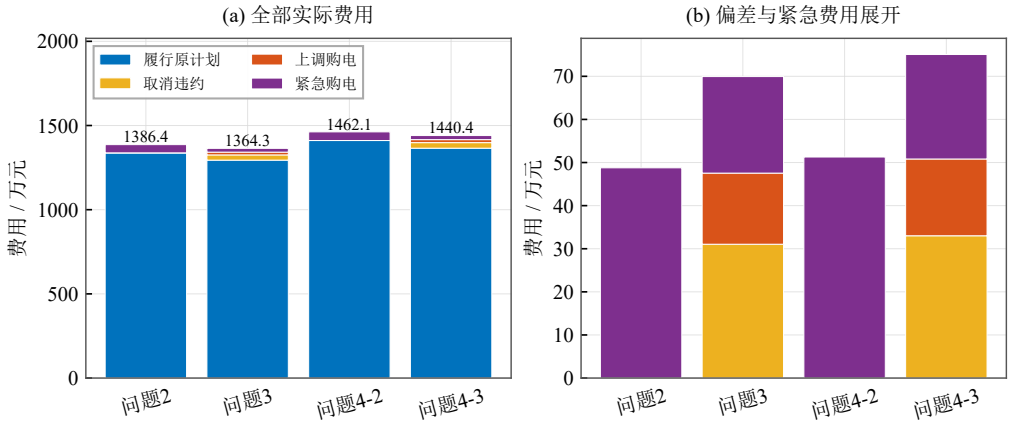


图 19　本样本的波动电价方案总费更高，履行原计划费占主要部分

报告期内，波动电价两套主方案总费约 1462.14 万元和 1440.37 万元，均高于对应的重复电价基准。图19把费用拆成履行原计划费、取消违约费、新增购电费、紧急费四个非负项，加起来恰等于表17的总费；其中履行原计划费是 $C_0 + 2C_-$、取消违约费是 $-C_-$。注意表里 C_- 按负数记，两种拆法不能混着加。另外，换价格后调度轨迹本身也变了，费用差不能全部解读为价格预测损失。

8.2.3 稀疏模型规模与计算耗时

剩余时域为 m 个时段、场景数为 S 时，日前模型包含 $(4+S)m$ 个连续变量、m 个状态等式与 Sm 个场景不等式；调整模型再增加 $2m$ 个偏差变量与 m 个计划差等式。矩阵非零元数量和构造空间均为 $O(Sm)$。成熟阶段 $S = 15$，全天日前模型有 2736 个连续变量；在 6 时更新时，$m = 108$，调整模型有 2268 个变量。

全年逐日前模型共调用 365 次 LP，四次预报滚动模型 1460 次；只计 2 月至 12 月分别为 334 次和 1336 次，其余用于 1 月初始化。求解时间受稀疏结构、退化性、算法选择和机器条件影响，不能由矩阵规模直接推出总运行时间也是 $O(Sm)$。实测耗时见表18，其中不含依赖安装、绘图、工作簿输出与 LaTeX 编译。

表 18　主模型的实测运行时间（含全年初始化）

模型	LP 次数	LP 耗时/s	全年仿真/s
问题 2	365	6.32	6.43
问题 3	1460	17.24	17.46
问题 4-2	365	5.46	5.55
问题 4-3	1460	16.87	17.09

九、模型检验与改进

9.1 物理约束、费用与信息时序核验

核验记录显示，15 组主模型及对照中，实际功率平衡的最大残差不超过 9.1×10^{-13} kW，日费用复算差不超过 5.9×10^{-11} 元，名义 LP 约束残差小于 10^{-7}；SOC 全程位于 1200–10800 kWh，未出现实际同时充放电。这些数字说明的是计算实现与模型一致，与预测误差无关。

因果性检查把第 181 天之后的实际量与预报做了扰动，前 180 天的问题 2 决策逐项不变；对问题 3 的四个决策时刻分别扰动当时尚未发布的信息，当前解同样不变。结算规则另用 5 kWh 分别调到 4、5、6、0 kWh 的例子核对，单价 1 元时应付 4.5、5、6.5、2.5 元，与程序输出一致。工作簿保存后重新读取，与轨迹逐项核对无误。

时间配对诊断暴露了风险代理的一项局限。对 9 月 23 日的场景逐列独立打乱（随机种子 20260911），LP 代理目标几乎不变，实际反馈费用却改变了（表63）。原因在于 LP 共用储能轨迹，逐时缺额代价没有利用跨时段配对。

因此，我们只将其称为**逐时缺额风险代理**。保存了整条历史误差，并不等于优化器已经利用了误差的时间相关性。

图20(a) 中，同一初值下代理目标变化约为零，实际反馈费用却增加约 199 元；(b) 单独列出门控的采纳数量。两张图分别对应评价环节的两个侧面。

固定问题 2 常规购电计划后，另试验 36 时段储能保留规则：根据日前预测的后续缺额留电，再在容量和功率边界内放电。它使报告期总费增加约 87.49 万元，紧急购电增加约 18 万 kWh；只有 20 天更便宜，214 天更贵。

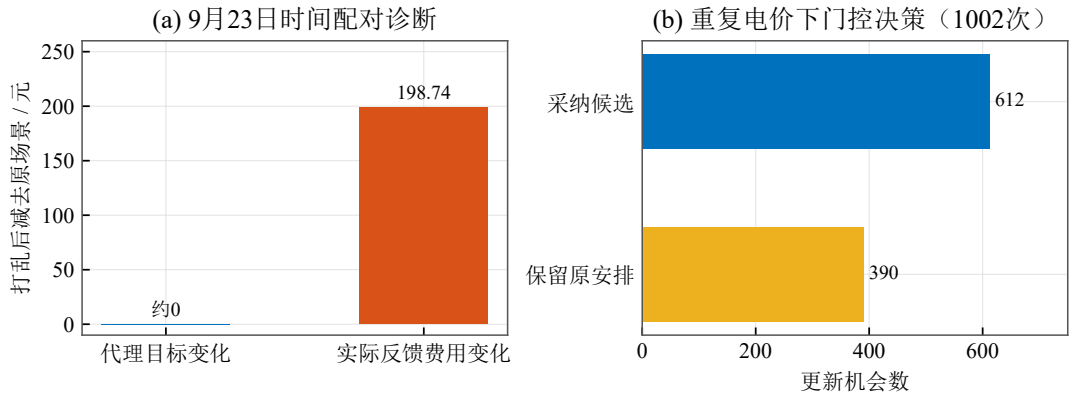


图 20　代理目标与实际反馈可有不同响应，门控也会保留部分原安排

该对照未改善即时放电策略：保留规则增加了当期高价补电，却未在后续兑现足够收益，但单次对照不足以判定即时放电最优。

9.2 结算与参数敏感性

主模型与各项独立试验的关键参数及确定方式见表19：涉及信息与风险的参数均由已完成日期的验证结果选定，报告期不再调整；涉及题意歧义的参数则并列两种解释重新求解，不因某种解释结果更低而选用它。

表 19　关键参数的取值与确定方式

参数	取值	确定方式
误差回看窗口 H	14 天	主模型事先设定，第 9.2 节比较 7、14、28 天窗口；属事后敏感性范围，不是置信区间
情景数 S	15	14 条历史误差轨迹加 1 条零误差轨迹，等权重近似误差经验分布
缺额计划权重 ρ	5	与题面紧急购电的 5 倍电价一致；第 6.3.4 节另取 2、3、5、8 作保守度试验，结算始终按 5 倍
终端价值系数 γ	0 或 0.5	由 1 月 15–31 日验证期的库存修正费用选出，2 月起冻结
预报融合权重 α_k	0.25/0.50/0.75/0	由 1 月份各发布时刻的剩余当日光伏 MAE 选出，零时用整日误差评价
采纳容差 ϵ	10^{-6} 元	数值容差，只用于判断候选是否严格更优，不是固定调整费
时间步长 Δt	1/6 h	题目给定的 10 分钟调度间隔
充放电效率 η_c, η_d	0.9	题面 90% 的单程解释；第 5.3.3 节并列回路效率等其他解释的单日费用

表 20　是否退回取消电量原费，会改变下调行为及总费

价格条件与口径	下调电量/kWh	下调净费/元	总费用/元
重复日内价，主口径	1022284.79	-310478.47	13642862.05
重复日内价，不退原费	0.00	0.00	13934495.57
波动电价，主口径	1037194.76	-330099.56	14403672.82
波动电价，不退原费	0.00	0.00	14711757.51

口径 B 下，问题 3 累计下调约 102 万 kWh，对应净退费约 31.05 万元；相对于原费不退且另罚的口径 A，总费少约 29.16 万元，波动价滚动方案少约 30.81 万元。这里两种解释各自重新优化，费用差并非给同一轨迹简单换费率的结果。

两种口径的取舍最终取决于题意，而不是哪种算出来便宜；好在两种口径下日内更新都成立：口径 A 的问题 3 合计 1393.45 万元，仍比同一零时预报下不更新的 1401.04 万元低 7.59 万元（0.54%），只是幅度远小于口径 B 的 36.75 万元。附录 D 表66给出三个窗口的精确结果。

7、14、28 天窗口的费用约为 1348.65 万、1364.29 万、1373.80 万元，越短的窗口在这个报告期表现越好。一种解释是较早的误差对当前分布的代表性在下降；但窗口长短也改变了跨日库存，单凭这组试验还分不清主因。

这三个值构成参数敏感性范围，不是重复实验的误差条。主窗口维持事先选定的 14 天；若想自适应选窗，每次决策前都只能用已完成日期验证。

不同策略在统计期边界保有的储能库存略有差异。为进行辅助比较，可取统一参考价值 $v = 0.9 \times 0.4248 \approx 0.382$ 元/kWh，定义库存调整费用

$$C_{\mathrm{adj}} = C_{\mathrm{total}} + vE_{32,0} - vE_{365,144}. \tag{34}$$

该量仅作同一库存估值下的比较，不替代题目要求的实际现金费用。问题 2 报告期净消耗库存 7788.15 kWh，按原参考值修正需增加约 2978 元，占现金费用约 0.02%。另以附件 4 全期价格范围乘放电效率，取统一估值 $v \in [0.007, 1.614]$ 元/内部 kWh 作事后敏感性分析，问题 2 修正量约为 53 至 12572 元。该范围不是决策时使用的价格信息，也不是置信区间。

在整个估值范围内，四套主模型的费用次序保持为问题 3、问题 2、问题 4-3、问题 4-2，价格已知相对预测价格的优势也没有反转。该次序仅针对这里列出的比较。

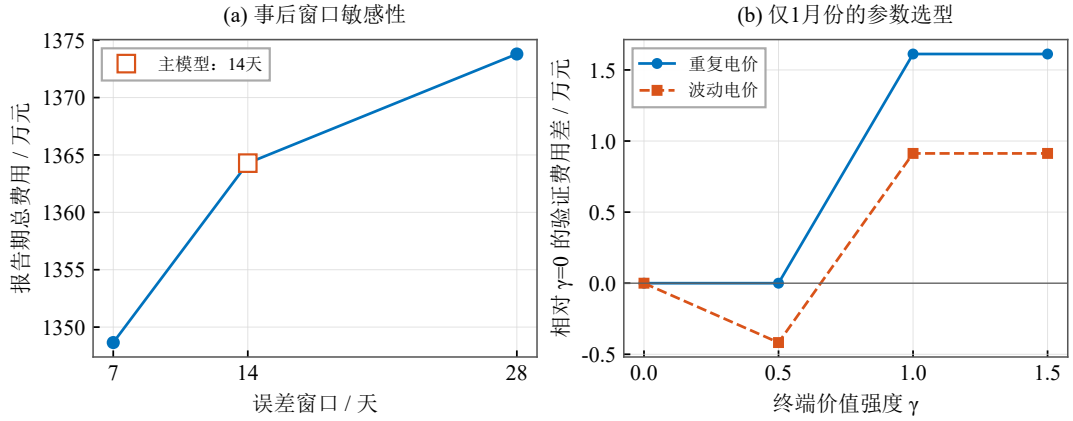


图 21　误差窗口的事后敏感性与终端价值的 1 月验证结果采用不同信息范围

9.3 历史信息与发布预报融合

发布预报在白天的优势较明显，历史预测对部分时段也有价值。我们用凸组合检验两类信息是否可以互补：

$$\widehat{P}^{(k),\mathrm{mix}}_{\mathrm{pv},d,t} = \alpha_k \widehat{P}^{(k)}_{\mathrm{pv},d,t} + (1-\alpha_k)\widehat{P}^{\mathrm{hist}}_{\mathrm{pv},d,t}. \tag{35}$$

对每个发布时刻，α_k 从 $\{0, 0.25, 0.5, 0.75, 1\}$ 中按 1 月 15–31 日剩余当日光伏预测的 MAE 选定，2 月起固定，不用报告期费用反选权重。零时的权重按全天误差来评，免得夜间误差占了大头、选出不合适的白天权重。1 月照旧用原发布预报热启动，融合从 2 月才开始用；算误差场景时，用各阶段实际采用的那套预测。

表 21　仅由 1 月 15–31 日剩余当日 MAE 确定的发布预报权重

发布时间	发布预报权重	历史预测权重	验证 MAE/kW
0 时	0.25	0.75	78.59
6 时	0.50	0.50	102.31
12 时	0.75	0.25	47.89
18 时	0.00	1.00	0.00

表 22　预报融合降低了总费，但紧急电量并未同步下降

价格与预报	计划费/万元	调整净费/万元	紧急量/kWh	总费用/万元
重复价，发布预报	1356.44	-14.55	43270.28	1364.29
重复价，融合预报	1329.23	-6.89	46172.34	1346.17
波动价，发布预报	1431.33	-15.20	43101.90	1440.37
波动价，融合预报	1402.47	-6.78	44752.90	1420.75

融合方案在重复电价和波动电价下分别少支出约 18.11 万元、19.62 万元，降幅约 1.33%、1.36%。但重复电价下的紧急电量由约 4.33 万 kWh 升到 4.62 万 kWh。总费下降与紧急量上升可以同时发生，选预测器时两个指标都得看。

融合结果单独报告，规定工作簿仍对应原发布预报主模型；一年的样本外结果尚不足以确认权重的跨年度稳定性。

9.4 费用驱动的预测选型

相同大小的预测误差，发生在不同价格和 SOC 条件下，费用代价可以相差很大。为检验误差选型是否合适，我们固定负荷预测、窗口和执行规则，仅更换光伏候选方法，按 1 月 15–31 日的实际费用选型，并以统一价格校正期末库存。两类指标在 1 月份的候选得分见附录 D 表65。

两种价格条件下，费用选型均选择 3 日均值，而 MAE 选型选择 7 日均值。候选在 1 月份逐日只使用之前的观测；选型完成后固定方法，报告期仅滚动更新历史输入。为隔离“选型指标”本身，MAE 对照同样使用全部候选的历史预测回放；两套策略共享原主模型 1 月末的实际 SOC，其他物理约束、场景窗口与结算规则一致。

表 23　相同历史回放下，费用选型未优于 MAE 选型（2–12 月）

选择规则	光伏方法	总费用/元	紧急电量/kWh
重复价：MAE 选型	7 日均值	13852291.37	85480.79
重复价：费用选型	3 日均值	13858729.92	80511.05
波动价：MAE 选型	7 日均值	14608269.34	83582.10
波动价：费用选型	3 日均值	14614183.70	78832.98

结果与预期并不一致。1 月份费用较低的 3 日均值，在报告期反而比同口径 MAE 选型多支出约 6439 元和 5914 元（分别对应两种价格）。短验证期的费用最优没有延续到全年，我们据此保留原主预测方案，不再用报告期挑选另一个赢家。

比较须使用表中的专设 MAE 对照：其历史回放从 1 月 15 日开始，与原主模型 1 月份的持久性预测缓存不同，直接对比会把缓存差异混入选型效果。

主模型与对照共 27 组年度配置，新增的 12 组轨迹均通过物理、费用与决策时序检查。各项改动的节省不可相加：终端价值、门控、费用选型与融合还没有做过组合验证。

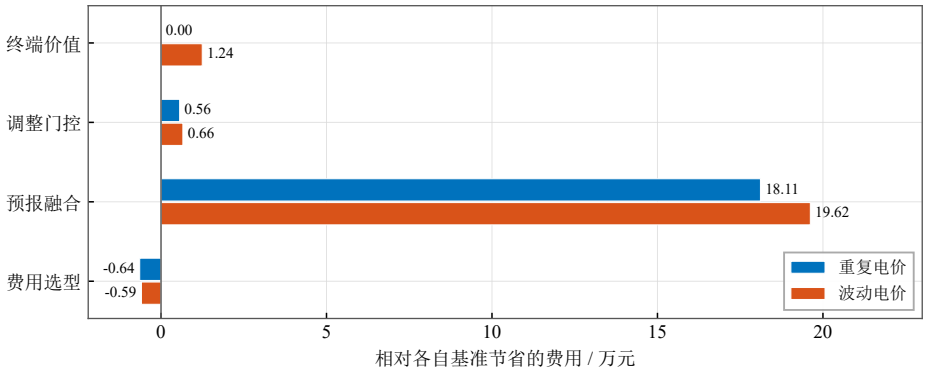


图 22　独立改动相对各自基准的费用变化：费用选型未获改善，各项收益不可相加

十、模型评价与推广

10.1 主要创新点

我们在建模、算法与评价三个层面都有区别于常规做法的处理。(1) 建模层面，用共同名义储能轨迹加情景缺额变量构成线性风险代理，把缺额的期望代价写成目标函数中的线性项，带不确定性的全年回测因此仍可用稀疏 LP 求解，不必引入场景相关的储能状态与整数变量。(2) 算法层面，给出保持储能状态与费用不变的同成本等价变换，证明主模型参数下总存在不同时充放电的最优解；同时以分段线性函数表达合同的三段结算，滚动调整问题保持线性。(3) 评价层面，所有改进都以同信息、同物理约束、同结算口径的基准对照呈现，把信息价值、算法收益与结算口径的影响分开计量；对代理目标与执行反馈的不一致还给出定量上界，而不是笼统断言模型有效。

10.2 主要结论

(1) 储能和日内预报更新都能降低电费，但幅度有限，也不是每个时段都成立：储能使问题 1 单日电费下降 26.90%；问题 3 的四次日内更新使全年费用降低 36.75 万元 (2.62%)，334 天里仍有 67 天更新后更贵；是否采纳某次更新要看评分，1002 次机会采纳了 612 次。

(2) 结论依赖结算口径与信息条件：取消费用改按“原费不退、另罚 50%”解释时，日内更新的节省从 36.75 万元缩到 7.59 万元，方向不变；当天价格从未知变为已知，全年费用也只再降 0.72%–0.84%。结算规则或信息发布时点一变，策略就要重算。

(3) 预测误差更小、紧急电量更少，都不等于总费用更低：融合试验里紧急电量上升而总费下降，费用选型里验证期更优的方法全年反而更贵。我们因此同时看误差指标与实际费用，不以单一指标替代调控目标。

10.3 模型优点

(1) 四问共用同一套功率、能量与交易口径，每次只改动信息条件或交易规则中的一项，费用变化能明确归因；可实施策略、当日先知、全期先知三种信息水平分开建模，节省不与信息优势混记。稀疏 LP 使全年逐日回测可行，轨迹可逐时复核。

(2) 把缺额按 5 倍电价补购的代价提前写进目标函数，让计划为预测误差留出余量：与点预测基准相比，问题 2 净节省约 213 万元，紧急电量由约 67.4 万 kWh 降到 8.55 万 kWh，是各问中收益最大的一处设计。

(3) 对容易歧义之处都做了对照计算：取消费用的两种解释、充放电效率的三种分配、风险权重与误差窗口敏感性、价格信息的两种可见性设定。主要结论在多种解释下方向一致，各表数字可复核、可复算。

10.4 模型不足

（1）场景缺额用一条共同的名义储能轨迹近似，实际电池却按观测反馈运行，代理目标与真实费用之间有系统差。以当日完全预知的执行为参照上界，报告期现金费用可由 1386.39 万元进一步降到 1380.16 万元，相差 6.23 万元（约 0.45%，见附录 D 表69）；差距不算大，但代理结构仍有改进空间，门控也保证不了真实天气下同样划算。

（2）电池循环寿命、温度与效率变化未进入模型，价格已知时等效循环升至 627 次，长期经济性无法判断；问题 2 常规购电峰值约 10203 kW，高于负荷峰值约 7979 kW，若实际并网容量受限，高负荷时段的方案需要重排。

（3）波动电价主方案没有日内价格预测器，日内调节因此受限；验证数据只有一年，终端奖励、短期费用选型、36 时段保留规则在本数据集上都没有收益，分不清是确实无效还是仅在本样本无效。

10.5 改进与推广

（1）先校正风险代理，再扩展时域：引入分场景储能状态与非预见性约束 [7]，检验代理目标与实际反馈费用的差距能否缩小，再把决策时域扩展到 24–48 小时，考察线性终端价值能替代多少跨日决策 [8]。

（2）获得电池循环寿命与温度资料后，把折旧成本计入目标，重新评价价格已知设定、融合方案等调节频繁策略的长期收益。

（3）方法与窗口按时间顺序、只用已完成日期重新验证，保留未参与选型的测试期；证据补齐之前，我们的结果适合作为可复核的调度基准，而不宜直接当作长期收益的承诺。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型讨论、代码生成与调试、数据核验、绘图及文字结构调整，详见支撑材料《AI 工具使用详情》。我们把辅助范围限定在上述环节，所有由 AI 参与的输出都在写入论文前经我们逐项人工复核；该文件中的每条数值主张，我们都能回溯到工作簿或结果数组。

十一、参考文献

我们的引用取自方法的原始文献、官方文档与所用求解器的正式出处，正文相应位置都给出了标注。我们只保留与建模、求解和检验直接相关的条目；凡是我们没有实际使用的方法，不列入凑数。

[1] Boyd S, Vandenberghe L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004.

[2] SciPy Developers. scipy.optimize.linprog: Linear programming[EB/OL]. [2026-09-10]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html.

[3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1):119–142. DOI:10.1007/s12532-017-0130-5.

[4] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021. [2026-09-10]. https://otexts.com/fpp3/.

[5] Birge J R, Louveaux F. Introduction to Stochastic Programming[M]. 2nd ed. New York: Springer, 2011. DOI:10.1007/978-1-4614-0237-4.

[6] Rawlings J B, Mayne D Q, Diehl M M. Model Predictive Control: Theory, Computation, and Design[M]. 2nd ed. Nob Hill Publishing, 2017.

[7] Shapiro A, Dentcheva D, Ruszczynski A. Lectures on Stochastic Programming: Modeling and Theory[M]. 3rd ed. Philadelphia: SIAM, 2021. DOI:10.1137/1.9781611976595.

[8] Bertsekas D P. Dynamic Programming and Optimal Control, Volume I[M]. 4th ed. Nashua: Athena Scientific, 2017.

附录 A　支撑材料与程序实现说明

A.1　运行环境与文件组织

计算结果来自所附程序的运行记录，环境为 Python 3.12.14、NumPy 2.5.3、SciPy 1.18.1，openpyxl 处理结果模板，Matplotlib 绘图，XeLaTeX 编译。论文保留 27 组主模型与对照结果，另用独立 LP 复算问题 1，并补充效率与风险权重试验。

绘图程序读取核对后的轨迹，生成 18 个论文图组及源数据清单；补充程序生成复核数据和 4 个新图组；verify_consistency.py 只读取已有工作簿、表格与结果数组，重算论文数字与工作簿的对应关系，不产生新结果。描述统计、图表和补充试验均不改写原主结果，具体脚本见下表。

文件或目录	内容
main.tex	完整论文源码
README.md	支撑材料总说明、目录结构与复现命令
code/model.py	数据读取、历史预测、场景构造、稀疏 LP、运行反馈与状态核验
code/run_experiments.py	单日求解、四套主模型、各类对照实验及计时
code/export_results.py	5 个规定工作簿、2 个价格已知补充表及保存后复核
code/make_evidence.py	由计算结果生成所有数值表格与矢量图形
code/check_model.py	功率、SOC、费用、边界及未来数据扰动检查
code/diagnostics.py	附件 3 预报误差、场景配对及储能反馈对照诊断
code/refinements.py	先知参照、终端价值、费用选型及门控实验与核验
code/refinement_evidence.py	新增实验表格与数值宏
code/polish_presentation.py	分析表标题与月度费用描述统计，不修改原结果
code/paper_figures.py	新版论文图组、源数据核验与绘图描述统计
code/emergency_surface.py	紧急购电月度小时分布的绘图程序（生成正文图10）
code/scientific_style.py	绘图公共样式与导出设置，被上述绘图脚本调用
code/requirements.txt	Python 依赖清单（含版本约束）
结果/result1.xlsx	单日计划购电量与储能分组电量
结果/result2.xlsx	日前计划、实际充放电及连续紧急事件
结果/result3.xlsx	增加最终生效的调整后常规购电工作表
结果/result4-2.xlsx, 结果/result4-3.xlsx	价格预测主设定下对应问题 2 与 3 的结果
code/review_*.py	风险权重、效率、能量、库存诊断、日内执行上界对照及配套图表
code/verify_consistency.py	独立核验：摘要与正文数值、指定时段表、五份规定工作簿与结果数组逐项对账
结果/补充/	当天价格已知条件的两份逐时工作簿
AI 工具使用详情.pdf, AI 工具使用详情.tex	本次 AI 辅助环节、采纳范围和人工核验状态（含可编辑源码）
results/*.json, daily_summary.csv	参数、费用分解、典型日与独立核验记录
tables/, figures/	论文直接调用的真实结果表及矢量图

A.2　运行流程与关键约束编码

先安装依赖，再依次执行以下命令。其中 DATA 表示包含附件 1 至 4 及附件 5 文件夹的原始附件目录；原始附件不修改，也不需要预先填入结果。命令应在论文工程目录运行。

```
python -m pip install -r code/requirements.txt
python code/run_experiments.py --data DATA
python code/export_results.py --data DATA
python code/check_model.py --data DATA
python code/diagnostics.py --data DATA
python code/make_evidence.py
python code/refinements.py --data DATA
python code/refinement_evidence.py
python code/polish_presentation.py
python code/paper_figures.py
python code/review_experiments.py
python code/review_figures.py
python code/emergency_surface.py
python code/review_oracle_execution.py
python code/verify_consistency.py
xelatex -interaction=nonstopmode main.tex
xelatex -interaction=nonstopmode main.tex
```

模型按购电、充电、放电、状态和场景缺额的顺序拼接变量。储能动态矩阵的状态部分为单位矩阵减去一阶下对角矩阵，充电与放电系数分别为 $-\eta_c\Delta t$ 和 $\Delta t/\eta_d$。场景功率约束采用重复单位矩阵组合，并将大于等于关系乘以负号转换为 `A_ub @ x <= b_ub`。问题 1 将日末状态上下界均置为 6000，紧急变量上下界均置为 0；其余问题释放日末边界并传递实际初值。

批处理逐日执行，仅在一次优化中构建当日剩余时域的稀疏矩阵，不构造全年大规模稠密矩阵。读取后的数据和运行轨迹保存在数组中，结束后统一导出。各方案的全年仿真耗时见正文表18；具体设备和库版本变化可能导致耗时以及等价最优解的个别时段分配略有变化。

A.3 工作簿口径与结果复现

各结果工作簿均保留题目规定的工作表名称。问题 3 及问题 4-3 采用取消部分仅付 50% 违约费的主口径 B；融合、终端价值、费用选型和门控实验单独成表，不混入主结果表，`results/refinements.json` 记录 12 组新增轨迹汇总、1 月份选型得分及全部日内采纳记录。计划购电量记录零时原始计划，调整购电量记录最终生效的常规购电量，两处“全天购电费”填的都是当日实际总费（计划费、偏差净费用与紧急费合计）。两列是同一笔费用的重复展示，相加没有意义。紧急购电量按连续 10 分钟事件合并，电量为组成时段之和；没有事件的日期标注“无紧急购电”并填 0。逐日费用分解见 `daily_summary.csv`，正文统一用总费用。

模板首个时段列为 0:10–0:20，我们采用 0:00–0:10；模板末列为次日 0:00–0:10，我们采用 23:50–24:00。输入值按区间末端标记对应，**只统一输出标签，不平移数值**。所有全天费用按同一个自然日计算。

充放电量按实际轨迹每 4 小时汇总，两端 SOC 是库存状态，不作时段求和。工作簿保留计算值、显示四位小数，指定表显示两位；正文按讨论尺度取约数，个别显示值直接相加可能出现末位舍入差，核对以底层数据为准。月度 ± 项为样本标准差，不代表求解误差或未来费用区间。

A.4 数值精度与描述统计口径

题目要求电量与费用表保留两位小数，便于逐项核对；工作簿内部存储未按显示精度截断的数值。正文讨论规模时改用约数、万 kWh 或万元，日期、次数与设备边界仍用准确值。表中金额保留到分只是核算习惯，不代表费用预测能精确到分。

预测误差用第 6.1 节的 MAE、RMSE 描述，运行差异用第 7.2.2 节的月度均值、标准差和范围描述；误差窗口实验给出参数敏感性。我们未得到重复观测下的费用置信区间，不给每项年度费用附加缺乏依据的 ± 值。

附录 B　核心可运行程序

我们按运行顺序完整列出求解、导出、诊断与绘图程序，源码与我们提交的支撑文件逐字节一致；若要复现我们的结果，按附录 A.2 的命令顺序执行即可。

B.1　数据、预测、优化与反馈

```python
"""C题：仅使用决策时刻可获得信息的预测、线性规划与执行反馈。"""
from pathlib import Path
from time import perf_counter
from datetime import datetime, timedelta
import numpy as np
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import eye, diags, hstack, vstack, csr_matrix

DT = 1 / 6
ETA = 0.9
EMIN, EMAX, EINIT, PMAX = 1200., 10800., 6000., 5000.
N = 144
METHODS = ('last', 'mean3', 'mean7', 'ewma14', 'weekday')


def read_rows(path, sheet=0):
    wb = load_workbook(path, read_only=True, data_only=True)
    ws = wb.worksheets[sheet] if isinstance(sheet, int) else wb[sheet]
    rows = list(ws.values)
    wb.close()
    return rows


def load_data(root):
    root = Path(root)
    r = read_rows(root / '附件1.xlsx')
    expected_dates = [(datetime(2025, 1, 1)+timedelta(days=d)).date() for d in range(365)
    ]
    def as_date(value):
        return value.date() if isinstance(value, datetime) else datetime.strptime(str(
    value), '%Y-%m-%d').date()
    def minutes(value):
        if hasattr(value, 'hour'):
            return value.hour*60+value.minute
        h, m = str(value).split('+')[0].split(':')[:2]
        return (int(h)*60+int(m)) % 1440
    expected_times = [(t*10) % 1440 for t in range(1, 145)]
    assert [minutes(row[0]) for row in r[1:]] == expected_times, 'attachment 1 time order
    '
    one = np.asarray([row[1:] for row in r[1:]], float)
    actual = []
    for sheet in ('小区负载', '光伏发电实际功率'):
        rows = read_rows(root / '附件2.xlsx', sheet)
        assert [as_date(row[0]) for row in rows[1:]] == expected_dates, sheet + ': date
    order'
        assert [minutes(v) for v in rows[0][1:145]] == expected_times, sheet + ': time
    order'
        actual.append(np.asarray([row[1:145] for row in rows[1:]], float))
    dates = [row[0].strftime('%Y-%m-%d') for row in rows[1:]]
    price_rows = read_rows(root/'附件4.xlsx')
    assert [as_date(row[0]) for row in price_rows[1:]] == expected_dates, 'price date
    order'
    assert [minutes(v) for v in price_rows[0][1:145]] == expected_times, 'price time
    order'
    price = np.asarray([row[1:145] for row in price_rows[1:]], float)
```

```
    issue_rows = read_rows(root/'附件3.xlsx')[1:]
    assert len(issue_rows) == 1460, 'issued forecast row count'
    for i, row in enumerate(issue_rows):
        if row[0] not in (None, ''):
            issue_date = as_date(row[0])
        assert issue_date == expected_dates[i//4], 'issued forecast date order'
        assert minutes(row[1]) == (i % 4)*360, 'forecast publication order'
    pv_hour = np.asarray([row[2:26] for row in issue_rows], float).reshape(365, 4, 24)
    data = dict(one=one, load=actual[0], pv=actual[1], price=price, pv_hour=pv_hour,
    dates=dates)
    for key in ('one', 'load', 'pv', 'price', 'pv_hour'):
        assert np.isfinite(data[key]).all(), key + ': missing/nonfinite data'
        assert (data[key] >= 0).all(), key + ': negative data'
    assert one.shape == (144, 3)
    assert all(data[k].shape == (365, 144) for k in ('load', 'pv', 'price'))
    assert len(set(dates)) == 365
    return data


def predict(history, fallback, method):
    k = len(history)
    if k == 0:
        return np.asarray(fallback).copy()
    if method == 'last':
        return history[-1].copy()
    if method == 'weekday':
        idx = [k-lag for lag in (7, 14, 21, 28) if k >= lag]
        if idx:
            return np.mean(history[idx], axis=0)
        return np.mean(history[-min(k, 7):], axis=0)
    if method.startswith('mean'):
        return np.mean(history[-min(k, int(method[4:])):], axis=0)
    block = history[-min(k, 14):]
    weights = np.exp(-np.arange(len(block)-1, -1, -1)/5)
    return np.average(block, weights=weights, axis=0)


def forecast_all(data):
    """1月15—31日选择方法；1月份实际热启动统一用持久性方法。"""
    forecasts, scores, selected = {}, {}, {}
    for key, fallback in [('load', data['one'][:, 1]), ('pv', data['one'][:, 2]), ('price
    ', data['one'][:, 0])]:
        values = data[key]
        scores[key] = {}
        for method in METHODS:
            errors = np.asarray([predict(values[:d], fallback, method)-values[d] for d in
     range(14, 31)])
            scores[key][method] = dict(mae=float(np.mean(abs(errors))), rmse=float(np.
    sqrt(np.mean(errors**2))))
        chosen = min(METHODS, key=lambda m: scores[key][m]['mae'])
        selected[key] = chosen
        forecasts[key] = np.asarray([predict(values[:d], fallback, 'last' if d < 31 else
    chosen) for d in range(365)])
    # 每次预测只连接已观测到的发布时刻端点和该次发布的未来24个整点预报。
    issued = np.zeros((365, 4, 144))
    for day in range(365):
        for issue in range(4):
            start = issue * 36
            current = data['pv'][day, start-1] if start else (data['pv'][day-1, -1] if
    day else 0.)
            x = (np.arange(start, 144)+1-start)/6
            issued[day, issue, start:] = np.interp(x, np.arange(25), np.r_[current, data[
    'pv_hour'][day, issue]])
```

```
    forecasts['issued'] = issued
    # 融合权重只在1月15—31日确定，2月起冻结；1月热启动仍用原始发布预报。
    historical_validation = np.asarray([predict(data['pv'][:d], data['one'][:,2],
    selected['pv']) for d in range(14,31)])
    blended = issued.copy()
    blend_scores, blend_weights = {}, []
    for issue in range(4):
        start = issue*36
        block = slice(start, 144)
        losses = {str(w): float(np.mean(abs(w*issued[14:31,issue,block]+(1-w)*
    historical_validation[:,block]-data['pv'][14:31,block])))
                  for w in (0., .25, .5, .75, 1.)}
        weight = float(min(losses, key=losses.get))
        blend_scores[str(issue*6)] = losses
        blend_weights.append(weight)
        blended[31:,issue,start:] = weight*issued[31:,issue,start:]+(1-weight)*forecasts[
    'pv'][31:,start:]
    forecasts['issued_blend'] = blended
    scores['pv_blend'] = blend_scores
    selected['pv_blend_weights'] = blend_weights
    return forecasts, scores, selected


def residual_scenarios(data, forecast, day, start, issue=None, history_days=14):
    """当前净负荷预测 + 最近14天在相同发布时刻产生的整条误差轨迹。"""
    pv_forecast = forecast['pv'] if issue is None else forecast['issued'][:, issue]
    nominal = forecast['load'][day, start:] - pv_forecast[day, start:]
    if history_days == 0 or day == 0:
        return nominal[None, :]
    left = max(0, day-history_days)
    errors = data['load'][left:day, start:]-data['pv'][left:day, start:] \
             -(forecast['load'][left:day, start:]-pv_forecast[left:day, start:])
    return np.vstack((nominal, nominal+errors))


def optimize(scenarios, prices, initial, base=None, settlement='refund', cyclic=False,
    eta=ETA, pmax=PMAX,
             terminal_value=0., final_energy=None):
    """稀疏LP。紧急量为场景中的缺额补救变量，实际费用由execute重新核算。"""
    start_time = perf_counter()
    if settlement not in ('refund', 'additive'):
        raise ValueError('Unknown settlement mode')
    s, n = scenarios.shape
    ident = eye(n, format='csr')
    zero = csr_matrix((n, n))
    delta = ident - diags(np.ones(n-1), -1, shape=(n, n), format='csr')
    extra = 2*n if base is not None else 0
    eq = hstack([zero, -eta*DT*ident, DT/eta*ident, delta, csr_matrix((n, s*n+extra))],
    format='csr')
    eq_rhs = np.zeros(n)
    eq_rhs[0] = initial
    tile = vstack([ident]*s, format='csr')
    ub = hstack([-tile, tile, -tile, csr_matrix((s*n,n)), -eye(s*n,format='csr'),
    csr_matrix((s*n,extra))],format='csr')
    c = np.r_[prices*DT if base is None else np.zeros(n), np.zeros(3*n), np.tile(5*prices
    *DT/s,s)]
    c[4*n-1] -= terminal_value  # 元/电池内部kWh；只用于规划，不计入实际电费。
    bounds = [(0,None)]*n + [(0,pmax)]*(2*n) + [(EMIN,EMAX)]*n + [(0,None)]*(s*n)
    if cyclic:
        bounds[4*n-1] = (initial,initial)
    if final_energy is not None:
        bounds[4*n-1] = (final_energy,final_energy)
    if base is not None:
```

```
        adjust_eq = hstack([ident,csr_matrix((n,3*n+s*n)),-ident,ident],format='csr')
        eq = vstack([eq,adjust_eq],format='csr')
        eq_rhs = np.r_[eq_rhs,base]
        c = np.r_[c,1.5*prices*DT,(0.5 if settlement=='additive' else -0.5)*prices*DT]
        bounds += [(0,None)]*(2*n)
    # 单日确定性问题必须由常规购电满足负荷，不使用场景紧急量。
    if cyclic:
        for idx in range(4*n,4*n+s*n):
            bounds[idx]=(0,0)
    result=linprog(c,A_ub=ub,b_ub=-scenarios.ravel(),A_eq=eq,b_eq=eq_rhs,bounds=bounds,
    method='highs')
    if not result.success:
        raise RuntimeError(result.message)
    g,ch,dis,energy=np.split(result.x[:4*n],4)
    # 精确消除同时充放电：状态增量不变，供给裕量增加，由弃电变量吸收。
    amount=np.minimum(ch,dis/eta**2)
    ch=ch-amount
    dis=dis-eta**2*amount
    assert np.max(np.minimum(ch,dis)) < 1e-6
    return dict(grid=g,ch=ch,dis=dis,energy=energy,objective=float(result.fun),seconds=
    perf_counter()-start_time,
                residual_eq=float(np.max(np.abs(eq@result.x-eq_rhs))),residual_ub=float(
    np.max(ub@result.x+scenarios.ravel())))


def execute(grid, load, pv, initial, eta=ETA, pmax=PMAX):
    """电网侧功率跟随控制，先用盈余充电；有缺额时放电，再紧急购电。"""
    n=len(grid)
    ch,dis,emergency,spill=np.zeros((4,n))
    energy=np.zeros(n+1)
    energy[0]=initial
    for t in range(n):
        residual=grid[t]+pv[t]-load[t]
        if residual >= 0:
            ch[t]=min(residual,pmax,max(0,(EMAX-energy[t])/(eta*DT)))
            spill[t]=residual-ch[t]
        else:
            dis[t]=min(-residual,pmax,max(0,(energy[t]-EMIN)*eta/DT))
            emergency[t]=-residual-dis[t]
        energy[t+1]=energy[t]+eta*DT*ch[t]-DT/eta*dis[t]
    return dict(ch=ch,dis=dis,emergency=emergency,spill=spill,energy=energy)


def execute_reserve(grid, load, pv, initial, forecast_load=None, forecast_pv=None,
                    prices=None, eta=ETA, pmax=PMAX, reserve_horizon=36):
    """固定常规购电计划下的可复现储能保留基准。

    反馈仍只使用当前实际负荷/光伏；为下一 ``reserve_horizon`` 个10分钟
    时段预留日前预测的缺额所需电量，避免把电池一次性放空。该规则不改动
    常规购电计划或结算，只改变实际充放电与紧急补电，因而可与 execute
    直接作同初值、同信息范围的连续回测。
    """
    n = len(grid)
    if forecast_load is None or forecast_pv is None:
        raise ValueError('Reserve feedback requires forecasts; future actual values are
    not a fallback')
    ch, dis, emergency, spill = np.zeros((4, n))
    energy = np.zeros(n + 1)
    energy[0] = initial
    for t in range(n):
        residual = grid[t] + pv[t] - load[t]
        if residual >= 0:
            ch[t] = min(residual, pmax, max(0, (EMAX-energy[t])/(eta*DT)))
```

```
            spill[t] = residual - ch[t]
        else:
            end = min(n, t + reserve_horizon + 1)
            expected_deficit = np.maximum(
                forecast_load[t+1:end] - forecast_pv[t+1:end] - grid[t+1:end], 0.0
            ).sum() * DT / eta
            reserve = min(EMAX-EMIN, expected_deficit)
            available = max(0, (energy[t] - EMIN - reserve) * eta / DT)
            dis[t] = min(-residual, pmax, available)
            emergency[t] = -residual - dis[t]
        energy[t+1] = energy[t] + eta*DT*ch[t] - DT/eta*dis[t]
    return dict(ch=ch, dis=dis, emergency=emergency, spill=spill, energy=energy)


def plan_at_issue(data, forecast, day, issue, state, base=None, kind=3,
                  price_mode='fixed', settlement='refund', history_days=14, eta=ETA):
    """同一决策入口用于实际回测与未发布信息扰动检验。"""
    start = issue*36
    prices = (data['one'][:,0] if price_mode=='fixed' else
              forecast['price'][day] if price_mode=='forecast' else data['price'][day])
    scenarios = residual_scenarios(data, forecast, day, start, None if kind==2 else issue
    , history_days)
    return optimize(scenarios, prices[start:], state, base, settlement, eta=eta)


def settlement_cost(plan, grid, emergency, prices, settlement='refund'):
    """原计划费、上调费、下调净退费、紧急费、总费；功率转换为时段电量。"""
    if settlement not in ('refund', 'additive'):
        raise ValueError('Unknown settlement mode')
    diff = grid-plan
    values = DT*np.array([np.dot(prices, plan),
                          np.dot(1.5*prices, np.maximum(diff,0)),
                          np.dot((-.5 if settlement=='refund' else .5)*prices, np.maximum
    (-diff,0)),
                          np.dot(5*prices, emergency)])
    return np.r_[values, values.sum()]


def run_case(data,forecast,kind=2,price_mode='fixed',updates=(0,6,12,18),settlement='
    refund',history_days=14,eta=ETA,pv_mode='issued'):
    begin=perf_counter()
    if pv_mode == 'blend':
        forecast = dict(forecast, issued=forecast['issued_blend'])
    elif pv_mode != 'issued':
        raise ValueError('Unknown PV forecast mode')
    arrays={key:np.zeros((365,144)) for key in ('plan','grid','ch','dis','emergency','
    spill')}
    arrays['energy']=np.zeros((365,145))
    costs=np.zeros((365,5))  # 计划、上调、下调、紧急、合计
    state=EINIT
    lp_seconds=0.; max_eq=0.; max_ub=0.; solves=0
    issue_numbers=[0] if kind==2 else [h//6 for h in updates]
    for day in range(365):
        realized_price=data['one'][:,0] if price_mode=='fixed' else data['price'][day]
        arrays['energy'][day,0]=state
        for i,issue in enumerate(issue_numbers):
            start=issue*36
            end=issue_numbers[i+1]*36 if i+1<len(issue_numbers) else 144
            base=arrays['plan'][day,start:] if start else None
            solution=plan_at_issue(data,forecast,day,issue,state,base,kind,price_mode,
    settlement,history_days,eta)
            lp_seconds+=solution['seconds']; solves+=1
```

```
            max_eq=max(max_eq,solution['residual_eq']);max_ub=max(max_ub,solution['residual_ub'])
            if start==0:
                arrays['plan'][day]=solution['grid']
            g=solution['grid'][:end-start]
            arrays['grid'][day,start:end]=g
            actual=execute(g,data['load'][day,start:end],data['pv'][day,start:end],state,eta=eta)
            for key in ('ch','dis','emergency','spill'):
                arrays[key][day,start:end]=actual[key]
            arrays['energy'][day,start:end+1]=actual['energy']
            state=actual['energy'][-1]
        costs[day]=settlement_cost(arrays['plan'][day],arrays['grid'][day],arrays['emergency'][day],realized_price,settlement)
    arrays['costs']=costs
    info=dict(kind=kind,price_mode=price_mode,updates=updates,settlement=settlement,history_days=history_days,eta=eta,pv_mode=pv_mode,
              seconds=perf_counter()-begin,lp_seconds=lp_seconds,solves=solves,max_eq_residual=max_eq,max_ub_violation=max_ub)
    audit(data,arrays,eta)
    return arrays,info


def audit(data,arrays,eta=ETA):
    g,ch,dis,em,spill=[arrays[k] for k in ('grid','ch','dis','emergency','spill')]
    e=arrays['energy']
    residual=g+data['pv']+dis+em-data['load']-ch-spill
    assert np.max(abs(residual))<1e-6
    assert np.min(e)>=EMIN-1e-6 and np.max(e)<=EMAX+1e-6
    assert np.max(np.minimum(ch,dis))<1e-6
    assert np.max(ch)<=PMAX+1e-6 and np.max(dis)<=PMAX+1e-6
    assert np.max(abs(np.diff(e,axis=1)-eta*DT*ch+DT/eta*dis))<1e-6
    assert np.max(abs(e[:-1,-1]-e[1:,0]))<1e-6
    assert abs(e[0,0]-EINIT)<1e-6
    assert (em>=-1e-6).all() and (g>=-1e-6).all()


def summarize(arrays,info):
    section=slice(31,365)
    c=arrays['costs'][section].sum(axis=0)
    out=dict(info)
    for name,val in zip(('plan_cost','up_cost','down_cost','emergency_cost','total_cost'),c):
        out[name]=float(val)
    for name in ('plan','grid','ch','dis','emergency','spill'):
        out[name+'_kwh']=float(arrays[name][section].sum()*DT)
    out['emergency_periods']=int(np.sum(arrays['emergency'][section]>1e-6))
    out['emergency_days']=int(np.sum(np.any(arrays['emergency'][section]>1e-6,axis=1)))
    out['start_energy']=float(arrays['energy'][31,0]);out['end_energy']=float(arrays['energy'][-1,-1])
    out['up_kwh']=float(np.maximum(arrays['grid'][section]-arrays['plan'][section],0).sum()*DT)
    out['down_kwh']=float(np.maximum(arrays['plan'][section]-arrays['grid'][section],0).sum()*DT)
    out['equivalent_cycles']=float((info['eta']*out['ch_kwh']+out['dis_kwh']/info['eta'])/(2*(EMAX-EMIN)))
    return out
```

B.2 实验驱动程序

```
"""运行全部模型与对照实验。输入原始附件目录，输出可核验的NPZ与JSON。"""
import argparse,json,sys,platform
from pathlib import Path
```

```
from time import perf_counter
import numpy as np
import scipy
from model import load_data,forecast_all,optimize,run_case,summarize,DT,EINIT

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--data',type=Path,required=True)
    parser.add_argument('--out',type=Path,default=Path(__file__).resolve().parents[1]/'results')
    parser.add_argument('--cases',nargs='*')
    args=parser.parse_args();args.out.mkdir(parents=True,exist_ok=True)
    start=perf_counter()
    data=load_data(args.data)
    forecast,scores,selected=forecast_all(data)
    np.savez_compressed(args.out/'inputs_processed.npz',**{k:v for k,v in data.items() if k!='dates'},**{'forecast_'+k:v for k,v in forecast.items()})
    metadata={'dates':data['dates'],'selected_forecasts':selected,'january_validation':scores,'python':platform.python_version(),'numpy':np.__version__,'scipy':scipy.__version__}
    metadata['forecast_feb_dec']={}
    for key in ('load','pv','price'):
        diff=forecast[key][31:]-data[key][31:]
        metadata['forecast_feb_dec'][key]={'mae':float(abs(diff).mean()),'rmse':float(np.sqrt((diff**2).mean())),'wape':float(abs(diff).sum()/data[key][31:].sum())}
    solution=optimize((data['one'][:,1]-data['one'][:,2])[None,:],data['one'][:,0],EINIT,cyclic=True)
    np.savez_compressed(args.out/'q1.npz',**solution,price=data['one'][:,0],load=data['one'][:,1],pv=data['one'][:,2])
    metadata['q1']={k:float(v) for k,v in solution.items() if np.isscalar(v)}
    metadata['q1'].update(grid_kwh=float(solution['grid'].sum()*DT),ch_kwh=float(solution['ch'].sum()*DT),dis_kwh=float(solution['dis'].sum()*DT),end_energy=float(solution['energy'][-1]),net_cost=float(np.maximum(data['one'][:,1]-data['one'][:,2],0)@data['one'][:,0]*DT))
    configs={
      'q2':{},
      'q2_nominal':{'history_days':0},
      'q3':{'kind':3},
      'q3_daily':{'kind':3,'updates':(0,)},
      'q3_noon':{'kind':3,'updates':(0,12)},
      'q3_additive':{'kind':3,'settlement':'additive'},
      'q4_2':{'price_mode':'forecast'},
      'q4_3':{'kind':3,'price_mode':'forecast'},
      'q4_2_known':{'price_mode':'known'},
      'q4_3_known':{'kind':3,'price_mode':'known'},
      'q4_3_additive':{'kind':3,'price_mode':'forecast','settlement':'additive'},
      'q3_history7':{'kind':3,'history_days':7},
      'q3_history28':{'kind':3,'history_days':28},
      'q3_blend':{'kind':3,'pv_mode':'blend'},
      'q4_3_blend':{'kind':3,'price_mode':'forecast','pv_mode':'blend'},
    }
    summaries={}
    existing=args.out/'summary.json'
    if args.cases and existing.exists():
        summaries=json.loads(existing.read_text(encoding='utf-8')).get('cases',{})
    if args.cases:
        unknown = set(args.cases)-set(configs)
        if unknown:
            raise ValueError('Unknown cases: '+', '.join(sorted(unknown)))
    for name,config in configs.items():
        if args.cases and name not in args.cases:
            continue
        arrays,info=run_case(data,forecast,**config)
```

```
        np.savez_compressed(args.out/(name+'.npz'),**arrays)
        summaries[name]=summarize(arrays,info)
        print(name,json.dumps(summaries[name],ensure_ascii=False),flush=True)
        metadata['cases']=summaries
        metadata['experiment_seconds']=perf_counter()-start
        existing.write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='
    utf-8')
    print('Completed',selected,flush=True)

if __name__=='__main__':
    if hasattr(sys.stdout,'reconfigure'):sys.stdout.reconfigure(encoding='utf-8')
    main()
```

B.3 结果工作簿导出

```
"""按附件5填充5份主结果和2份价格已知补充表；全天购电费包含紧急费。"""
import argparse,json,csv
from pathlib import Path
from copy import copy
from datetime import datetime
import numpy as np
from openpyxl import load_workbook
from openpyxl.styles import Alignment,Font
from model import DT,EINIT


def clock(t):
    minutes=t*10
    return f'{minutes//60}:{minutes%60:02d}'


def interval(t):
    return clock(t)+'-'+clock(t+1)


def events(power):
    out=[];start=None
    for t in range(145):
        active=t<144 and power[t]>1e-6
        if active and start is None:start=t
        if not active and start is not None:
            out.append((clock(start)+'-'+clock(t),float(power[start:t].sum()*DT)))
            start=None
    return out


def reset_body(ws):
    for merged in list(ws.merged_cells.ranges):ws.unmerge_cells(str(merged))
    styles=[copy(ws.cell(2,c)._style) for c in range(1,ws.max_column+1)]
    if ws.max_row>1:ws.delete_rows(2,ws.max_row-1)
    return styles


def append_row(ws,values,styles):
    ws.append(values)
    r=ws.max_row
    for c,v in enumerate(values,1):
        cell=ws.cell(r,c)
        if c<=len(styles):cell._style=copy(styles[c-1])
        if isinstance(v,(float,np.floating)):cell.number_format='0.0000'
        if isinstance(v,datetime):cell.number_format='yyyy/m/d'
        cell.alignment=Alignment(horizontal='center',vertical='center')
    ws.row_dimensions[r].height=20
```

```python


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--data',type=Path,required=True)
    parser.add_argument('--root',type=Path,default=Path(__file__).resolve().parents[1]);args=parser.parse_args()
    root=args.root;result=root/'results';dest=root/'结果';dest.mkdir(exist_ok=True)
    meta=json.loads((result/'summary.json').read_text(encoding='utf-8'));dates=meta['dates']
    q1=np.load(result/'q1.npz')
    wb=load_workbook(args.data/'附件5'/'result1.xlsx')
    ws=wb['计划购电量'];styles=reset_body(ws)
    for t in range(144):append_row(ws,[interval(t),float(q1['grid'][t]*DT)],styles)
    ws=wb['充放电量'];styles=reset_body(ws)
    for block in range(6):
        sl=slice(block*24,(block+1)*24)
        append_row(ws,[f'{block*4}:00-{(block+1)*4}:00',float(q1['ch'][sl].sum()*DT),float(q1['dis'][sl].sum()*DT),('0:00' if block==0 else '24:00' if block==1 else None),(EINIT if block==0 else float(q1['energy'][-1]) if block==1 else None)],styles)
    wb.save(dest/'result1.xlsx')
    daily=[];checks={}
    cases=[('q2','result2.xlsx'),('q3','result3.xlsx'),('q4_2','result4-2.xlsx'),('q4_3','result4-3.xlsx'),
           ('q4_2_known','result4-2.xlsx'),('q4_3_known','result4-3.xlsx')]
    supplement=dest/'补充';supplement.mkdir(exist_ok=True)
    for case,filename in cases:
        output=(supplement/('补充_价格已知_'+filename)) if case.endswith('_known') else dest/filename
        a=np.load(result/(case+'.npz'));wb=load_workbook(args.data/'附件5'/filename)
        for sheet,key in [('计划购电量','plan')]+([('调整购电量','grid')] if '调整购电量' in wb.sheetnames else []):
            ws=wb[sheet];styles=reset_body(ws)
            for t in range(144):ws.cell(1,t+2,interval(t))
            for d in range(31,365):
                # 两个购电工作表显示同一当日总费用，不能把两列再次相加。
                cost=a['costs'][d,4]
                append_row(ws,[datetime.fromisoformat(dates[d])]+(a[key][d]*DT).tolist()+[float(a[key][d].sum()*DT),float(cost)],styles)
            ws.freeze_panes='B2'
        ws=wb['充放电量'];styles=reset_body(ws)
        for d in range(31,365):
            for block in range(6):
                sl=slice(block*24,(block+1)*24)
                append_row(ws,[datetime.fromisoformat(dates[d]) if block==0 else None,f'{block*4}:00-{(block+1)*4}:00',float(a['ch'][d,sl].sum()*DT),float(a['dis'][d,sl].sum()*DT),('0:00' if block==0 else '24:00' if block==1 else None),(float(a['energy'][d,0]) if block==0 else float(a['energy'][d,-1]) if block==1 else None)],styles)
        ws.freeze_panes='C2'
        ws=wb['紧急购电量'];styles=reset_body(ws)
        for d in range(31,365):
            for i,(times,kwh) in enumerate(events(a['emergency'][d]) or [('无紧急购电',0.)]):
                append_row(ws,[datetime.fromisoformat(dates[d]) if i==0 else None,times,kwh],styles)
            daily.append([case,dates[d],*a['costs'][d].tolist(),float(a['plan'][d].sum()*DT),float(a['grid'][d].sum()*DT),float(a['emergency'][d].sum()*DT),float(a['energy'][d,0]),float(a['energy'][d,-1])])
        for ws in wb:
            ws.column_dimensions['A'].width=max(ws.column_dimensions['A'].width or 12,16)
            if ws.title=='紧急购电量':ws.column_dimensions['B'].width=24;ws.column_dimensions['C'].width=18
            if ws.title=='充放电量':
```

```
                for c in 'BCDEF':ws.column_dimensions[c].width=17
        wb.save(output)
        # 独立重新读取保存文件，逐日核对电量总和及费用列。
        check=load_workbook(output,read_only=True,data_only=True)
        for sheet,key in [('计划购电量','plan')]+([('调整购电量','grid')] if '调整购电量' in check.sheetnames else []):
            rows=list(check[sheet].values)
            assert list(rows[0][1:145])==[interval(t) for t in range(144)]
            assert [v[0].strftime('%Y-%m-%d') for v in rows[1:]]==dates[31:]
            matrix=np.asarray([row[1:] for row in rows[1:]],float)
            expected_cost=a['costs'][31:,4]
            assert np.max(abs(matrix[:,:144]-a[key][31:]*DT))<1e-6
            assert np.max(abs(matrix[:,:144].sum(axis=1)-matrix[:,144]))<1e-6
            assert np.max(abs(matrix[:,145]-expected_cost))<1e-6
        rows=list(check['充放电量'].values)[1:]
        for col,key in [(2,'ch'),(3,'dis')]:
            expected=a[key][31:].reshape(334,6,24).sum(axis=2).ravel()*DT
            assert np.max(abs(np.array([r[col] for r in rows])-expected))<1e-6
        assert np.max(abs(np.array([rows[d*6][5] for d in range(334)])-a['energy'][31:,0]))<1e-6
        assert np.max(abs(np.array([rows[d*6+1][5] for d in range(334)])-a['energy'][31:,-1]))<1e-6
        expected_events=[(times,kwh) for d in range(31,365) for times,kwh in (events(a['emergency'][d]) or [('无紧急购电',0.)])]
        stored_events=list(check['紧急购电量'].values)[1:]
        assert len(expected_events)==len(stored_events)
        for stored,(times,kwh) in zip(stored_events,expected_events):
            assert stored[1]==times and abs(stored[2]-kwh)<1e-6
        checks[str(output.relative_to(dest))]={'days':334,'quantity_cost_storage_events':'passed',
            'settlement':meta['cases'][case]['settlement'],'case':case,'total_cost':float(a['costs'][31:,4].sum())}
        check.close()
    with (result/'daily_summary.csv').open('w',newline='',encoding='utf-8-sig') as f:
        writer=csv.writer(f);writer.writerow(['case','date','plan_cost','up_cost','down_cost','emergency_cost','total_cost','plan_kwh','grid_kwh','emergency_kwh','start_energy','end_energy']);writer.writerows(daily)
    check=load_workbook(dest/'result1.xlsx',read_only=True,data_only=True)
    rows=list(check['计划购电量'].values)[1:]
    assert [r[0] for r in rows]==[interval(t) for t in range(144)]
    assert np.max(abs(np.array([r[1] for r in rows])-q1['grid']*DT))<1e-6
    rows=list(check['充放电量'].values)[1:]
    for col,key in [(1,'ch'),(2,'dis')]:
        assert np.max(abs(np.array([r[col] for r in rows])-q1[key].reshape(6,24).sum(axis=1)*DT))<1e-6
    assert abs(rows[0][4]-EINIT)<1e-6 and abs(rows[1][4]-q1['energy'][-1])<1e-6
    check.close();checks['result1.xlsx']={'quantity_storage':'passed'}
    (result/'workbook_verification.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8')
    print('Exported and reloaded 5 main and 2 supplementary workbooks under 结果/; quantities, total costs, storage and events passed.')

if __name__=='__main__':main()
```

B.4 独立核验程序

```
"""对关键物理约束、费用、时间因果性与导出数据做独立检查。"""
import argparse,json
from pathlib import Path
import numpy as np
from model import load_data,forecast_all,run_case,audit,execute,plan_at_issue,settlement_cost,EMIN,EMAX,PMAX,DT,ETA,EINIT

```

```
def main():
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,required=True);args=p.parse_args()
    root=Path(__file__).resolve().parents[1]/'results';data=load_data(args.data)
    meta=json.loads((root/'summary.json').read_text(encoding='utf-8'))
    metrics={}
    for name,info in meta['cases'].items():
        a=dict(np.load(root/(name+'.npz')));audit(data,a,info['eta'])
        prices=np.broadcast_to(data['one'][:,0],(365,144)) if info['price_mode']=='fixed' else data['price']
        diff=a['grid']-a['plan']
        # 使用“履行量+取消罚金+新增量”独立复算，避免只重复实现中的净退费表达。
        ordinary = (prices*np.minimum(a['plan'],a['grid'])+.5*prices*np.maximum(-diff,0)+1.5*prices*np.maximum(diff,0)) if info['settlement']=='refund' else (prices*a['plan']+.5*prices*np.maximum(-diff,0)+1.5*prices*np.maximum(diff,0))
        direct=(ordinary+5*prices*a['emergency']).sum(axis=1)*DT
        assert np.max(abs(direct-a['costs'][:,4]))<1e-6
        metrics[name]={'max_power_residual':float(abs(a['grid']+data['pv']+a['dis']+a['emergency']-data['load']-a['ch']-a['spill']).max()),'max_cost_residual':float(abs(direct-a['costs'][:,4]).max()),'min_energy':float(a['energy'].min()),'max_energy':float(a['energy'].max()),'simultaneous_periods':int((np.minimum(a['ch'],a['dis'])>1e-6).sum())}
    q=dict(np.load(root/'q1.npz'));energy=np.r_[EINIT,q['energy']]
    assert abs(energy[-1]-EINIT)<1e-6
    assert np.max(abs(np.diff(energy)-ETA*q['ch']*DT+q['dis']*DT/ETA))<1e-6
    assert np.min(q['grid']+q['pv']+q['dis']-q['load']-q['ch'])>=-1e-6
    assert abs(np.dot(q['price'],q['grid'])*DT-float(q['objective']))<1e-6
    # 未来实际数据扰动不应改变此前已发布的预测及方法选择。
    original,_,selected=forecast_all(data)
    changed={k:(v.copy() if isinstance(v,np.ndarray) else v) for k,v in data.items()}
    for key in ('load','pv','price'):changed[key][180:]*=1.5
    changed['pv_hour'][180:]*=.5
    altered,_,selected2=forecast_all(changed)
    assert selected==selected2
    for key in ('load','pv','price','issued'):assert np.array_equal(original[key][:180],altered[key][:180])
    # 进一步重跑日前决策：报告期后数据扰动不能改变此前已形成的计划。
    original_q2 = dict(np.load(root/'q2.npz'))
    altered_q2, _ = run_case(changed, altered, kind=2)
    assert np.array_equal(original_q2['plan'][:180], altered_q2['plan'][:180])
    assert np.array_equal(original_q2['grid'][:180], altered_q2['grid'][:180])
    # 改动当天未来实际量和尚未发布的预报，当前同一决策入口的解应不变。
    q3 = dict(np.load(root/'q3.npz')); day=100
    decision_checks=[]
    for issue in range(4):
        start=issue*36
        mutated={k:(v.copy() if isinstance(v,np.ndarray) else list(v)) for k,v in data.items()}
        for key in ('load','pv','price'):
            mutated[key][day,start:] *= 1.7
        mutated['pv_hour'][day,issue+1:] *= .2
        future_fc,_,future_selected=forecast_all(mutated)
        assert selected == future_selected
        base=q3['plan'][day,start:] if issue else None
        state=float(q3['energy'][day,start])
        for mode in ('issued','blend'):
            f0=original if mode=='issued' else dict(original,issued=original['issued_blend'])
            f1=future_fc if mode=='issued' else dict(future_fc,issued=future_fc['issued_blend'])
            before=plan_at_issue(data,f0,day,issue,state,base,price_mode='forecast')
            after=plan_at_issue(mutated,f1,day,issue,state,base,price_mode='forecast')
            delta=float(np.max(abs(before['grid']-after['grid'])))
```

```
            assert delta<1e-8
            decision_checks.append({'hour':issue*6,'pv_mode':mode,'max_grid_difference':delta})
    # 计划5kWh，下调为4、保持5、上调为6、全取消，应分别支付4.5、5、6.5、2.5元。
    for actual, expected in ((4.,4.5),(5.,5.),(6.,6.5),(0.,2.5)):
        bill=settlement_cost(np.array([5./DT]),np.array([actual/DT]),np.zeros(1),np.ones(1))
        assert abs(bill[-1]-expected)<1e-10
    # 缺电且SOC下限、盈余且SOC上限两种边界。
    low=execute(np.array([0.]),np.array([100.]),np.array([0.]),EMIN)
    high=execute(np.array([100.]),np.array([0.]),np.array([0.]),EMAX)
    assert low['emergency'][0]==100 and low['dis'][0]==0
    assert high['spill'][0]==100 and high['ch'][0]==0
    output={'cases':metrics,'causality_future_perturbation':'passed (forecasts and q2 decisions through day 180)',
            'unpublished_information_decisions':decision_checks,'settlement_examples':'passed: 4.5, 5, 6.5, 2.5 yuan',
            'input_date_time_publication_order':'passed','physical_boundary_checks':'passed','q1_checks':'passed'}
    (root/'verification.json').write_text(json.dumps(output,indent=2),encoding='utf-8')
    print(json.dumps(output,indent=2))
if __name__=='__main__':main()
```

B.5 图表生成程序

```
"""从计算结果自动生成论文数值宏、表格、图形和典型日分析数据。"""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from model import DT,EINIT,ETA,EMAX,EMIN
from export_results import events, interval

ROOT=Path(__file__).resolve().parents[1]
R=ROOT/'results';T=ROOT/'tables';F=ROOT/'figures'
T.mkdir(exist_ok=True);F.mkdir(exist_ok=True)
S=json.loads((R/'summary.json').read_text(encoding='utf-8'))
D=json.loads((R/'diagnostics.json').read_text(encoding='utf-8'))
CASES={k:dict(np.load(R/(k+'.npz'))) for k in S['cases']}
Q=dict(np.load(R/'q1.npz'));I=dict(np.load(R/'inputs_processed.npz'))
DATES=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
INDEX=[S['dates'].index(d) for d in DATES]
LABELS={'q2':'问题2','q3':'问题3','q4_2':'问题4-2','q4_3':'问题4-3','q2_nominal':'点预测基准',
    'q3_daily':'仅0时','q3_noon':'0、12时','q3_additive':'不退原费对照','q4_2_known':'价格已知：日前','q4_3_known':'价格已知：滚动','q4_3_additive':'波动价不退原费','q3_history7':'7天窗口','q3_history28':'28天窗口','q3_blend':'固定价预报融合','q4_3_blend':'波动价预报融合'}
def num(x):return f'{float(x):.2f}'
def save_table(name,caption,headers,rows,label=None):
    text='\\begin{table}[H]\n\\centering\\small\n\\caption{'+caption+'}\n'
    if label:text+='\\label{'+label+'}\n'
    text+='\\begin{tabular}{'+'l'+'r'*(len(headers)-1)+'}\\toprule\n'
    text+=' & '.join(headers)+' \\\\\\midrule\n'
    text+='\n'.join(' & '.join(map(str,row))+' \\\\' for row in rows)
    text+='\n\\bottomrule\\end{tabular}\n\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

def purchase(name,caption,g,cost):
    hours=[10,12,14,16,18,20]
    rows=[]
    for line in range(2):
```

```
        row=[]
        for h in hours[line*3:line*3+3]:row += [f'{h}:00--{h}:10',num(g[h*6]*DT)]
        rows.append(row)
    text='\\begin{table}[H]\\centering\\small\n\\caption{'+caption+'}\n\\label{tab:'+name
    +'}\n\\begin{tabular}{lrlrlr}\\toprule\n'
    text+='时段 & 购电量 & 时段 & 购电量 & 时段 & 购电量 \\\\\\midrule\n'
    text+='\n'.join(' & '.join(row)+' \\\\' for row in rows)
    text+='\n\\midrule\n\\multicolumn{3}{l}{全天常规购电量: '+num(g.sum()*DT)+' kWh} & \\
    multicolumn{3}{r}{全天总费用: '+num(cost)+' 元}\\\\\n\\bottomrule\\end{tabular}\\end{
    table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

def storage(name,caption,ch,dis,e0,e1):
    text='\\begin{table}[H]\\centering\\small\n\\caption{'+caption+'}\n\\label{tab:'+name
    +'}\n\\begin{tabular}{lrrlrr}\\toprule\n时段 & 充电量 & 放电量 & 时段 & 充电量 & 放电量
    \\\\\\midrule\n'
    for pair in range(3):
        row=[]
        for b in [pair*2,pair*2+1]:
            sl=slice(b*24,(b+1)*24)
            row += [f'{b*4}:00--{(b+1)*4}:00',num(ch[sl].sum()*DT),num(dis[sl].sum()*DT)]
        text+=' & '.join(row)+' \\\\\n'
    text+='\\midrule\\multicolumn{3}{l}{0:00储电量: '+num(e0)+'} & \\multicolumn{3}{r
    }{24:00储电量: '+num(e1)+'}\\\\\n\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

purchase('q1_purchase','问题1指定时段常规购电量（电量单位: kWh）',Q['grid'],Q['objective'])
storage('q1_storage','问题1储能充放电量与边界电量（单位: kWh）',Q['ch'],Q['dis'],EINIT,Q['
    energy'][-1])
typical={}
for key in ['q2','q3','q4_2','q4_3']:
    a=CASES[key];typical[key]=[]
    for day,idx in zip(DATES,INDEX):
        token=day[5:].replace('-','')
        purchase(key+'_'+token+'_purchase',LABELS[key]+': '+day+'指定时段常规购电量（kWh）',a
    ['grid'][idx],a['costs'][idx,4])
        storage(key+'_'+token+'_storage',LABELS[key]+': '+day+'实际充放电量（kWh）',a['ch'][
    idx],a['dis'][idx],a['energy'][idx,0],a['energy'][idx,-1])
        typical[key].append(dict(date=day,plan=float(a['plan'][idx].sum()*DT),grid=float(
    a['grid'][idx].sum()*DT),emergency=float(a['emergency'][idx].sum()*DT),cost=a['costs'
    ][idx].tolist(),start=float(a['energy'][idx,0]),end=float(a['energy'][idx,-1]),events
    =events(a['emergency'][idx])))
    groups=[events(a['emergency'][idx]) or [('无',0.)] for idx in INDEX]
    text='\\begin{table}[H]\\centering\\scriptsize\n\\caption{'+LABELS[key]+'四个指定日期的
    连续紧急购电事件（kWh）}\\label{tab:'+key+'_emergency}\n\\begin{tabular}{lrlrlrlr}\\
    toprule\n'
    text+=' & '.join('\\multicolumn{2}{c}{'+d[5:]+'}' for d in DATES)+' \\\\\n'
    text+=' & '.join(['时段 & 电量']*4)+' \\\\\\midrule\n'
    for r in range(max(map(len,groups))):
        row=[]
        for g in groups:row += [g[r][0].replace('-','--'),num(g[r][1])] if r<len(g) else
    ['--','--']
        text+=' & '.join(row)+' \\\\\n'
    text+='\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(key+'_emergency.tex')).write_text(text,encoding='utf-8')
    save_table(key+'_daily_summary',LABELS[key]+'典型日购电与费用汇总',['日期','原计划/kWh','调
    整后/kWh','紧急/kWh','总费用/元'],[[r['date'][5:],num(r['plan']),num(r['grid']),num(r['
    emergency']),num(r['cost'][4])] for r in typical[key]],'tab:'+key+'_daily_summary')

# 指定时段同时展示零时计划、最终生效量及差额，避免把调整后数量误作原计划。
idx = INDEX[0]; a = CASES['q3']; ts = [10*6,12*6,14*6,16*6,18*6,20*6]
save_table('q3_plan_adjust', '问题3 2025年3月20日指定时段计划与调整对照（kWh）',
           ['时段','零时计划','最终生效','差额'],
```

```
            [[interval(t).replace('-','--'), num(a['plan'][idx,t]*DT), num(a['grid'][idx,t]*DT), num((a['grid'][idx,t]-a['plan'][idx,t])*DT)] for t in ts],
            'tab:q3_plan_adjust')

save_table('annual','2--12月主模型费用分解（单位：万元）',['模型','计划费','上调费','下调净费','紧急费','总费用'],[[LABELS[k]]+[num(S['cases'][k][field]/10000) for field in ['plan_cost','up_cost','down_cost','emergency_cost','total_cost']] for k in ['q2','q3','q4_2','q4_3']],'tab:annual')
save_table('risk_compare','问题2是否纳入历史误差风险的比较',['方案','计划电量/万kWh','紧急电量/kWh','紧急天数','总费用/万元'],[[LABELS[k],num(S['cases'][k]['plan_kwh']/1e4),num(S['cases'][k]['emergency_kwh']),S['cases'][k]['emergency_days'],num(S['cases'][k]['total_cost']/1e4)] for k in ['q2_nominal','q2']],'tab:risk')
save_table('frequency','问题3不同预报更新频率的比较',['预报时刻','上调费/元','下调净费/元','紧急电量/kWh','总费用/元'],[[('0、6、12、18时' if k=='q3' else LABELS[k]),num(S['cases'][k]['up_cost']),num(S['cases'][k]['down_cost']),num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost'])] for k in ['q3_daily','q3_noon','q3']],'tab:frequency')
save_table('settlement','两种结算口径的对照比较',['价格条件与口径','下调电量/kWh','下调净费/元','总费用/元'],[[label,num(S['cases'][k]['down_kwh']),num(S['cases'][k]['down_cost']),num(S['cases'][k]['total_cost'])] for k,label in [('q3','重复日内价，主口径'),('q3_additive','重复日内价，不退原费'),('q4_3','波动电价，主口径'),('q4_3_additive','波动电价，不退原费')]],'tab:settlement')
save_table('price_information','波动电价下价格信息的作用',['模型与信息','计划费/万元','紧急电量/kWh','总费用/万元','等效循环'],[[label,num(S['cases'][k]['plan_cost']/1e4),num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost']/1e4),num(S['cases'][k]['equivalent_cycles'])] for k,label in [('q4_2','日前，价格预测'),('q4_2_known','日前，价格已知'),('q4_3','滚动，价格预测'),('q4_3_known','滚动，价格已知')]],'tab:price_information')
save_table('window','问题3历史误差窗口的敏感性比较',['历史窗口/天','紧急电量/kWh','总费用/元','2月初SOC/kWh','年底SOC/kWh'],[[w,num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost']),num(S['cases'][k]['start_energy']),num(S['cases'][k]['end_energy'])] for w,k in [(7,'q3_history7'),(14,'q3'),(28,'q3_history28')]],'tab:window')
save_table('issued_forecast','同一目标区间的光伏预报误差比较（2--12月，kW）',['目标区间','历史MAE','0时预报MAE','本次预报MAE','本次RMSE'],[[f'{k}--{int(k)+6}时',num(v['historical_same_block']['mae']),num(v['zero_same_block']['mae']),num(v['six_hour']['mae']),num(v['six_hour']['rmse'])] for k,v in D['issued_pv'].items() if k.isdigit()],'tab:issued_forecast')
save_table('update_robustness','问题3更新相对仅0时的月度差异（差值均为仅0时减四次）',['月份','费用差/元','紧急电量差/kWh'],[[m,num(v),num(D['update_emergency_by_month'][m])] for m,v in D['update_cost_by_month'].items()],'tab:update_robustness')
fb=D['feedback_benchmark']
save_table('feedback_benchmark','固定购电计划的储能反馈对照（2--12月）',['反馈规则','总费用/万元','紧急量/MWh','年底SOC/kWh'],[['即时放电',num(fb['base_total_cost']/1e4),num(fb['base_emergency_kwh']/1e3),num(fb['base_end_soc'])],['36时段保留',num(fb['reserve_total_cost']/1e4),num(fb['reserve_emergency_kwh']/1e3),num(fb['reserve_end_soc'])]],'tab:feedback_benchmark')
save_table('scenario_diagnostic','场景时间配对诊断：2025年9月23日',['指标','原始轨迹','逐时打乱','差值'],[["LP代理目标/元",num(D['scenario_shuffle']['original_objective']),num(D['scenario_shuffle']['permuted_objective']),num(D['scenario_shuffle']['objective_difference'])],["固定计划实际反馈费/元",num(D['scenario_shuffle']['original_feedback_cost']),num(D['scenario_shuffle']['permuted_feedback_cost']),num(D['scenario_shuffle']['permuted_feedback_cost']-D['scenario_shuffle']['original_feedback_cost'])]],'tab:scenario_diagnostic')
save_table('runtime','主模型的实测运行时间（含全年初始化）',['模型','LP次数','LP耗时/s','全年仿真/s'],[[LABELS[k],S['cases'][k]['solves'],num(S['cases'][k]['lp_seconds']),num(S['cases'][k]['seconds'])] for k in ['q2','q3','q4_2','q4_3']],'tab:runtime')
save_table('forecast','历史预测误差（1月验证，2--12月测试）',['变量','选定方法','验证MAE','测试MAE','测试RMSE','测试WAPE'],[[label,method,num(S['january_validation'][k][S['selected_forecasts'][k]]['mae']),num(D['historical'][k]['mae']) if k!='price' else f"{D['historical'][k]['mae']:.4f}",num(D['historical'][k]['rmse']) if k!='price' else f"{D['historical'][k]['rmse']:.4f}",f"{100*D['historical'][k]['wape']:.2f}\\%"] for k,label,method in [('load','负载/kW','同星期均值'),('pv','光伏/kW','7日移动均值'),('price','电价/(元/kWh)','同星期均值')]],'tab:forecast')
save_table('blend_weights','仅由1月15--31日剩余当日MAE确定的发布预报权重',
           ['发布时间','发布预报权重','历史预测权重','验证MAE/kW'],
```

```
            [[f'{i*6}时',num(w),num(1-w),num(S['january_validation']['pv_blend'][str(i*6)][str(w)])]
             for i,w in enumerate(S['selected_forecasts']['pv_blend_weights'])],'tab:blend_weights')
save_table('blend_results','预报融合的样本外费用对照（2--12月，主结算口径）',
           ['价格与预报','计划费/万元','调整净费/万元','紧急量/kWh','总费用/万元'],
           [[label,num(S['cases'][k]['plan_cost']/1e4),num((S['cases'][k]['up_cost']+S['cases'][k]['down_cost'])/1e4),num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost']/1e4)]
            for k,label in [('q3','重复价，发布预报'),('q3_blend','重复价，融合预报'),('q4_3','波动价，发布预报'),('q4_3_blend','波动价，融合预报')]],'tab:blend_results')

font_candidates = [
    Path('C:/Windows/Fonts/msyh.ttc'),
    Path('/tmp/.TinyTeX/texmf-dist/fonts/opentype/public/fandol/FandolHei-Regular.otf'),
    Path('/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc'),
    Path('/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc'),
]
font_path = next((p for p in font_candidates if p.exists()), None)
if font_path is not None:
    font_manager.fontManager.addfont(str(font_path))
    plot_font = font_manager.FontProperties(fname=str(font_path)).get_name()
else:
    plot_font = 'DejaVu Sans'
plt.rcParams.update({'font.family':plot_font,'font.size':9,'axes.unicode_minus':False,'pdf.fonttype':42,'axes.spines.top':False,'axes.spines.right':False,'axes.grid':True,'grid.alpha':.18,'savefig.bbox':'tight'})
BLUE='#245e8c';ORANGE='#c57639';GREEN='#37776b';GRAY='#6a727b'
hours=np.arange(144)/6
fig,ax=plt.subplots(3,1,figsize=(7.0,6.4),sharex=True,layout='constrained')
ax[0].plot(hours,Q['load']/1000,label='负载',color=GRAY);ax[0].plot(hours,Q['pv']/1000,label='光伏',color=ORANGE);ax[0].step(hours,Q['grid']/1000,label='计划购电',color=BLUE,where='post');ax[0].set_ylabel('功率 / MW');ax[0].legend(ncol=3,loc='upper center',fontsize=8)
ax[1].step(hours,Q['ch']/1000,color=GREEN,label='充电',where='post');ax[1].step(hours,-Q['dis']/1000,color=ORANGE,label='放电（负向）',where='post');ax[1].set_ylabel('功率 / MW');ax[1].legend(ncol=2,fontsize=8)
ax[2].plot(np.arange(145)/6,np.r_[EINIT,Q['energy']]/1000,color=BLUE,label='储电量');ax[2].axhline(1.2,color=GRAY,ls='--',lw=.8);ax[2].axhline(10.8,color=GRAY,ls='--',lw=.8);ax[2].set_ylabel('储电量 / MWh');ax[2].set_xlabel('时刻 / h');right=ax[2].twinx();right.plot(hours,Q['price'],color=ORANGE,alpha=.7,label='电价');right.set_ylabel('电价 / (元/kWh)');ax[2].set_xlim(0,24);ax[2].set_xticks(np.arange(0,25,4));fig.savefig(F/'q1_dispatch.pdf');plt.close(fig)

fig,ax=plt.subplots(2,1,figsize=(7,4.8),sharex=True,layout='constrained')
months=np.array([int(d[5:7]) for d in S['dates']]);monthly={}
for k,color in [('q2',BLUE),('q3',GREEN)]:
    em=[CASES[k]['emergency'][months==m].sum()*DT/1000 for m in range(2,13)]
    cost=[CASES[k]['costs'][months==m,4].sum()/10000 for m in range(2,13)]
    ax[0].plot(range(2,13),em,'o-',color=color,label=LABELS[k]);ax[1].plot(range(2,13),cost,'o-',color=color,label=LABELS[k]);monthly[k]={'emergency_mwh':em,'cost_10k':cost}
ax[0].set_ylabel('紧急购电量 / MWh');ax[0].legend(ncol=2);ax[1].set_ylabel('总费用 / 万元');ax[1].set_xlabel('月份');ax[1].set_xticks(range(2,13));fig.savefig(F/'monthly_comparison.pdf');plt.close(fig)
fig, ax = plt.subplots(figsize=(7, 3.2), layout='constrained')
months_report = np.arange(2, 13)
diff_month = np.array([D['update_cost_by_month'][str(m)] for m in months_report]) / 10000
ax.axhline(0, color=GRAY, lw=.8)
ax.bar(months_report, diff_month, color=[GREEN if x >= 0 else ORANGE for x in diff_month], width=.65)
ax.set_xlabel('月份'); ax.set_ylabel('费用差 / 万元\n（仅0时 - 四次更新）'); ax.set_xticks(months_report)
```

```
fig.savefig(F/'update_robustness.pdf'); plt.close(fig)

idx=INDEX[0];a=CASES['q3'];fig,ax=plt.subplots(2,1,figsize=(7,4.5),sharex=True,layout='constrained')
ax[0].step(hours,a['plan'][idx]/1000,label='0时计划',color=GRAY,where='post');ax[0].step(hours,a['grid'][idx]/1000,label='最终常规购电',color=BLUE,where='post');ax[0].plot(hours,a['emergency'][idx]/1000,label='紧急购电',color=ORANGE);ax[0].set_ylabel('购电功率 / MW');ax[0].legend(ncol=3,fontsize=8)
ax[1].plot(np.arange(145)/6,a['energy'][idx]/1000,color=GREEN,label='实际储电量');ax[1].set_ylabel('储电量 / MWh');ax[1].set_xlabel('时刻 / h');ax[1].legend();ax[1].set_xticks(np.arange(0,25,6));ax[1].set_xlim(0,24)
for axis in ax:
    for x in [6,12,18]:axis.axvline(x,color=GRAY,lw=.7,ls='--')
fig.savefig(F/'q3_typical.pdf');plt.close(fig)

idx=INDEX[1];fig,ax=plt.subplots(2,1,figsize=(7,4.5),sharex=True,layout='constrained')
ax[0].plot(hours,I['price'][idx],color=ORANGE,label='实际波动电价');ax[0].plot(hours,I['forecast_price'][idx],color=BLUE,label='日前价格预测');ax[0].plot(hours,Q['price'],color=GRAY,ls='--',label='重复日内电价');ax[0].set_ylabel('电价 / (元/kWh)');ax[0].legend(ncol=3,fontsize=8)
for k,color in [('q4_3',BLUE),('q4_3_known',GREEN)]:ax[1].plot(np.arange(145)/6,CASES[k]['energy'][idx]/1000,color=color,label='价格预测' if k=='q4_3' else '价格已知')
ax[1].set_ylabel('储电量 / MWh');ax[1].set_xlabel('时刻 / h');ax[1].set_xlim(0,24);ax[1].set_xticks(np.arange(0,25,4));ax[1].legend(ncol=2);fig.savefig(F/'q4_price.pdf');plt.close(fig)

fig,ax=plt.subplots(figsize=(7,3.5),layout='constrained');keys=['q2','q3','q4_2','q4_3'];x=np.arange(4);bottom=np.zeros(4)
components=[('实际履行原计划费',BLUE,lambda x:x['plan_cost']+2*x['down_cost']),
            ('取消违约费',GRAY,lambda x:-x['down_cost']),
            ('上调购电费',ORANGE,lambda x:x['up_cost']),
            ('紧急费',GREEN,lambda x:x['emergency_cost'])]
for label,color,compute in components:
    val=np.array([compute(S['cases'][k])/10000 for k in keys]);ax.bar(x,val,bottom=bottom,label=label,color=color,width=.55);bottom+=val
assert np.allclose(bottom,[S['cases'][k]['total_cost']/1e4 for k in keys])
for xx,yy in zip(x,bottom):ax.text(xx,yy+8,f'{yy:.2f}',ha='center',fontsize=9)
ax.set_xticks(x,[LABELS[k] for k in keys]);ax.set_ylabel('费用 / 万元');ax.set_ylim(0,max(bottom)*1.18);ax.legend(ncol=2,loc='upper left');fig.savefig(F/'annual_costs.pdf');plt.close(fig)

metrics={'typical':typical,'monthly':monthly,'q1_saving_percent':100*(1-float(Q['objective'])/S['q1']['net_cost']),'q2_risk_saving_percent':100*(1-S['cases']['q2']['total_cost']/S['cases']['q2_nominal']['total_cost']),'q3_frequency_saving_percent':100*(1-S['cases']['q3']['total_cost']/S['cases']['q3_daily']['total_cost']),'q3_emergency_reduction_percent':100*(1-S['cases']['q3']['emergency_kwh']/S['cases']['q3_daily']['emergency_kwh'])}
(R/'analysis_metrics.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding='utf-8')
cases=S['cases']
derived={'case_count':len(cases)}
for token,base,other in [('frequency','q3_daily','q3'),('noon','q3_noon','q3'),('q3_vs_q2','q2','q3'),
                         ('price2','q4_2','q4_2_known'),('price3','q4_3','q4_3_known'),
                         ('settlement3','q3_additive','q3'),('settlement4','q4_3_additive','q4_3'),
                         ('blend3','q3','q3_blend'),('blend4','q4_3','q4_3_blend')]:
    derived[token+'_saving']=cases[base]['total_cost']-cases[other]['total_cost']
    derived[token+'_percent']=100*derived[token+'_saving']/cases[base]['total_cost']
derived['frequency_em_percent']=100*(1-cases['q3']['emergency_kwh']/cases['q3_daily']['emergency_kwh'])
derived.update(D['update_days'])
for k in ('q3','q4_3'):
```

```
    for row,idx in zip(typical[k],INDEX):
        token=k+'d'+row['date'][5:].replace('-','')
        delta=CASES[k]['grid'][idx]-CASES[k]['plan'][idx]
        cases[token]={'plan':row['plan'],'grid':row['grid'],'emergency':row['emergency'],
                      'total':row['cost'][4],'up_cost':row['cost'][1],'down_cost':row['cost'][2],
                      'up':float(np.maximum(delta,0).sum()*DT),'down':float(np.maximum(-delta,0).sum()*DT)}
groups=dict(cases,derived=derived)
macro_lines=['% Automatically generated; do not edit numerical values by hand.']
for group,values in groups.items():
    for field,value in values.items():
        if isinstance(value,(int,float)):
            rendered=str(value) if isinstance(value,int) else num(value)
            macro_lines.append('\\expandafter\\def\\csname result'+group+field+'\\endcsname{'+rendered+'}')
(T/'metrics.tex').write_text('\n'.join(macro_lines)+'\n',encoding='utf-8')
print(json.dumps({'q1':S['q1'],'metrics':{k:v for k,v in metrics.items() if k not in ['typical','monthly']},'typical':typical},ensure_ascii=False))
```

B.6 补充诊断程序

```
"""补充诊断：预报误差、场景配对和储能反馈保留规则。"""
import json
from pathlib import Path
import numpy as np

from model import (DT, EINIT, ETA, load_data, forecast_all, residual_scenarios,
                   optimize, execute, execute_reserve)


def metrics(error, truth):
    error = np.asarray(error, float); truth = np.asarray(truth, float)
    return {
        'mae': float(np.mean(np.abs(error))),
        'rmse': float(np.sqrt(np.mean(error**2))),
        'wape': float(np.sum(np.abs(error)) / max(np.sum(np.abs(truth)), 1e-12)),
    }


def run(data_root, out_root):
    out_root = Path(out_root); out_root.mkdir(parents=True, exist_ok=True)
    data = load_data(data_root); fc, _, selected = forecast_all(data)
    report = {'selected_forecasts': selected, 'historical': {}, 'issued_pv': {},
              'scenario_shuffle': {}, 'feedback_benchmark': {}}
    sl = slice(31, 365)
    for key in ('load', 'pv', 'price'):
        report['historical'][key] = metrics(fc[key][sl] - data[key][sl], data[key][sl])
    # 同一目标6小时块和从发布时间起的剩余全天，均按论文插值方法评价。
    for issue in range(4):
        start = issue * 36
        block = slice(start, min(start + 36, 144))
        rest = slice(start, 144)
        truth_block, truth_rest = data['pv'][sl, block], data['pv'][sl, rest]
        pred = fc['issued'][sl, issue]
        report['issued_pv'][str(issue * 6)] = {
            'target_hours': [issue*6, issue*6+6],
            'historical_same_block': metrics(fc['pv'][sl,block]-truth_block,truth_block),
            'zero_same_block': metrics(fc['issued'][sl,0,block]-truth_block,truth_block),
            'six_hour': metrics(pred[:, block] - truth_block, truth_block),
            'remaining_day': metrics(pred[:, rest] - truth_rest, truth_rest),
            'blend_weight': selected['pv_blend_weights'][issue],
```

```
        'blend_same_block': metrics(fc['issued_blend'][sl,issue,block]-truth_block, truth_block),
    }
report['issued_pv']['historical_full_day'] = metrics(
    fc['pv'][sl] - data['pv'][sl], data['pv'][sl])
report['issued_pv']['zero_full_day'] = metrics(
    fc['issued'][sl, 0] - data['pv'][sl], data['pv'][sl])

# 指定日期上独立打乱每个时间列，检验目标是否只使用边际分布。
day = data['dates'].index('2025-09-23')
scenarios = residual_scenarios(data, fc, day, 0, issue=0)
rng = np.random.default_rng(20260911)
shuffled = np.column_stack([rng.permutation(col) for col in scenarios.T])
q3_for_diag = np.load(out_root / 'q3.npz')
initial = float(q3_for_diag['energy'][day, 0])
s1, s2 = optimize(scenarios, data['one'][:, 0], initial), optimize(shuffled, data['one'][:, 0], initial)
f1 = execute(s1['grid'], data['load'][day], data['pv'][day], initial)
f2 = execute(s2['grid'], data['load'][day], data['pv'][day], initial)
report['scenario_shuffle'] = {
    'date': data['dates'][day], 'seed': 20260911,
    'original_objective': s1['objective'], 'permuted_objective': s2['objective'],
    'objective_difference': s2['objective'] - s1['objective'],
    'max_grid_difference': float(np.max(np.abs(s2['grid'] - s1['grid']))),
    'original_feedback_cost': float(DT*np.dot(data['one'][:, 0], s1['grid'] + 5*f1['emergency'])),
    'permuted_feedback_cost': float(DT*np.dot(data['one'][:, 0], s2['grid'] + 5*f2['emergency'])),
}

# 连续全年回测：固定 q2 常规购电计划，只替换实际储能反馈规则。
q2 = np.load(out_root / 'q2.npz')
q3 = np.load(out_root / 'q3.npz'); q3_daily = np.load(out_root / 'q3_daily.npz')
delta = q3_daily['costs'][31:, 4] - q3['costs'][31:, 4]
report['update_cost_by_month'] = {
    str(m): float(delta[[int(x[5:7]) == m for x in data['dates'][31:]]].sum()) for m in range(2, 13)
}
report['update_emergency_by_month'] = {
    str(m): float((q3_daily['emergency'][31:][[int(x[5:7]) == m for x in data['dates'][31:]]].sum()
                   - q3['emergency'][31:][[int(x[5:7]) == m for x in data['dates'][31:]]].sum()) * DT)
    for m in range(2, 13)
}
report['update_days'] = {
    'cheaper': int(np.sum(delta > 1e-8)),
    'more_expensive': int(np.sum(delta < -1e-8)),
    'same': int(np.sum(np.abs(delta) <= 1e-8)),
}
state_g, state_r = EINIT, EINIT
rows = []
for d in range(365):
    base = execute(q2['plan'][d], data['load'][d], data['pv'][d], state_g)
    reserve = execute_reserve(q2['plan'][d], data['load'][d], data['pv'][d], state_r,
                              fc['load'][d], fc['pv'][d])
    p = data['one'][:, 0]
    c_base = DT*np.dot(p, q2['plan'][d] + 5*base['emergency'])
    c_res = DT*np.dot(p, q2['plan'][d] + 5*reserve['emergency'])
    rows.append((d, c_base, c_res, DT*base['emergency'].sum(), DT*reserve['emergency'].sum(),
                 base['energy'][-1], reserve['energy'][-1]))
    state_g, state_r = base['energy'][-1], reserve['energy'][-1]
```

```
    rows = np.asarray(rows)
    rr = rows[31:]
    report['feedback_benchmark'] = {
        'rule': '预留未来36个10分钟时段日前预测缺额所需电量',
        'horizon_periods': 36,
        'base_total_cost': float(rr[:, 1].sum()), 'reserve_total_cost': float(rr[:, 2].sum()),
        'cost_difference_reserve_minus_base': float(rr[:, 2].sum()-rr[:, 1].sum()),
        'base_emergency_kwh': float(rr[:, 3].sum()), 'reserve_emergency_kwh': float(rr[:, 4].sum()),
        'emergency_difference_kwh': float(rr[:, 4].sum()-rr[:, 3].sum()),
        'cheaper_days': int(np.sum(rr[:, 2] < rr[:, 1]-1e-8)),
        'more_expensive_days': int(np.sum(rr[:, 2] > rr[:, 1]+1e-8)),
        'same_days': int(np.sum(np.abs(rr[:, 2]-rr[:, 1]) <= 1e-8)),
        'base_end_soc': float(state_g), 'reserve_end_soc': float(state_r),
    }
    # 重新计算两条连续轨迹的全程SOC边界，避免只报告末态。
    state_g, state_r = EINIT, EINIT; mins = [1e9, 1e9]; maxs = [-1e9, -1e9]
    for d in range(365):
        b = execute(q2['plan'][d], data['load'][d], data['pv'][d], state_g)
        r = execute_reserve(q2['plan'][d], data['load'][d], data['pv'][d], state_r, fc['load'][d], fc['pv'][d])
        mins = [min(mins[0], b['energy'].min()), min(mins[1], r['energy'].min())]
        maxs = [max(maxs[0], b['energy'].max()), max(maxs[1], r['energy'].max())]
        state_g, state_r = b['energy'][-1], r['energy'][-1]
    report['feedback_benchmark'].update({'base_min_soc': mins[0], 'reserve_min_soc': mins[1],
                                         'base_max_soc': maxs[0], 'reserve_max_soc': maxs[1]})
    (out_root / 'diagnostics.json').write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
    print(json.dumps(report, ensure_ascii=False, indent=2))


if __name__ == '__main__':
    import argparse
    ap = argparse.ArgumentParser(); ap.add_argument('--data', required=True); ap.add_argument('--out', default=Path(__file__).resolve().parents[1]/'results')
    a = ap.parse_args(); run(a.data, a.out)
```

B.7 信息参照与策略迭代实验

```
"""第二轮审稿实验: 信息参照、终端库存价值、经济性门控与费用驱动选型。

主结果保持为事先定义的四套基准。新增策略均从同一主策略的1月实际
状态启动；参数只用1月15—31日选择，2—12月仅检验，不反选参数。
"""
import argparse, json
from pathlib import Path
from time import perf_counter
import numpy as np
from model import (DT, ETA, EMIN, EMAX, EINIT, PMAX, METHODS, load_data,
                   forecast_all, predict, residual_scenarios, optimize,
                   execute, settlement_cost, audit, summarize)

ROOT = Path(__file__).resolve().parents[1]
R = ROOT/'results'


def next_value(data, forecast, day, price_mode, gamma):
    """日前估计的次日均价；当天尚未观测的价格不参与次日预测。"""
    if price_mode == 'fixed':
        prices = data['one'][:, 0]
```

```
    else:
        idx = [day+1-lag for lag in (7, 14, 21, 28) if 0 <= day+1-lag < day]
        prices = np.mean(data['price'][idx], axis=0) if idx else forecast['price'][day]
    return float(gamma*ETA*np.mean(prices))


def policy_value(grid, base, scenarios, prices, initial, terminal_value=0.):
    """在相同净负荷场景下，按实际反馈规则评估剩余现金费用减期末库存价值。

    一条误差轨迹内部按时间连续推进SOC；只计算未执行部分。
    """
    state = np.full(len(scenarios), initial)
    emergency_bill = np.zeros(len(scenarios))
    for t in range(len(grid)):
        residual = grid[t]-scenarios[:, t]
        charge = np.minimum(np.maximum(residual, 0), np.minimum(PMAX, (EMAX-state)/(ETA*
    DT)))
        discharge = np.minimum(np.maximum(-residual, 0), np.minimum(PMAX, (state-EMIN)*
    ETA/DT))
        emergency = np.maximum(-residual-discharge, 0)
        state += ETA*DT*charge-DT/ETA*discharge
        emergency_bill += 5*prices[t]*emergency*DT
    ordinary = settlement_cost(base, grid, np.zeros(len(grid)), prices)[-1]
    return float(ordinary+emergency_bill.mean()-terminal_value*state.mean())


def choose_adjustment(candidate, incumbent, base, scenarios, prices, state, value=0.):
    old = policy_value(incumbent, base, scenarios, prices, state, value)
    new = policy_value(candidate, base, scenarios, prices, state, value)
    accepted = bool(old-new > 1e-6)  # 仅为数值容差，无额外未给定的固定调整费。
    return (candidate if accepted else incumbent).copy(), dict(
        keep_value=old, candidate_value=new, expected_saving=old-new, accepted=accepted)


def alternative_forecast(data, forecast, method):
    fc = dict(forecast)
    fc['pv'] = forecast['pv'].copy()
    # 1月15日开始对候选方法逐日回放；预测每一天仍只用此前真实数据。
    for day in range(14, 365):
        fc['pv'][day] = predict(data['pv'][:day], data['one'][:, 2], method)
    return fc


def run_refined(data, forecast, baseline, kind=2, price_mode='fixed', gamma=0.,
                gate=False, start_day=31, stop_day=365, daily_oracle=False):
    started = perf_counter()
    arrays = {k: v.copy() for k, v in baseline.items()}
    state = float(arrays['energy'][start_day, 0])
    decisions = []
    lp_seconds = 0.; solves = 0
    for day in range(start_day, stop_day):
        prices = data['one'][:, 0] if price_mode == 'fixed' else forecast['price'][day]
        realized = data['one'][:, 0] if price_mode == 'fixed' else data['price'][day]
        if daily_oracle:
            prices = realized
        value = next_value(data, forecast, day, price_mode, gamma)
        committed = None
        for issue in ([0] if kind == 2 else range(4)):
            start = issue*36
            end = min(start+36, 144) if kind == 3 else 144
            scenarios = (data['load'][day]-data['pv'][day])[None, :] if daily_oracle else
     residual_scenarios(
                data, forecast, day, start, None if kind == 2 else issue)
```

```
            base = arrays['plan'][day, start:].copy() if issue else None
            sol = optimize(scenarios, prices[start:], state, base=base, terminal_value=
    value)
            lp_seconds += sol['seconds']; solves += 1
            if issue == 0:
                arrays['plan'][day] = sol['grid']
                committed = sol['grid'].copy()
            elif gate:
                chosen, record = choose_adjustment(sol['grid'], committed[start:], base,
                                                   scenarios, prices[start:], state,
    value)
                record.update(day=day, hour=issue*6)
                decisions.append(record)
                committed[start:] = chosen
            else:
                committed[start:] = sol['grid']
            g = committed[start:end]
            arrays['grid'][day, start:end] = g
            actual = execute(g, data['load'][day, start:end], data['pv'][day, start:end],
     state)
            for key in ('ch', 'dis', 'emergency', 'spill'):
                arrays[key][day, start:end] = actual[key]
            arrays['energy'][day, start:end+1] = actual['energy']
            state = float(actual['energy'][-1])
        arrays['costs'][day] = settlement_cost(arrays['plan'][day], arrays['grid'][day],
                                               arrays['emergency'][day], realized)
    info = dict(kind=kind, price_mode=price_mode, eta=ETA, settlement='refund', gamma=
    gamma,
                gate=gate, start_day=start_day, stop_day=stop_day, daily_oracle=
    daily_oracle,
                seconds=perf_counter()-started, lp_seconds=lp_seconds, solves=solves,
                accepted=sum(r['accepted'] for r in decisions), rejected=sum(not r['
    accepted'] for r in decisions))
    if stop_day == 365:
        audit(data, arrays)
    return arrays, info, decisions


def validation_score(arrays, price_ref):
    """同一1月15日初值；按统一估价扣回候选间不同期末库存。"""
    cash = float(arrays['costs'][14:31, 4].sum())
    final = float(arrays['energy'][30, -1])
    return dict(cash_cost=cash, final_energy=final, adjusted_cost=cash-price_ref*final)


def oracle_full(data, baseline, price_mode):
    """报告期全部未来量已知的稀疏LP，期初和期末SOC与主策略完全一致。"""
    n = 334*144
    prices = np.tile(data['one'][:, 0], 334) if price_mode == 'fixed' else data['price'
    ][31:].ravel()
    net = (data['load'][31:]-data['pv'][31:]).reshape(1, n)
    sol = optimize(net, prices, float(baseline['energy'][31, 0]),
                   final_energy=float(baseline['energy'][-1, -1]))
    arrays = {k: v.copy() for k, v in baseline.items()}
    for k in ('grid', 'ch', 'dis'):
        arrays[k][31:] = sol[k].reshape(334, 144)
    arrays['plan'][31:] = arrays['grid'][31:]
    arrays['emergency'][31:] = 0.
    arrays['spill'][31:] = (sol['grid']+sol['dis']-sol['ch']-net[0]).reshape(334, 144)
    states = np.r_[baseline['energy'][31, 0], sol['energy']]
    for d in range(334):
        arrays['energy'][31+d] = states[d*144:(d+1)*144+1]
    arrays['costs'][31:] = 0.
```

```
    costs = (prices*sol['grid']).reshape(334, 144).sum(axis=1)*DT
    arrays['costs'][31:, 0] = costs
    arrays['costs'][31:, 4] = costs
    assert abs(costs.sum()-sol['objective']) < 1e-4
    assert np.min(arrays['spill'][31:]) >= -1e-6
    audit(data, arrays)
    info = dict(kind=2, price_mode=price_mode, eta=ETA, settlement='refund',
                information='all report-period future actuals', seconds=sol['seconds'],
    solves=1)
    return arrays, info


def check(data, forecast, bases, output):
    # 功率/能量单位与单程效率的独立数值例证。
    charged = execute(np.array([600.]), np.array([0.]), np.array([0.]), 6000.)
    discharged = execute(np.array([0.]), np.array([600.]), np.array([0.]), 6090.)
    assert abs(charged['energy'][-1]-6090.) < 1e-9
    assert abs(discharged['energy'][-1]-(6090.-100/.9)) < 1e-9
    # 独立费用与门控检查；过去原计划不被日内覆盖。
    report = {'energy_example': {'grid_side_charge_kwh': 100., 'internal_increment_kwh':
    90.,
                                 'grid_side_discharge_kwh': 100., 'internal_decrement_kwh
    ': 100/.9}}
    # 用逐场景标量反馈独立复算向量化门控评分，涵盖SOC上下限与充放电切换。
    grid=np.array([500.,0.,3000.,0.]);base=np.array([600.,100.,2800.,20.])
    scenes=np.array([[0.,4000.,100.,6000.],[1000.,0.,3500.,500.]])
    price=np.array([.4,1.2,.6,1.4])
    for initial in (EMIN,EINIT,EMAX):
        values=[]
        for net in scenes:
            feedback=execute(grid,net,np.zeros(4),initial)
            values.append(settlement_cost(base,grid,feedback['emergency'],price)[-1])
        assert abs(policy_value(grid,base,scenes,price,initial)-np.mean(values))<1e-9
    report['gate_scoring_independent'] = 'passed: scalar scene replay at three SOC
    boundaries'
    for key, item in output['cases'].items():
        a = dict(np.load(R/(key+'.npz')))
        audit(data, a)
        price = np.broadcast_to(data['one'][:,0], (365,144)) if item['price_mode']==='
    fixed' else data['price']
        up = np.maximum(a['grid']-a['plan'], 0)
        down = np.maximum(a['plan']-a['grid'], 0)
        direct = (price*np.minimum(a['plan'],a['grid'])+.5*price*down+1.5*price*up+5*
    price*a['emergency']).sum(axis=1)*DT
        assert np.max(abs(direct-a['costs'][:,4])) < 1e-6
        report[key] = {'max_cost_error': float(np.max(abs(direct-a['costs'][:,4]))),
                       'max_cross_day_error': float(np.max(abs(a['energy'][:-1,-1]-a['
    energy'][1:,0])))}
    # 修改某次决策之后的实际量及尚未发布预报，应保持候选解、门控评分与决定一致。
    day, issue = 100, 2
    changed = {k: (v.copy() if isinstance(v,np.ndarray) else v) for k,v in data.items()}
    for key in ('load','pv','price'):
        changed[key][day,issue*36:] *= 1.8
        changed[key][day+1:] *= 1.4
    changed['pv_hour'][day,issue+1:] *= .3
    changed['pv_hour'][day+1:] *= .2
    altered,_,_ = forecast_all(changed)
    outcomes = []
    b = bases['q4_3']; start=issue*36
    for dd, ff in ((data,forecast),(changed,altered)):
        scenarios=residual_scenarios(dd,ff,day,start,issue)
        value=next_value(dd,ff,day,'forecast',1.)
```

```
        sol=optimize(scenarios,ff['price'][day,start:],b['energy'][day,start],base=b['plan'][day,start:],terminal_value=value)
        g,r=choose_adjustment(sol['grid'],b['plan'][day,start:],b['plan'][day,start:],scenarios,ff['price'][day,start:],b['energy'][day,start],value)
        outcomes.append((g,r,value))
    assert np.array_equal(outcomes[0][0],outcomes[1][0]) and outcomes[0][1:] == outcomes[1][1:]
    report['new_decision_causality'] = 'passed: candidate, gate scores/decision and next-day value unchanged'
    # 与前稿完全相同的已运行基准复算若干决策，核查新增可选参数未改变默认模型。
    for day in (31,100,364):
        b=bases['q2'];sc=residual_scenarios(data,forecast,day,0)
        sol=optimize(sc,data['one'][:,0],b['energy'][day,0])
        assert np.max(abs(sol['grid']-b['plan'][day])) < 1e-7
    report['baseline_regression'] = 'passed: 3 dates; defaults unchanged'
    output['verification'] = report


def main():
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,required=True);args=p.parse_args()
    data=load_data(args.data);forecast,_,selected=forecast_all(data)
    bases={k:dict(np.load(R/(k+'.npz'))) for k in ('q2','q3','q4_2','q4_3')}
    output={'protocol':'Jan 15-31 calibration; Feb 1-Dec 31 frozen holdout; shared January actual warmup',
            'cases':{},'validation':{},'gate_decisions':{}}
    def save(name, a, info, decisions=None):
        np.savez_compressed(R/(name+'.npz'),**a)
        output['cases'][name]=summarize(a,info)
        if decisions is not None: output['gate_decisions'][name]=decisions
        (R/'refinements.json').write_text(json.dumps(output,ensure_ascii=False,indent=2),encoding='utf-8')
        print(name,json.dumps(output['cases'][name],ensure_ascii=False),flush=True)
    # 不同价格条件各自仅在一月份选择终端价值强度和光伏预测方法。
    for mode, basekey in (('fixed','q2'),('forecast','q4_2')):
        baseline=bases[basekey]
        price_ref=ETA*float(data['one'][:36,0].mean())
        terminal={}
        for gamma in (0., .5, 1., 1.5):
            a,_,_=run_refined(data,forecast,baseline,price_mode=mode,gamma=gamma,start_day=14,stop_day=31)
            terminal[str(gamma)]=validation_score(a,price_ref)
        chosen_gamma=float(min(terminal,key=lambda x:terminal[x]['adjusted_cost']))
        output['validation'][basekey+'_terminal']={'scores':terminal,'selected':chosen_gamma,'inventory_price':price_ref}
        a,info,_=run_refined(data,forecast,baseline,price_mode=mode,gamma=chosen_gamma)
        save(basekey+'_terminal',a,info)
        economic={}
        for method in METHODS:
            fc=alternative_forecast(data,forecast,method)
            a,_,_=run_refined(data,fc,baseline,price_mode=mode,start_day=14,stop_day=31)
            score=validation_score(a,price_ref)
            error=fc['pv'][14:31]-data['pv'][14:31]
            score['mae']=float(np.mean(abs(error)));score['rmse']=float(np.sqrt(np.mean(error**2)))
            economic[method]=score
        chosen_method=min(economic,key=lambda x:economic[x]['adjusted_cost'])
        output['validation'][basekey+'_economic']={'scores':economic,'selected':chosen_method,'mae_selected':selected['pv'],'inventory_price':price_ref}
        fc=alternative_forecast(data,forecast,chosen_method)
        a,info,_=run_refined(data,fc,baseline,price_mode=mode);info['pv_method']=chosen_method
        save(basekey+'_economic',a,info)
```

```
        # MAE选型也使用同样的候选历史回放，避免1月底误差缓存不同造成混杂。
        fc=alternative_forecast(data,forecast,selected['pv'])
        a,info,_=run_refined(data,fc,baseline,price_mode=mode);info['pv_method']=selected['pv']
        save(basekey+'_mae_control',a,info)
        # 从完全相同1月末SOC出发的逐日先知参照，日内全曲线已知，但仍只优化当天。
        a,info,_=run_refined(data,forecast,baseline,price_mode=mode,daily_oracle=True)
        save(basekey+'_daily_oracle',a,info)
        a,info=oracle_full(data,baseline,mode)
        save(basekey+'_full_oracle',a,info)
    for mode, basekey in (('fixed','q3'),('forecast','q4_3')):
        a,info,decisions=run_refined(data,forecast,bases[basekey],kind=3,price_mode=mode,gate=True)
        assert all((r['expected_saving']>1e-6)==r['accepted'] for r in decisions)
        save(basekey+'_gate',a,info,decisions)
    check(data,forecast,bases,output)
    (R/'refinements.json').write_text(json.dumps(output,ensure_ascii=False,indent=2),encoding='utf-8')
    print('All refinement experiments and independent checks passed.',flush=True)


if __name__=='__main__':main()
```

B.8 新增实验证据生成

```
"""把第二轮实验结果写成LaTeX表格与数值宏，不改五份主结果工作簿。"""
import json
from pathlib import Path
import numpy as np
ROOT=Path(__file__).resolve().parents[1];R=ROOT/'results';T=ROOT/'tables'
S=json.loads((R/'summary.json').read_text(encoding='utf-8'))['cases']
D=json.loads((R/'refinements.json').read_text(encoding='utf-8'));C=D['cases'];V=D['validation']
def number(x):return f'{float(x):.2f}'
def table(name,caption,headers,rows):
    text='\\begin{table}[H]\\centering\\small\n\\caption{'+caption+'}\\label{tab:'+name+'}\n'
    text+='\\begin{tabular}{l'+'r'*(len(headers)-1)+'}\\toprule\n'
    text+=' & '.join(headers)+r' \\\midrule'+'\n'
    text+='\n'.join(' & '.join(map(str,row))+r' \\' for row in rows)
    text+='\n\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')
rows=[]
for base,prefix in [('q2','重复价'),('q4_2','波动价')]:
    for key,desc in [(base,'可实施日前'),(base+'_daily_oracle','当日先知'),(base+'_full_oracle','全期先知')]:
        c=S[key] if key in S else C[key]
        rows.append([prefix+'：'+desc,number(c['total_cost']),number(c['emergency_kwh']),number(c['start_energy']),number(c['end_energy'])])
table('information_benchmark','信息设定对照；费用为元，电量为kWh',
      ['信息与策略','总费用','紧急电量','期初SOC','期末SOC'],rows)
rows=[]
for g in ('0.0','0.5','1.0','1.5'):
    rows.append([g,number(V['q2_terminal']['scores'][g]['adjusted_cost']),number(V['q4_2_terminal']['scores'][g]['adjusted_cost'])])
table('terminal_validation','仅用1月15--31日选择终端价值；数值为库存修正费用/元',
      [r'$\gamma$','重复电价','波动电价'],rows)
rows=[]
for base,label in [('q2','重复电价'),('q4_2','波动电价')]:
    for key,desc in [(base,'原模型'),(base+'_terminal','终端价值')]:
        c=S[key] if key in S else C[key]
        rows.append([label+'：'+desc,number(c.get('gamma',0)),number(c['total_cost']),number(c['emergency_kwh']),number(c['end_energy'])])
```

```
table('terminal_results','终端价值的报告期检验；不将虚拟库存收益计入现金费用',
      ['方案',r'$\gamma$','总费用/元','紧急电量/kWh','期末SOC/kWh'],rows)
names={'last':'前一日','mean3':'3日均值','mean7':'7日均值','ewma14':'指数加权','weekday':'同星期'}
rows=[]
for method in names:
    a=V['q2_economic']['scores'][method];b=V['q4_2_economic']['scores'][method]
    rows.append([names[method],number(a['mae']),number(a['rmse']),number(a['adjusted_cost']),number(b['adjusted_cost'])])
table('economic_validation','光伏候选的1月顺序验证；费用均已按统一价格修正期末库存',
      ['候选方法','MAE/kW','RMSE/kW','重复价费用/元','波动价费用/元'],rows)
rows=[]
for base,label in [('q2','重复价'),('q4_2','波动价')]:
    for suffix,desc in [('_mae_control','MAE选型'),('_economic','费用选型')]:
        c=C[base+suffix]
        rows.append([label+'：'+desc,names[c['pv_method']],number(c['total_cost']),number(c['emergency_kwh'])])
table('economic_results','相同候选历史回放下的预测选型对照（2--12月）',
      ['选择规则','光伏方法','总费用/元','紧急电量/kWh'],rows)
rows=[]
for base,label in [('q3','重复价'),('q4_3','波动价')]:
    for key,desc in [(base,'直接滚动'),(base+'_gate','经济门控')]:
        c=S[key] if key in S else C[key]
        rows.append([label+'：'+desc,number(c['total_cost']),number(c['emergency_kwh']),str(c.get('accepted',1002)),str(c.get('rejected',0))])
table('gate_results','门控与直接滚动的同信息比较；次数为日内候选采纳/保留次数',
      ['方案','总费用/元','紧急电量/kWh','采纳','保留'],rows)
derived={}
for base in ('q2','q4_2'):
    derived[base+'oracle_gap']=S[base]['total_cost']-C[base+'_full_oracle']['total_cost']
    derived[base+'terminal_saving']=S[base]['total_cost']-C[base+'_terminal']['total_cost']
    derived[base+'economic_saving']=C[base+'_mae_control']['total_cost']-C[base+'_economic']['total_cost']
    derived[base+'economic_increase']=-derived[base+'economic_saving']
for base in ('q3','q4_3'):
    derived[base+'gate_saving']=S[base]['total_cost']-C[base+'_gate']['total_cost']
    derived[base+'gate_percent']=100*derived[base+'gate_saving']/S[base]['total_cost']
    ds=D['gate_decisions'][base+'_gate']
    derived[base+'gate_negative']=sum(x['expected_saving'] < -1e-6 for x in ds)
    derived[base+'gate_ties']=sum(abs(x['expected_saving'])<=1e-6 for x in ds)
macros=['% Generated by refinement_evidence.py; no handwritten result values.']
for group,values in dict(C,extra=derived).items():
    for field,value in values.items():
        if isinstance(value,(int,float)):
            macros.append('\\expandafter\\def\\csname result'+group+field+'\\endcsname{'+(str(value) if isinstance(value,int) else number(value))+'}')
(T/'refinement_metrics.tex').write_text('\n'.join(macros)+'\n',encoding='utf-8')
(R/'refinement_metrics.json').write_text(json.dumps(derived,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(derived,ensure_ascii=False,indent=2))
```

B.9 描述统计与表题

```
"""编辑层处理：分析表标题及描述统计；不更改优化模型或原始结果。"""
from pathlib import Path
import json,re,statistics

ROOT=Path(__file__).resolve().parents[1]
CAPTIONS={
    'risk_compare':'计入缺额风险后，常规购电增加而总费用下降（本算例）',
    'information_benchmark':'先知参照费用更低，比较仍须区分信息与库存边界（费用/元，电量/kWh）',
    'terminal_validation':'1月验证未支持重复电价下的正终端奖励（库存修正费用/元）',
```

```
    'terminal_results':'所选终端价值仅在波动电价日前方案中降低报告期费用',
    'gate_results':'计费后筛选调整小幅节省总费，紧急电量略增（采纳、保留以次计）',
    'frequency':'相同零时预报下，四次更新的报告期费用最低',
    'issued_forecast':'6时与12时更新改善对应白天时段的预报误差（误差单位：kW）',
    'update_robustness':'11个月均节省费用，5月的紧急电量仍有增加',
    'price_information':'提前获知价格降低回测费用，并增加储能循环强度',
    'annual':'本样本中，波动电价下两套主方案的费用均高于重复电价基准',
    'scenario_diagnostic':'打乱时间配对未改变LP代理目标，却改变了反馈费用',
    'feedback_benchmark':'本算例的36时段保留规则增加了紧急购电与总费',
    'settlement':'是否退回取消电量原费，会改变下调行为及总费',
    'window':'7天窗口回测费用较低，尚不能据此认定全年最优窗口',
    'blend_results':'预报融合降低了总费，但紧急电量并未同步下降',
    'economic_validation':'1月库存修正费用选型偏好3日均值，误差选型偏好7日均值',
    'economic_results':'相同历史回放下，费用选型未优于MAE选型（2--12月）',
}

def main():
    for stem,title in CAPTIONS.items():
        p=ROOT/'tables'/(stem+'.tex');s=p.read_text(encoding='utf-8')
        s,n=re.subn(r'\\caption\{[^{}]*\}',lambda _:r'\caption{'+title+'}',s,count=1)
        assert n==1,stem
        p.write_text(s,encoding='utf-8')
    d=json.loads((ROOT/'results/diagnostics.json').read_text(encoding='utf-8'))
    months=d['update_cost_by_month']
    values=[float(months[str(m)]) for m in range(2,13)]
    stats={'source':'results/diagnostics.json:update_cost_by_month',
           'definition':'monthly cost: q3_daily minus q3, yuan',
           'n':len(values),'ddof':1,'values_yuan':values,
           'mean_yuan':statistics.mean(values),'sd_yuan':statistics.stdev(values),
           'min_yuan':min(values),'max_yuan':max(values),
           'interpretation':'descriptive month-to-month spread; not a confidence or
    prediction interval'}
    assert all(x>0 for x in values) and len(values)==11
    (ROOT/'results/editorial_statistics.json').write_text(json.dumps(stats,ensure_ascii=
    False,indent=2),encoding='utf-8')
    summary=json.loads((ROOT/'results/summary.json').read_text(encoding='utf-8'))['cases'
    ]
    groups={}
    for key in ('q2','q3','q4_2','q4_3'):
        groups[key+'_cost_wan']=summary[key]['total_cost']/10000
    groups.update(monthly_mean_wan=stats['mean_yuan']/10000,monthly_sd_wan=stats['sd_yuan
    ']/10000,
                  monthly_min_wan=stats['min_yuan']/10000,monthly_max_wan=stats['max_yuan
    ']/10000)
    lines=['% Presentation precision and descriptive statistics; source arrays are
    unchanged.']
    for k,v in groups.items():
        lines.append('\\expandafter\\def\\csname resulteditorial'+k+'\\endcsname{'+f'{v
    :.2f}'+'}')
    (ROOT/'tables/editorial_metrics.tex').write_text('\n'.join(lines)+'\n',encoding='utf
    -8')
    print(json.dumps(stats,ensure_ascii=False,indent=2))

if __name__=='__main__':main()
```

B.10 论文图组

```
"""从已核对的轨迹绘制论文图组；白底、细框、多线型，采用 MATLAB 常用配色。
只生成图和描述统计，不重新求解，不改写主结果。先运行原复现流程获得 NPZ。
"""
from pathlib import Path
import json, hashlib
import numpy as np
```

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Rectangle, FancyArrowPatch, Polygon
from matplotlib.ticker import AutoMinorLocator

ROOT=Path(__file__).resolve().parents[1];R=ROOT/'results';F=ROOT/'figures'
F.mkdir(exist_ok=True);P=F/'previews';P.mkdir(exist_ok=True)
S=json.loads((R/'summary.json').read_text(encoding='utf-8'))
D=json.loads((R/'diagnostics.json').read_text(encoding='utf-8'))
RF=json.loads((R/'refinements.json').read_text(encoding='utf-8'))
Q=dict(np.load(R/'q1.npz'));I=dict(np.load(R/'inputs_processed.npz'))
keys=('q2','q3','q4_2','q4_3','q3_daily','q3_noon','q4_3_known')
A={k:dict(np.load(R/(k+'.npz'))) for k in keys}
DT=1/6;start=31;h=np.arange(144)/6;edges=np.arange(145)/6
dates=np.array(S['dates']);months=np.array([int(d[5:7]) for d in dates]);ms=np.arange
    (2,13)
for k,a in A.items():
    assert abs(a['costs'][start:,4].sum()-S['cases'][k]['total_cost'])<1e-5,k
    assert abs(a['emergency'][start:].sum()*DT-S['cases'][k]['emergency_kwh'])<1e-5,k
    assert np.max(abs(a['energy'][:-1,-1]-a['energy'][1:,0]))<1e-7,k
for font in ('simsun.ttc','times.ttf'):
    p=Path('C:/Windows/Fonts')/font
    if p.exists():font_manager.fontManager.addfont(str(p))
plt.rcParams.update({'font.family':['Times New Roman','SimSun','DejaVu Sans'],'font.size'
    :10,
 'axes.labelsize':10,'axes.titlesize':10,'axes.linewidth':.7,'lines.linewidth':1.25,
 'xtick.direction':'in','ytick.direction':'in','xtick.major.size':3,'ytick.major.size':3,
 'axes.unicode_minus':False,'pdf.fonttype':42,'ps.fonttype':42,'savefig.facecolor':'white
    ',
 'axes.facecolor':'white','figure.facecolor':'white','legend.frameon':True,'legend.
    fancybox':False,
 'legend.edgecolor':'0.65','legend.framealpha':.95,'legend.fontsize':9,'grid.linewidth'
    :.45,
 'axes.spines.top':True,'axes.spines.right':True,'savefig.bbox':'tight'})
BLUE='#0072BD';ORANGE='#D95319';YELLOW='#EDB120';PURPLE='#7E2F8E';GREEN='#77AC30';CYAN='
    #4DBEEE';RED='#A2142F';GRAY='#606060'
colors=[BLUE,ORANGE,YELLOW,PURPLE,GREEN,CYAN,RED]
metrics={'source':'Existing trajectories; 2025-02-01 to 2025-12-31 unless stated
    otherwise',
 'plot_style':'White background, fine boxed axes, restrained grid, differentiated lines
    and markers; MATLAB palette',
 'main_result_checks':'passed','figures':[]}

def axes_style(ax,grid=True):
    if grid:ax.grid(True,color='#D9D9D9',alpha=.75)
    ax.set_axisbelow(True)
    for s in ax.spines.values():s.set_color('#555555')

def sub(ax,title):ax.set_title(title,pad=8)

def save(fig,name,source):
    fig.savefig(F/(name+'.pdf'));fig.savefig(P/(name+'.png'),dpi=175);plt.close(fig)
    metrics['figures'].append({'file':name+'.pdf','data_or_definition':source})

def box(ax,xy,w,hh,text,face='white',size=10,style='-'):
    x,y=xy;ax.add_patch(Rectangle((x,y),w,hh,facecolor=face,edgecolor='#444444',lw=.8,ls=
    style))
    ax.text(x+w/2,y+hh/2,text,ha='center',va='center',fontsize=size)

def arrow(ax,a,b,label=None,offset=(0,0),style='->',color='#444444'):
```

```
    ax.add_patch(FancyArrowPatch(a,b,arrowstyle=style,mutation_scale=10,lw=.85,color=color))
    if label:ax.text((a[0]+b[0])/2+offset[0],(a[1]+b[1])/2+offset[1],label,ha='center',va='center',fontsize=9,bbox={'fc':'white','ec':'none','pad':1})

# 1. 微网物理结构；上下箭头分别显示充放电方向。
fig,ax=plt.subplots(figsize=(6.8,2.65));ax.set(xlim=(0,10),ylim=(0,4.4));ax.axis('off')
box(ax,(.25,2.55),2,1,'外部电网\n常规 / 紧急购电','#F7E2CD')
box(ax,(.25,.55),2,1,'光伏发电\n实际出力','#E8F1D8')
box(ax,(4.15,1.75),1.6,1,'交流母线','#EFF3F8')
box(ax,(7.8,2.55),1.9,1,'小区负载\n功率需求')
box(ax,(7.8,.55),1.9,1,'储能电池\n1200–10800 kWh','#E8F1D8')
arrow(ax,(2.25,3.05),(4.15,2.45));arrow(ax,(2.25,1.05),(4.15,2.05))
arrow(ax,(5.75,2.45),(7.8,3.05));arrow(ax,(5.75,2.1),(7.8,1.25),'入库 × 0.9',(0,.18))
arrow(ax,(7.8,.72),(5.45,1.75),'耗库 ÷ 0.9',(0,-.15))
ax.text(4.9,.47,'盈余供给可弃置；不设售电',ha='center',fontsize=9)
ax.text(4.9,3.85,'功率平衡在母线侧计量，电量状态在电池内部递推',ha='center',fontsize=10)
save(fig,'energy_network','Equipment parameters and balance/SOC equations; conceptual diagram, not measured data')

# 2. 分层流程借鉴范文的浅色分组与细线边框。
fig,ax=plt.subplots(figsize=(6.8,3.3));ax.set(xlim=(0,13.4),ylim=(0,6));ax.axis('off')
groups=[(.1,2.25,'信息输入','#FBF5E9'),(2.9,2.25,'基础模型','#F7E2CD'),(5.7,3.4,'分问决策','#EDF4E3'),(9.65,3.55,'执行与评价','#EFF3F8')]
for x,w,title,c in groups:
    box(ax,(x,.45),w,5.25,'',c,style='--');ax.text(x+w/2,5.38,title,ha='center',fontsize=10)
for y,t in [(3.9,'历史负荷\n历史光伏'),(2.65,'当前实际\n储电量'),(1.4,'已知电价\n已发布预报')]:box(ax,(.28,y),1.9,.85,t,size=8.7)
for y,t in [(3.9,'功率平衡'),(2.65,'储能动态'),(1.4,'容量与\n功率边界')]:box(ax,(3.08,y),1.9,.85,t,size=9)
for y,t in [(4.05,'问题1：确定性LP'),(2.85,'问题2：历史场景日前'),(1.65,'问题3：日内滚动调整'),(.65,'问题4：替换价格信息')]:box(ax,(5.9,y),3,.7,t,size=9)
box(ax,(9.9,3.65),3.05,1.1,'实际反馈执行\n补电、盈余与SOC',size=9)
box(ax,(9.9,1.75),3.05,1.1,'分项结算与结果核验\n图表、敏感性与对照',size=9)
arrow(ax,(2.35,3.1),(2.9,3.1));arrow(ax,(5.15,3.1),(5.7,3.1));arrow(ax,(9.1,3.1),(9.65,3.1))
arrow(ax,(11.42,3.65),(11.42,2.85));ax.text(11.42,1.05,'实际SOC传递至下一日',ha='center',fontsize=9)
save(fig,'study_flow','Four-problem modeling architecture and information boundary')

# 3. 单日调度，真实阶梯功率；图中阴影仅代表低价时段。
fig,ax=plt.subplots(3,1,figsize=(6.8,5.7),sharex=True,layout='constrained')
ax[0].plot(h,Q['load']/1000,color=BLUE,label='负载');ax[0].plot(h,Q['pv']/1000,color=GREEN,label='光伏')
ax[0].step(h,Q['grid']/1000,where='post',color=ORANGE,label='常规购电');ax[0].set_ylabel('功率 / MW');ax[0].legend(ncol=3,loc='upper center')
sub(ax[0],'(a) 供需与购电')
ax[1].step(h,Q['ch']/1000,where='post',color=BLUE,label='充电');ax[1].step(h,-Q['dis']/1000,where='post',color=ORANGE,label='放电（负向）')
ax[1].axhline(0,color=GRAY,lw=.65);ax[1].set_ylabel('功率 / MW');ax[1].legend(ncol=2,loc='upper center');sub(ax[1],'(b) 充放电切换')
ax[2].plot(edges,np.r_[6000,Q['energy']]/1000,color=BLUE,label='储电量');ax[2].axhline(1.2,color=GRAY,ls='--',lw=.8);ax[2].axhline(10.8,color=GRAY,ls='--',lw=.8)
ax[2].set_ylabel('储电量 / MWh');rt=ax[2].twinx();rt.step(h,Q['price'],color=ORANGE,where='post',ls='--',lw=1,label='电价');rt.set_ylabel('电价 / (元/kWh)')
ax[2].legend(loc='upper left');rt.legend(loc='upper right');sub(ax[2],'(c) 库存与价格');ax[2].set(xlim=(0,24),xticks=np.arange(0,25,4),xlabel='时刻 / h')
for aa in ax:axes_style(aa)
save(fig,'q1_dispatch','q1.npz: 144 period bus-side power, 145 SOC boundaries')

# 4. 输入样本结构，均值曲线不作为零时已知输入。
fig,ax=plt.subplots(1,2,figsize=(6.8,2.65),layout='constrained')
```

```
for j,m in enumerate((3,6,9,12)):
    mask=months==m
    ax[0].plot(h,I['load'][mask].mean(axis=0)/1000,color=colors[j],ls=['-','--','-.',':'][j],label=f'{m}月')
    ax[1].plot(h,I['pv'][mask].mean(axis=0)/1000,color=colors[j],ls=['-','--','-.',':'][j],label=f'{m}月')
for aa,t in zip(ax,('(a) 负荷月内平均曲线','(b) 光伏月内平均曲线')):
    axes_style(aa);sub(aa,t);aa.set(xlim=(0,24),xticks=np.arange(0,25,6),xlabel='时刻 / h',ylabel='平均功率 / MW');aa.legend(ncol=2,fontsize=8)
save(fig,'input_profiles','inputs_processed.npz: ex-post monthly mean power; descriptive only')

# 5. 日前预测误差分布，不构造虚假的误差带。
fig,ax=plt.subplots(1,2,figsize=(6.8,2.65),layout='constrained')
for aa,key,label,c in zip(ax,('load','pv'),('负荷','光伏'),(BLUE,ORANGE)):
    err=(I['forecast_'+key][start:]-I[key][start:]).ravel()/1000
    aa.hist(err,bins=55,color=c,alpha=.65,edgecolor='white',linewidth=.3,weights=np.ones(len(err))*100/len(err))
    aa.axvline(0,color=GRAY,lw=.85,ls='--');axes_style(aa);aa.set(xlabel='预测 − 实际 / MW',ylabel='时段比例 / %');sub(aa,f'{label}日前预测误差')
    mae=np.mean(abs(err))*1000;rmse=np.sqrt(np.mean(err**2))*1000
    aa.text(.97,.96,f'MAE = {mae:.1f} kW\nRMSE = {rmse:.1f} kW',ha='right',va='top',transform=aa.transAxes,fontsize=8.5,bbox={'fc':'white','ec':'.8','pad':3})
    metrics[key+'_forecast']={'n':len(err),'mae_kw':mae,'rmse_kw':rmse,'mean_error_kw':float(err.mean()*1000)}
save(fig,'forecast_errors','inputs_processed.npz: all 48096 holdout periods, prediction minus actual; histogram percentages')

# 6. 发布时间/执行窗口示意，灰色条为已执行或未采用的时域。
fig,ax=plt.subplots(figsize=(6.8,2.4),layout='constrained')
for j,k in enumerate((0,6,12,18)):
    y=3-j;ax.barh(y,24-k,left=k,height=.5,color=colors[j],alpha=.23,edgecolor=colors[j]);ax.barh(y,6,left=k,height=.5,color=colors[j],alpha=.9)
    ax.plot(k,y,'|',color='black',markersize=14);ax.text(k+3,y,'本轮执行',ha='center',va='center',fontsize=8,color='white' if j in (0,3) else 'black')
ax.set(yticks=[3,2,1,0],yticklabels=['0时发布','6时发布','12时发布','18时发布'],xlim=(0,24),xticks=np.arange(0,25,6),xlabel='当日时刻 / h')
axes_style(ax);ax.set_title('深色：执行到下一更新；浅色：当日剩余优化范围',fontsize=9)
save(fig,'information_timeline','Rolling algorithm: schematic publication time and action horizon, not result intervals')

# 7. 连续三日实际状态，不将日初重置为6000。
days=[list(dates).index(d) for d in ('2025-09-22','2025-09-23','2025-09-24')]
e=np.r_[A['q2']['energy'][days[0],0],A['q2']['energy'][days,1:].ravel()]/1000
fig,ax=plt.subplots(figsize=(6.8,2.5),layout='constrained');ax.plot(np.arange(len(e))/6,e,color=BLUE,lw=1.2)
for x in (24,48):ax.axvline(x,color=GRAY,ls='--',lw=.85);ax.plot(x,e[int(x*6)],'o',color=ORANGE,ms=4)
ax.axhline(1.2,color=RED,ls=':',lw=.85);ax.axhline(10.8,color=RED,ls=':',lw=.85)
ax.set(xlim=(0,72),ylim=(.65,11.35),xticks=np.arange(0,73,12),xticklabels=['9/22\n0时','12时','9/23\n0时','12时','9/24\n0时','12时','9/25\n0时'],ylabel='实际储电量 / MWh')
axes_style(ax);save(fig,'soc_continuity','q2.npz: actual SOC, 2025-09-22 00:00 to 2025-09-25 00:00')

# 8. 紧急电量按日、时段分布；同一色标，不截断最大值。
fig,ax=plt.subplots(1,2,figsize=(6.8,3.4),sharey=True,layout='constrained')
vmax=max(A[k]['emergency'][start:].max()*DT for k in ('q2','q3'))
for aa,k,t in zip(ax,('q2','q3'),('(a) 问题2','(b) 问题3')):
    vals=A[k]['emergency'][start:]*DT
    im=aa.imshow(vals,origin='upper',aspect='auto',extent=(0,24,334,0),cmap='YlOrRd',vmin=0,vmax=vmax,interpolation='nearest')
    aa.set(xticks=np.arange(0,25,6),xlabel='时刻 / h');sub(aa,t)
    ticks=[list(dates[start:]).index(f'2025-{m:02}-01') for m in (2,4,6,8,10,12)]
```

```
    aa.set_yticks(ticks,[f'{m}月1日' for m in (2,4,6,8,10,12)])
fig.colorbar(im,ax=ax,label='每10分钟紧急电量 / kWh',shrink=.92)
save(fig,'emergency_heatmap','q2/q3.npz: bus-side emergency power times 1/6 h; all report dates and shared linear scale')

# 9. 月度电量、费用分别显示。
fig,ax=plt.subplots(2,1,figsize=(6.8,3.8),sharex=True,layout='constrained')
for k,c,marker in (('q2',BLUE,'o'),('q3',ORANGE,'s')):
    em=[A[k]['emergency'][months==m].sum()*DT/1000 for m in ms];cost=[A[k]['costs'][months==m,4].sum()/10000 for m in ms]
    ax[0].plot(ms,em,marker+'-',color=c,label='问题2' if k=='q2' else '问题3',ms=4);ax[1].plot(ms,cost,marker+'-',color=c,ms=4)
ax[0].set_ylabel('紧急电量 / MWh');ax[0].legend(ncol=2);sub(ax[0],'(a) 紧急购电量')
ax[1].set(ylabel='总费用 / 万元',xlabel='月份',xticks=ms);sub(ax[1],'(b) 实际结算费用')
for aa in ax:axes_style(aa)
save(fig,'monthly_comparison','q2/q3.npz: sum by calendar month, February to December')

# 10. 分段结算曲线，由论文公式直接计算。
x=np.linspace(0,9,181);cb=np.minimum(x,5)+.5*np.maximum(5-x,0)+1.5*np.maximum(x-5,0);ca=5+.5*np.maximum(5-x,0)+1.5*np.maximum(x-5,0)
fig,ax=plt.subplots(figsize=(6.8,2.65),layout='constrained');ax.plot(x,cb,color=BLUE,label='主口径：取消量仅付50%');ax.plot(x,ca,'--',color=ORANGE,label='对照：原费不退、另收50%')
for y,c,txt in ((4.5,BLUE,'(4, 4.5)'),(5.5,ORANGE,'(4, 5.5)')):ax.plot(4,y,'o',ms=4,color=c);ax.annotate(txt,(4,y),xytext=(2.0,y+(.6 if y>5 else -.7)),arrowprops={'arrowstyle':'-','color':c,'lw':.7},fontsize=9)
ax.plot(5,5,'ks',ms=4);ax.set(xlim=(0,9),ylim=(0,12),xlabel='正式调整后的常规电量 / kWh',ylabel='常规交易费用 / 元');axes_style(ax);ax.legend(loc='upper left',fontsize=9)
save(fig,'settlement_curve','Piecewise contract equations: original plan 5 kWh, price 1 yuan/kWh; emergency excluded')

# 11. 经济门控流程。图内概念对应公式，不借用迭代收敛曲线。
fig,ax=plt.subplots(figsize=(6.8,2.8));ax.set(xlim=(0,13),ylim=(0,5));ax.axis('off')
box(ax,(.1,2.05),2.3,1.15,'最新预报\n当前实际SOC','#EFF3F8',9)
box(ax,(3.0,3.15),2.5,.95,'优化候选安排 g*','#F7E2CD',9)
box(ax,(3.0,1.05),2.5,.95,'保留原执行安排',size=9)
diamond=np.array([[7.3,3.6],[8.7,2.55],[7.3,1.5],[5.9,2.55]])
ax.add_patch(Polygon(diamond,fc='#EDF4E3',ec='#444444',lw=.8));ax.text(7.3,2.55,'同场景评分\n预计节省\n大于容差？',ha='center',va='center',fontsize=8.5)
box(ax,(10,3.15),2.5,.95,'采纳候选','#EDF4E3',9);box(ax,(10,1.05),2.5,.95,'继续现行安排',size=9)
arrow(ax,(2.4,2.85),(3,3.62));arrow(ax,(2.4,2.35),(3,1.52));arrow(ax,(5.5,3.62),(6.35,2.9));arrow(ax,(5.5,1.52),(6.35,2.2));arrow(ax,(8.4,2.78),(10,3.62),'是');arrow(ax,(8.4,2.3),(10,1.52),'否')
ax.text(6.5,.3,'原始零时合同不改；评分已含调整费；仅更新未来执行段',ha='center',fontsize=10)
save(fig,'gate_flow','Same-information candidate gate defined in paper; no extra adjustment-fee subtraction')

# 12. 典型日计划、状态及操作区间，末层借鉴范文的时间区间图。
d=list(dates).index('2025-03-20');a=A['q3']
fig,ax=plt.subplots(3,1,figsize=(6.8,5.15),sharex=True,gridspec_kw={'height_ratios':[1.3,1.2,.55]},layout='constrained')
ax[0].step(h,a['plan'][d]/1000,where='post',color=GRAY,ls='--',label='零时计划');ax[0].step(h,a['grid'][d]/1000,where='post',color=BLUE,label='最终常规');ax[0].plot(h,a['emergency'][d]/1000,color=ORANGE,label='紧急')
ax[0].set_ylabel('购电功率 / MW');ax[0].legend(ncol=3,fontsize=8.5);sub(ax[0],'(a) 原计划与最终执行')
ax[1].plot(edges,a['energy'][d]/1000,color=BLUE);ax[1].set_ylabel('储电量 / MWh');sub(ax[1],'(b) 实际储能状态')
for row,key,c in ((2,'ch',BLUE),(1,'dis',ORANGE),(0,'emergency',RED)):
    ids=np.flatnonzero(a[key][d]>1e-7)
    ax[2].broken_barh([(v/6,DT) for v in ids],(row-.25,.5),facecolors=c,edgecolors='none')
```

```
ax[2].set(yticks=[0,1,2],yticklabels=['紧急','放电','充电'],xlabel='时刻 / h',xlim=(0,24),xticks=np.arange(0,25,6),ylim=(-.55,2.6));sub(ax[2],'(c) 实际操作区间')
for aa in ax:
    axes_style(aa)
    for xx in (6,12,18):aa.axvline(xx,color=GRAY,lw=.65,ls=':')
save(fig,'q3_typical','q3.npz: 2025-03-20; active intervals use bus-side power > 1e-7 kW')

# 13. 月度差异与逐日累计；不把累积曲线称为迭代收敛。
delta=A['q3_daily']['costs'][start:,4]-A['q3']['costs'][start:,4]
monthly=np.array([delta[months[start:]==m].sum() for m in ms])
assert np.allclose(monthly,[D['update_cost_by_month'][str(m)] for m in ms])
fig,ax=plt.subplots(1,2,figsize=(6.8,2.6),layout='constrained')
ax[0].bar(ms,monthly/1e4,color=BLUE,width=.65,edgecolor='#333333',lw=.35);ax[0].set(xticks=ms,xlabel='月份',ylabel='月度节省 / 万元');sub(ax[0],'(a) 11个月均有净节省')
ax[1].plot(np.arange(1,335),np.cumsum(delta)/1e4,color=BLUE);ax[1].axhline(0,color=GRAY,lw=.65);ax[1].set(xlabel='报告期累计天数',ylabel='累计节省 / 万元');sub(ax[1],'(b) 逐日累计实际费用差')
for aa in ax:axes_style(aa)
metrics['daily_update_saving']={'total_yuan':float(delta.sum()),'min_yuan':float(delta.min()),'max_yuan':float(delta.max()),'more_expensive_days':int((delta<-1e-6).sum())}
save(fig,'update_robustness','q3_daily minus q3 actual daily fees; monthly sums and sequential cumulative sum')

# 14. 电价信息对照。
d=list(dates).index('2025-06-21');fig,ax=plt.subplots(2,1,figsize=(6.8,3.8),sharex=True,layout='constrained')
for arr,c,ls,l in ((I['price'][d],ORANGE,'-','实际波动价'),(I['forecast_price'][d],BLUE,'--','日前预测价'),(Q['price'],GRAY,':','重复日内价')):ax[0].plot(h,arr,color=c,ls=ls,label=l)
ax[0].set_ylabel('电价 / (元/kWh)');ax[0].legend(ncol=3,fontsize=8.5);sub(ax[0],'(a) 价格输入与实际结算价')
for k,c,ls,l in (('q4_3',BLUE,'-','未来价格未知'),('q4_3_known',ORANGE,'--','当天价格已知')):ax[1].plot(edges,A[k]['energy'][d]/1000,color=c,ls=ls,label=l)
ax[1].set(ylabel='储电量 / MWh',xlabel='时刻 / h',xticks=np.arange(0,25,4),xlim=(0,24));ax[1].legend(ncol=2,fontsize=9);sub(ax[1],'(b) 相应的实际储能轨迹')
for aa in ax:axes_style(aa)
save(fig,'q4_price','inputs_processed, q4_3 and q4_3_known; 2025-06-21')

# 15. 所有分项非负；低比例分项通过右图单独展开。
kk=['q2','q3','q4_2','q4_3'];xx=np.arange(4);labels=['问题2','问题3','问题4-2','问题4-3']
components=[('履行原计划',BLUE,lambda a:a['plan_cost']+2*a['down_cost']),('取消违约',YELLOW,lambda a:-a['down_cost']),('上调购电',ORANGE,lambda a:a['up_cost']),('紧急购电',PURPLE,lambda a:a['emergency_cost'])]
fig,ax=plt.subplots(1,2,figsize=(6.8,2.9),layout='constrained');bottom=np.zeros(4);small=np.zeros(4)
for idx,(label,c,fun) in enumerate(components):
    vals=np.array([fun(S['cases'][k])/1e4 for k in kk]);ax[0].bar(xx,vals,bottom=bottom,label=label,color=c,edgecolor='white',linewidth=.25);bottom+=vals
    if idx:ax[1].bar(xx,vals,bottom=small,color=c,label=label,edgecolor='white',linewidth=.25);small+=vals
assert np.allclose(bottom,[S['cases'][k]['total_cost']/1e4 for k in kk])
for i,v in enumerate(bottom):ax[0].text(i,v+14,f'{v:.1f}',ha='center',fontsize=8)
for aa in ax:axes_style(aa);aa.set_xticks(xx,labels,rotation=15);aa.set_ylabel('费用 / 万元')
ax[0].set_ylim(0,max(bottom)*1.38);ax[0].legend(ncol=2,fontsize=7.5,loc='upper left');sub(ax[0],'(a) 全部实际费用');sub(ax[1],'(b) 偏差与紧急费用展开')
save(fig,'annual_costs','summary.json: four nonnegative settlement components, right panel expands three smaller components')

# 16. 参数选择与事后敏感性分栏，以免信息混淆。
fig,ax=plt.subplots(1,2,figsize=(6.8,2.7),layout='constrained')
```

```
windows=[7,14,28];cost=[S['cases'][k]['total_cost']/1e4 for k in ('q3_history7','q3','q3_history28')]
ax[0].plot(windows,cost,'o-',color=BLUE,ms=5);ax[0].plot(14,cost[1],'s',mfc='white',mec=ORANGE,ms=8,label='主模型: 14天');ax[0].set(xticks=windows,xlabel='误差窗口 / 天',ylabel='报告期总费用 / 万元');ax[0].legend(loc='upper left',fontsize=8);sub(ax[0],'(a) 事后窗口敏感性')
for k,c,l in (('q2',BLUE,'重复电价'),('q4_2',ORANGE,'波动电价')):
    score=RF['validation'][k+'_terminal']['scores'];gammas=np.array([0,.5,1,1.5]);yy=np.array([score[str(g)]['adjusted_cost'] for g in gammas]);yy=(yy-yy[0])/10000
    ax[1].plot(gammas,yy,'o-' if k=='q2' else 's--',color=c,label=l,ms=4)
ax[1].axhline(0,color=GRAY,lw=.6);ax[1].set(xticks=[0,.5,1,1.5],xlabel='终端价值强度 γ',ylabel='相对 γ=0 的验证费用差 / 万元');ax[1].legend(fontsize=8);sub(ax[1],'(b) 仅1月份的参数选型')
for aa in ax:axes_style(aa)
save(fig,'parameter_sensitivity','summary.json holdout window costs; refinements.json January inventory-adjusted terminal validation scores')

# 17. 独立扩展相对各自基准，保留不改善项与负面结果。
rows=[('终端价值','q2','q4_2','terminal'),('调整门控','q3','q4_3','gate'),('预报融合','q3','q4_3','blend'),('费用选型','q2','q4_2','economic')]
fig,ax=plt.subplots(figsize=(6.8,2.75),layout='constrained');ys=np.arange(4)
for offset,c,which,l in ((-.15,BLUE,0,'重复电价'),(.15,ORANGE,1,'波动电价')):
    vals=[]
    for title,b1,b2,suffix in rows:
        base=(b1,b2)[which];alt=base+'_'+suffix;allcases={**S['cases'],**RF['cases']}
        if suffix=='economic':base=base+'_mae_control'
        vals.append((allcases[base]['total_cost']-allcases[alt]['total_cost'])/10000)
    ax.barh(ys+offset,vals,height=.28,color=c,label=l,edgecolor='white',lw=.25)
    for y,v in zip(ys+offset,vals):ax.text(v+(.15 if v>=0 else -.15),y,f'{v:.2f}',ha='left' if v>=0 else 'right',va='center',fontsize=8)
ax.axvline(0,color=GRAY,lw=.8);ax.set(yticks=ys,yticklabels=[r[0] for r in rows],xlabel='相对各自基准节省的费用 / 万元',xlim=(-2.2,23));ax.invert_yaxis();axes_style(ax);ax.legend(loc='lower right',fontsize=9)
save(fig,'strategy_gains','Each independent refinement against its own stated baseline; economic selection versus dedicated MAE control; gains cannot be added')

# 18. 分时预报误差，已发布预报比较在相同目标区间内进行。
fig,ax=plt.subplots(figsize=(6.8,2.5),layout='constrained');xx=np.arange(4)
for j,(field,c,l) in enumerate((('historical_same_block',GRAY,'历史预测'),('zero_same_block',BLUE,'零时发布'),('six_hour',ORANGE,'本次发布'))):
    yy=[D['issued_pv'][str(k)][field]['mae'] for k in (0,6,12,18)];ax.bar(xx+(j-1)*.24,yy,width=.22,color=c,label=l,edgecolor='white',lw=.3)
ax.set(xticks=xx,xticklabels=['0–6时','6–12时','12–18时','18–24时'],xlabel='共同目标时段',ylabel='光伏预测 MAE / kW');axes_style(ax);ax.legend(ncol=3,fontsize=9)
save(fig,'forecast_updates','diagnostics.json issued_pv: historical/zero/current publication compared over identical target six-hour blocks')

metrics['trace_sha256']={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(R.glob('*.npz'))}
(R/'figure_metrics.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding='utf-8')
print('Generated',len(metrics['figures']),'verified figure groups.')
```

B.11 独立复核与敏感性计算

```
"""3.7复核补充: 独立LP、计划风险权重、能量守恒与库存估值。

只写 review_* 结果，不修改原27组主模型与对照。权重扫描从相同的
问题2 1月末实际SOC开始；历史预测不变，真实紧急电始终按5倍价结算。
"""
from pathlib import Path
import json, sys, platform
from time import perf_counter
```

```
import numpy as np
import scipy
from scipy.optimize import linprog
from scipy.sparse import eye, diags, hstack, vstack, csr_matrix
from model import DT, EMIN, EMAX, EINIT, PMAX, execute, residual_scenarios,
    settlement_cost, audit

ROOT=Path(__file__).resolve().parents[1]
R=ROOT/'results'

def solve(scenarios,price,initial,rho=5.,eta_c=.9,eta_d=.9,cyclic=False):
    s,n=scenarios.shape
    ident=eye(n,format='csr');zero=csr_matrix((n,n))
    delta=ident-diags(np.ones(n-1),-1,shape=(n,n),format='csr')
    eq=hstack([zero,-eta_c*DT*ident,DT/eta_d*ident,delta,csr_matrix((n,s*n))],format='csr
    ')
    rhs=np.zeros(n);rhs[0]=initial
    tile=vstack([ident]*s,format='csr')
    ub=hstack([-tile,tile,-tile,csr_matrix((s*n,n)),-eye(s*n,format='csr')],format='csr')
    obj=np.r_[price*DT,np.zeros(3*n),np.tile(rho*price*DT/s,s)]
    bounds=[(0,None)]*n+[(0,PMAX)]*(2*n)+[(EMIN,EMAX)]*n+[(0,None)]*(s*n)
    if cyclic:
        bounds[4*n-1]=(initial,initial)
        bounds[4*n:]=[(0,0)]*(s*n)
    fit=linprog(obj,A_eq=eq,b_eq=rhs,A_ub=ub,b_ub=-scenarios.ravel(),bounds=bounds,method
    ='highs')
    if not fit.success:raise RuntimeError(fit.message)
    grid,ch,dis,energy=np.split(fit.x[:4*n],4)
    amount=np.minimum(ch,dis/(eta_c*eta_d));ch=ch-amount;dis=dis-eta_c*eta_d*amount
    assert np.max(abs(eq@fit.x-rhs))<1e-6
    assert np.max(ub@fit.x+scenarios.ravel())<1e-6
    assert np.max(np.minimum(ch,dis))<1e-6
    return dict(grid=grid,ch=ch,dis=dis,energy=energy,objective=float(fit.fun))

def main():
    begin=perf_counter();data=dict(np.load(R/'inputs_processed.npz'))
    forecast={k[9:]:v for k,v in data.items() if k.startswith('forecast_')}
    base=dict(np.load(R/'q2.npz'));price=data['one'][:,0]
    report={'python':platform.python_version(),'numpy':np.__version__,'scipy':scipy.
    __version__,
            'report_period':'2025-02-01 to 2025-12-31','efficiency':[], 'risk_scan':[], '
    inventory':{}}
    for label,ec,ed in [('both_0.9',.9,.9),('round_trip_0.9',np.sqrt(.9),np.sqrt(.9)),('
    charging_loss_only',.9,1.),('discharging_loss_only',1.,.9)]:
        a=solve((data['one'][:,1]-data['one'][:,2])[None,:],price,EINIT,eta_c=ec,eta_d=ed
    ,cyclic=True)
        q=dict(label=label,eta_c=float(ec),eta_d=float(ed),cost=a['objective'],grid_kwh=
    float(a['grid'].sum()*DT))
        report['efficiency'].append(q)
    assert abs(report['efficiency'][0]['cost']-float(np.load(R/'q1.npz')['objective']))<1
    e-6
    reuse='--reuse-scan' in sys.argv
    if reuse:
        previous=json.loads((R/'review_diagnostics.json').read_text(encoding='utf-8'))
        report['risk_scan']=previous['risk_scan'];report['risk5_baseline_max_difference'
    ]=previous['risk5_baseline_max_difference']
    for rho in ([] if reuse else [2,3,5,8]):
        tick=perf_counter();a={k:v.copy() for k,v in base.items()};state=base['energy'
    ][31,0]
        for day in range(31,365):
            scene=residual_scenarios(data,forecast,day,0,history_days=14)
            plan=solve(scene,price,state,rho=rho)['grid']
            actual=execute(plan,data['load'][day],data['pv'][day],state)
```

```
            a['grid'][day]=plan;a['plan'][day]=plan
            for k in actual:a[k][day]=actual[k]
            a['costs'][day]=settlement_cost(plan,plan,actual['emergency'],price)
            state=actual['energy'][-1]
        audit(data,a)
        if rho==5:
            diff=max(float(np.max(abs(a[k]-base[k]))) for k in base)
            assert diff<1e-6,diff
            report['risk5_baseline_max_difference']=diff
        np.savez_compressed(R/f'review_risk{rho}.npz',**a)
        row=dict(rho=rho,cost=float(a['costs'][31:,4].sum()),emergency_kwh=float(a['emergency'][31:].sum()*DT),
                 spill_kwh=float(a['spill'][31:].sum()*DT),start_energy=float(a['energy'][31,0]),end_energy=float(state),seconds=perf_counter()-tick)
        report['risk_scan'].append(row);print(json.dumps(row),flush=True)
    sl=slice(31,None);a=base;pv=data['pv'][sl];load=data['load'][sl]
    # 来源归属约定：光伏优先承担实际负荷与实际充电；其余未利用供给归为已购电。
    # 这是固定轨迹下的归属，不是“即使重新优化也不可避免”的弃光下界。
    pv_spill=np.maximum(pv-load-a['ch'][sl],0)
    purchased_spill=np.maximum(a['spill'][sl]-pv_spill,0)
    assert np.max(abs(pv_spill+purchased_spill-a['spill'][sl]))<1e-6
    loss=((1-.9)*a['ch'][sl]+(1/.9-1)*a['dis'][sl]).sum()*DT
    stock=float(a['energy'][-1,-1]-a['energy'][31,0])
    grid=float((a['grid'][sl]+a['emergency'][sl]).sum()*DT);net=float((load-pv).sum()*DT)
    spill=float(a['spill'][sl].sum()*DT)
    assert abs((grid-net)-(spill+loss+stock))<1e-6
    report['q2_energy']=dict(net_kwh=net,purchased_kwh=grid,excess_percent=(grid/net-1)*100,loss_kwh=float(loss),
        stock_change_kwh=stock,spill_kwh=spill,pv_spill_kwh=float(pv_spill.sum()*DT),
        paid_spill_kwh=float(purchased_spill.sum()*DT),paid_spill_cost=float((purchased_spill*price).sum()*DT),
        spill_emergency_ratio=spill/(a['emergency'][sl].sum()*DT),peak_regular_kw=float(a['grid'][sl].max()),
        peak_load_kw=float(load.max()),days_with_spill=int((a['spill'][sl].sum(axis=1)>1e-6).sum()))
    # 光伏优先满足负荷后，剩余光伏与外购电的充电来源分配仍非唯一。
    # 两端分别优先用外购电/剩余光伏充电；这不是概率置信区间。
    paid_min=np.maximum(a['spill'][sl]-np.maximum(pv-load,0),0)
    paid_max=np.minimum(a['spill'][sl],a['grid'][sl])
    assert np.max(abs(paid_max-purchased_spill))<1e-6
    report['q2_energy'].update(regular_kwh=float(a['grid'][sl].sum()*DT),
        paid_spill_kwh_interval=[float(paid_min.sum()*DT),float(paid_max.sum()*DT)],
        paid_spill_cost_interval=[float((paid_min*price).sum()*DT),float((paid_max*price).sum()*DT)])
    # 一律使用同一内部库存估值区间，包含原稿参考值；端点检查可确定线性差的排序。
    vlo=.9*float(data['price'].min());vhi=.9*float(data['price'].max());vref=.9*.4248
    for case in ['q2','q3','q4_2','q4_3','q4_2_known','q4_3_known']:
        c=dict(np.load(R/(case+'.npz')));delta=float(c['energy'][31,0]-c['energy'][-1,-1]);cost=float(c['costs'][31:,4].sum())
        report['inventory'][case]=dict(cash_cost=cost,start_energy=float(c['energy'][31,0]),end_energy=float(c['energy'][-1,-1]),
            consumed_kwh=delta,reference_adjustment=vref*delta,adjustment_interval=sorted([vlo*delta,vhi*delta]))
    main_cases=['q2','q3','q4_2','q4_3']
    ranks=[[k for k in sorted(main_cases,key=lambda k:report['inventory'][k]['cash_cost']+v*report['inventory'][k]['consumed_kwh'])] for v in [0,vlo,vhi]]
    assert ranks[0]==ranks[1]==ranks[2]
    report['inventory_value_interval']=[vlo,vhi];report['main_cost_order']=ranks[0]
    report['seconds']=perf_counter()-begin
    (R/'review_diagnostics.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps(report['q2_energy'],ensure_ascii=False),flush=True)

```

```
if __name__=='__main__':
    if hasattr(sys.stdout,'reconfigure'):sys.stdout.reconfigure(encoding='utf-8')
    main()
```

B.12 补充图表

```
"""绘制3.7复核补充图表；只使用实际计算记录，沿用论文 MATLAB 配色。"""
from pathlib import Path
import json,hashlib
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager

ROOT=Path(__file__).resolve().parents[1];R=ROOT/'results';F=ROOT/'figures';T=ROOT/'tables'
D=json.loads((R/'review_diagnostics.json').read_text(encoding='utf-8'))
X=json.loads((R/'diagnostics.json').read_text(encoding='utf-8'))
RF=json.loads((R/'refinements.json').read_text(encoding='utf-8'))
for name in ['simsun.ttc','times.ttf']:
    p=Path('C:/Windows/Fonts')/name
    if p.exists():font_manager.fontManager.addfont(str(p))
plt.rcParams.update({'font.family':['Times New Roman','SimSun','DejaVu Sans'],'font.size':10,'axes.labelsize':10,
 'axes.linewidth':.7,'xtick.direction':'in','ytick.direction':'in','axes.unicode_minus':False,
 'pdf.fonttype':42,'savefig.bbox':'tight','figure.facecolor':'white','axes.facecolor':'white',
 'legend.fancybox':False,'legend.edgecolor':'.65','legend.fontsize':9,'grid.linewidth':.45})
B='#0072BD';O='#D95319';Y='#EDB120';P='#7E2F8E';G='#77AC30';C='#4DBEEE'
manifest=[]
def tidy(ax):
    ax.grid(True,color='#D9D9D9',alpha=.75);ax.set_axisbelow(True)
    for s in ax.spines.values():s.set_color('#555555')
def save(fig,name,source):
    fig.savefig(F/(name+'.pdf'));fig.savefig(F/'previews'/(name+'.png'),dpi=175);plt.close(fig)
    manifest.append({'file':name+'.pdf','source':source})
def table(name,caption,label,head,rows,spec):
    lines=[r'\begin{table}[H]\centering\small',r'\caption{'+caption+'}'+r'\label{tab:'+label+'}',r'\begin{tabular}{'+spec+r'}\toprule', ' & '.join(head)+r'\\\midrule']
    lines+=[' & '.join(map(str,row))+r'\\' for row in rows]
    lines += [r'\bottomrule\end{tabular}\end{table}']
    (T/(name+'.tex')).write_text('\n'.join(lines)+'\n',encoding='utf-8')

e=D['q2_energy'];fig,axs=plt.subplots(1,2,figsize=(6.8,2.75),layout='constrained')
ax=axs[0];vals=[e['spill_kwh']/1e4,e['loss_kwh']/1e4,e['stock_change_kwh']/1e4]
ax.bar([0,1,2],vals,color=[O,B,G],width=.6,edgecolor='white')
for i,v in enumerate(vals):ax.annotate(f'{v:.2f}',(i,v),xytext=(0,4 if v>=0 else -12),textcoords='offset points',ha='center',fontsize=9)
ax.axhline(0,color='.4',lw=.7);ax.set(xticks=[0,1,2],xticklabels=['未利用供给','储能损耗','库存净增加'],ylabel='电量 / 万kWh',ylim=(-30,320),title='(a) 总购电超过净负荷的构成');tidy(ax)
ax=axs[1];paid=np.array(e['paid_spill_kwh_interval'])/1e4;pv=e['spill_kwh']/1e4-paid
ax.barh([0,1],pv,color=Y,label='光伏来源');ax.barh([0,1],paid,left=pv,color=B,label='外购来源')
for i in [0,1]:
    ax.text(pv[i]/2,i,f'{pv[i]:.1f}',ha='center',va='center');ax.text(pv[i]+paid[i]/2,i,f'{paid[i]:.1f}',ha='center',va='center',color='white')
```

```
ax.set(yticks=[0,1],yticklabels=['外购优先充电','光伏优先充电'],xlabel='未利用电量 / 万kWh',xlim=(0,300),ylim=(-.7,1.8),title='(b) 光伏先供负荷后的两种来源归属');ax.legend(loc='upper right',ncol=2,fontsize=8);tidy(ax)
save(fig,'unused_energy','review_diagnostics.json q2_energy; source attribution conditional on PV-first load; no new cash fee')

rows=D['risk_scan'];xx=np.arange(4);fig,axs=plt.subplots(1,2,figsize=(6.8,2.7),layout='constrained')
ax=axs[0];cost=np.array([r['cost'] for r in rows])/1e4;ax.plot(xx,cost,'o-',color=B,lw=1.3)
for i,v in enumerate(cost):ax.annotate(f'{v:.2f}',(i,v),xytext=(0,9),textcoords='offset points',ha='center',fontsize=8)
ax.set(xticks=xx,xticklabels=[r['rho'] for r in rows],xlabel='计划阶段惩罚权重 ρ',ylabel='实际总费用 / 万元',xlim=(-.3,3.35),ylim=(1370,1510),title='(a) 实际费用在四组中以ρ=5最低');tidy(ax)
ax=axs[1];spill=np.array([r['spill_kwh'] for r in rows])/1e4;em=np.array([r['emergency_kwh'] for r in rows])/1e4
ax.plot(spill,em,'s--',color=O,lw=1.2)
for row,x,y in zip(rows,spill,em):ax.annotate('ρ='+str(row['rho']),(x,y),xytext=(6,5),textcoords='offset points',fontsize=9)
ax.set(xlabel='未利用电量 / 万kWh',ylabel='紧急电量 / 万kWh',xlim=(140,380),ylim=(0,51),title='(b) 更保守的计划减少缺额、增加弃电');tidy(ax)
save(fig,'risk_tradeoff','review_risk{2,3,5,8}.npz; common 2025-02-01 initial SOC; actual emergency price remains 5 times')
table('review_risk','仅改变计划惩罚权重的连续回测（实际紧急电仍按5倍价）','review_risk',
 ['计划权重','总费用/万元','未利用电量/万kWh','紧急电量/万kWh'],
 [[r['rho'],f"{r['cost']/1e4:.2f}",f"{r['spill_kwh']/1e4:.2f}",f"{r['emergency_kwh']/1e4:.2f}"] for r in rows],'rrrr')

eff=D['efficiency'];labels=['单程均0.9（主设定）','回路0.9，对称分配','仅充电有损耗','仅放电有损耗']
fig,ax=plt.subplots(figsize=(6.8,2.35),layout='constrained');yy=np.arange(4);vals=[r['cost']/1e4 for r in eff]
ax.barh(yy,vals,color=[B,O,G,P],height=.57,edgecolor='white')
for i,(r,v) in enumerate(zip(eff,vals)):ax.text(v+.035,i,f"{r['cost']:.2f} 元",va='center',fontsize=9)
ax.set(yticks=yy,yticklabels=labels,xlabel='附件1单日购电费用 / 万元',xlim=(0,4.5));ax.invert_yaxis();tidy(ax)
save(fig,'efficiency_interpretation','Independent LP in review_experiments.py, same attachment1 and same capacity and endpoints; different efficiencies')
table('review_efficiency','效率解释改变问题1费用，主设定维持两侧各0.9','review_efficiency',
 ['效率解释',r'$\eta_c$',r'$\eta_d$','费用/元','较主设定变化'],
 [[l,f"{r['eta_c']:.4f}",f"{r['eta_d']:.4f}",f"{r['cost']:.2f}",f"{(r['cost']/eff[0]['cost']-1)*100:.2f}"+r'\%'] for l,r in zip(labels,eff)],'lrrrr')

sh=X['scenario_shuffle'];fig,axs=plt.subplots(1,2,figsize=(6.8,2.55),layout='constrained')
ax=axs[0];delta=[sh['objective_difference'],sh['permuted_feedback_cost']-sh['original_feedback_cost']]
ax.bar([0,1],delta,color=[B,O],width=.5)
for i,v in enumerate(delta):ax.text(i,v+8,'约0' if i==0 else f'{v:.2f}',ha='center',fontsize=9)
ax.set(xticks=[0,1],xticklabels=['代理目标变化','实际反馈费用变化'],ylabel='打乱后减去原场景 / 元',ylim=(-10,260),title='(a) 9月23日时间配对诊断');tidy(ax)
ax=axs[1];gate=RF['cases']['q3_gate'];acc=gate['accepted'];rej=gate['rejected']
ax.barh([0,1],[acc,rej],color=[B,Y],height=.55)
for i,v in enumerate([acc,rej]):ax.text(v+10,i,str(v),va='center',fontsize=9)
ax.set(yticks=[0,1],yticklabels=['采纳候选','保留原安排'],xlabel='更新机会数',xlim=(0,750),title='(b) 重复电价下门控决策（1002次）');ax.invert_yaxis();tidy(ax)
save(fig,'proxy_and_gate','diagnostics.json scenario_shuffle and refinements.json q3_gate; separate diagnostics, not causal relation')

```

```
report={'figures':manifest,'source_hashes':{p.name:hashlib.sha256(p.read_bytes()).
    hexdigest() for p in [R/'review_diagnostics.json',R/'diagnostics.json',R/'refinements
    .json']}}
(R/'review_figure_metrics.json').write_text(json.dumps(report,ensure_ascii=False,indent
    =2),encoding='utf-8')
print('Generated 4 supplementary figure groups and 2 tables.')
```

B.13 日内执行上界对照

```
"""补充：同一计划下的日内执行上界（问题2）。

固定已提交的问题2计划购电量与实际负荷、光伏，把日内储能执行由正文的因果
反馈规则改为"当日完全预知的最优调度"，逐日链式传递实际储电量，用于界定
第10.1.2节"风险代理与执行反馈不一致"这一局限的费用量级。

该上界要求当日负荷与光伏在零时即完全已知，不可实现，只是量级参照，不替换
任何主结果，也不写入五份规定工作簿。
"""
from pathlib import Path
import json, platform
import numpy as np, scipy
from scipy.optimize import linprog
from model import DT, EMIN, EMAX, PMAX

ROOT = Path(__file__).resolve().parents[1]
R = ROOT / 'results'
FIRST, LAST = 31, 365          # 0 基日索引：31 = 2025-02-01，364 = 2025-12-31
INVENTORY_PRICE = 'one_first6h'   # 统一库存估值：附件1零至6时均价乘放电效率


def oracle_day(plan, load, pv, price, initial, eta_c=0.9, eta_d=0.9):
    """给定当日计划购电功率，求最小实际紧急购电费用的日内执行安排。"""
    n = plan.size
    # 变量顺序：[ch, dis, em, sp, E_0..E_n]
    def ix(kind, t):
        return {'ch': 0, 'dis': n, 'em': 2 * n, 'sp': 3 * n, 'E': 4 * n}[kind] + t

    obj = np.zeros(4 * n + n + 1)
    for t in range(n):
        obj[ix('em', t)] = 5.0 * price[t] * DT

    a_eq, b_eq = [], []
    for t in range(n):                          # 母线平衡：计划已定，缺额只能紧急补电
        row = np.zeros(obj.size)
        row[ix('dis', t)] = 1.0
        row[ix('em', t)] = 1.0
        row[ix('ch', t)] = -1.0
        row[ix('sp', t)] = -1.0
        a_eq.append(row)
        b_eq.append(load[t] - pv[t] - plan[t])
    for t in range(1, n + 1):                   # 储能动态
        row = np.zeros(obj.size)
        row[ix('E', t)] = 1.0
        row[ix('E', t - 1)] = -1.0
        row[ix('ch', t - 1)] = -eta_c * DT
        row[ix('dis', t - 1)] = DT / eta_d
        a_eq.append(row)
        b_eq.append(0.0)

    bounds = [(0, PMAX)] * (2 * n) + [(0, None)] * (2 * n) + [(EMIN, EMAX)] * (n + 1)
    bounds[ix('E', 0)] = (initial, initial)
    fit = linprog(obj, A_eq=np.array(a_eq), b_eq=np.array(b_eq), bounds=bounds, method='
    highs')
```

```
    if not fit.success:
        raise RuntimeError(fit.message)
    return fit


def main():
    data = dict(np.load(R / 'inputs_processed.npz'))
    base = np.load(R / 'q2.npz')
    one = data['one']
    price = one[:, 0]
    load, pv = data['load'], data['pv']

    initial = float(base['energy'][FIRST, 0])
    cash_emergency = 0.0
    emergency_kwh = 0.0
    spill_kwh = 0.0
    daily = []
    for day in range(FIRST, LAST):
        fit = oracle_day(base['plan'][day], load[day], pv[day], price, initial)
        n = load.shape[1]
        em = fit.x[2 * n:3 * n]
        ch = fit.x[0:n]
        dis = fit.x[n:2 * n]
        sp = fit.x[3 * n:4 * n]
        energy = fit.x[4 * n:5 * n + 1]
        cash_emergency += float((5.0 * price * em * DT).sum())
        emergency_kwh += float(em.sum() * DT)
        spill_kwh += float(sp.sum() * DT)
        initial = float(energy[-1])
        daily.append({'date_index': day, 'emergency_kwh': float(em.sum() * DT),
                      'end_energy': initial})

    plan_cash = float(base['costs'][FIRST:LAST, 0].sum())      # 与主结果一致的计划费
    base_total = float(base['costs'][FIRST:LAST, 4].sum())     # 主结果报告期现金费用
    base_end = float(base['energy'][LAST - 1, 144])
    v = float(np.mean(one[:36, 0])) * 0.9                      # 统一库存参考价值
    oracle_cash = plan_cash + cash_emergency
    adj_oracle = oracle_cash + v * float(base['energy'][FIRST, 0]) - v * initial
    adj_base = base_total + v * float(base['energy'][FIRST, 0]) - v * base_end

    report = {
        'purpose': '同一计划、同一实际负荷与光伏下，日内执行改为当日完全预知的最优调度',
        'report_period': '2025-02-01 to 2025-12-31',
        'python': platform.python_version(), 'numpy': np.__version__, 'scipy': scipy.
    __version__,
        'inventory_value_yuan_per_kwh': v,
        'base_total_cost': base_total, 'base_end_energy': base_end,
        'oracle_total_cost': oracle_cash, 'oracle_end_energy': initial,
        'cash_saving': base_total - oracle_cash,
        'inventory_adjusted_base': adj_base, 'inventory_adjusted_oracle': adj_oracle,
        'inventory_adjusted_saving': adj_base - adj_oracle,
        'saving_percent_of_base': (base_total - oracle_cash) / base_total * 100.0,
        'oracle_emergency_kwh': emergency_kwh, 'base_emergency_kwh':
            float(base['emergency'][FIRST:LAST].sum() * DT),
        'oracle_spill_kwh': spill_kwh,
        'base_spill_kwh': float(base['spill'][FIRST:LAST].sum() * DT),
        'daily': daily,
    }
    (R / 'review_oracle_execution.json').write_text(
        json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
    print(json.dumps({k: v for k, v in report.items() if k != 'daily'},
                     ensure_ascii=False, indent=2))

```

```

if __name__ == '__main__':
    main()
```

B.14 交付物一致性核验

```
"""交付物一致性核验：论文表格 / 结果工作簿 / 结果数组三者互相对账。

只读取 结果/、tables/、results/ 中的既有文件，不做任何计算或写回，
因此可以独立于建模程序反复运行。运行方式（在工程根目录）：

    python code/verify_consistency.py

核验项：
1. 每份多日工作簿中，144 个时段列之和等于"全天购电量"列；
2. 问题3、问题4-3的"计划购电量"与"调整购电量"两张表的"全天购电费"逐日相同；
3. 工作簿汇总（天数、购电量、费用、紧急电量）等于 results/summary.json；
4. results/daily_summary.csv 的每个方案汇总等于 results/summary.json；
5. 论文指定时段表 tables/*_purchase.tex 的时段值与全天值等于工作簿对应单元；
6. 问题1工作簿的时段值、全天购电量等于 tables/q1_purchase.tex 与 summary.json；
7. 摘要与正文出现的关键数值，逐条等于其数据源（工作簿、summary.json、
   review_diagnostics.json、refinement_metrics.json）算出的值。
"""
from __future__ import annotations

import csv
import json
import re
from datetime import datetime
from pathlib import Path

import openpyxl

ROOT = Path(__file__).resolve().parent.parent
TOL = 5e-3          # 表格按两位小数排版，取半个末位的容差
SLOTS = ["10:00--10:10", "12:00--12:10", "14:00--14:10",
         "16:00--16:10", "18:00--18:10", "20:00--20:10"]

# 工作簿 -> (逐时工作表, 典型日表格前缀)
CASES = {
    "q2":     ("result2.xlsx",   "计划购电量", "q2"),
    "q3":     ("result3.xlsx",   "调整购电量", "q3"),
    "q4_2":   ("result4-2.xlsx", "计划购电量", "q4_2"),
    "q4_3":   ("result4-3.xlsx", "调整购电量", "q4_3"),
}
TYPICAL_DAYS = ["0320", "0621", "0923", "1221"]
failures: list[str] = []
checks = 0


def check(ok: bool, message: str) -> None:
    global checks
    checks += 1
    if not ok:
        failures.append(message)


def col_of(label: str) -> int:
    """'0:00--0:10' 对应第 2 列。"""
    hour, minute = label.split("--")[0].split(":")
    return 2 + int(hour) * 6 + int(minute) // 10


```

```python
def parse_purchase_table(path: Path) -> tuple[dict[str, float], float, float]:
    text = path.read_text(encoding="utf-8")
    slots = {f"{a}--{b}": float(v)
             for a, b, v in re.findall(r"(\d\d:\d\d)--(\d\d:\d\d) & ([\d.]+)", text)}
    day = float(re.search(r"全天常规购电量: ([\d.]+)", text).group(1))
    cost = float(re.search(r"全天总费用: ([\d.]+)", text).group(1))
    return slots, day, cost


def row_of(ws, date: datetime) -> int:
    for r in range(2, ws.max_row + 1):
        if ws.cell(r, 1).value == date:
            return r
    raise AssertionError(f"{ws.title} 缺少日期 {date:%Y-%m-%d}")


def main() -> int:
    summary = json.loads((ROOT / "results/summary.json").read_text(encoding="utf-8"))
    cases = summary["cases"]

    # ---- 1、2、3、5: 工作簿内部与工作簿同结果数组 ----
    for name, (book, sheet, prefix) in CASES.items():
        wb = openpyxl.load_workbook(ROOT / "结果" / book, data_only=True)
        ws = wb[sheet]
        energy = cost = emergency = 0.0
        days = 0
        worst_row = 0.0
        for r in range(2, ws.max_row + 1):
            if ws.cell(r, 1).value is None:
                continue
            days += 1
            row_sum = sum((ws.cell(r, c).value or 0.0) for c in range(2, 146))
            worst_row = max(worst_row, abs(row_sum - (ws.cell(r, 146).value or 0.0)))
            energy += ws.cell(r, 146).value or 0.0
            cost += ws.cell(r, 147).value or 0.0
        check(worst_row < 1e-6, f"{book}/{sheet}: 逐时和与全天购电量最大差 {worst_row:.2e}")
        check(days == 334, f"{book}/{sheet}: 报告日数 {days} ≠ 334")

        if "调整购电量" in wb.sheetnames:
            plan, adj = wb["计划购电量"], wb["调整购电量"]
            worst = max(abs((plan.cell(r, 147).value or 0.0) - (adj.cell(r, 147).value or
 0.0))
                        for r in range(2, plan.max_row + 1) if plan.cell(r, 1).value is
not None)
            check(worst < 1e-6, f"{book}: 两表全天购电费最大差 {worst:.2e}（不应相加）")

        ws_em = wb["紧急购电量"]
        for r in range(2, ws_em.max_row + 1):
            v = ws_em.cell(r, 3).value
            if isinstance(v, (int, float)):
                emergency += v
        exp = cases[name]
        check(abs(cost - exp["total_cost"]) <= TOL,
              f"{book}: 费用合计 {cost:.2f} ≠ summary.json {exp['total_cost']:.2f}")
        check(abs(energy - exp["grid_kwh"]) <= TOL,
              f"{book}: 购电量合计 {energy:.2f} ≠ summary.json {exp['grid_kwh']:.2f}")
        check(abs(emergency - exp["emergency_kwh"]) <= TOL,
              f"{book}: 紧急电量合计 {emergency:.2f} ≠ summary.json {exp['emergency_kwh
']:.2f}")

        for d in TYPICAL_DAYS:
            date = datetime(2025, int(d[:2]), int(d[2:]))
            r = row_of(ws, date)
```

```
            slots, tab_day, tab_cost = parse_purchase_table(
                ROOT / "tables" / f"{prefix}_{d}_purchase.tex")
            for label in SLOTS:
                got = ws.cell(r, col_of(label)).value or 0.0
                check(abs(got - slots[label]) <= TOL,
                      f"{book}/{sheet} {date:%Y-%m-%d} {label}: 表 {slots[label]:.2f} 簿 {got:.2f}")
            check(abs((ws.cell(r, 146).value or 0.0) - tab_day) <= TOL,
                  f"{book}/{sheet} {date:%Y-%m-%d}: 全天购电量 表 {tab_day:.2f} 簿 {ws.cell(r,146).value:.2f}")
            check(abs((ws.cell(r, 147).value or 0.0) - tab_cost) <= TOL,
                  f"{book}/{sheet} {date:%Y-%m-%d}: 全天总费用 表 {tab_cost:.2f} 簿 {ws.cell(r,147).value:.2f}")

    # ---- 4: daily_summary.csv 同 summary.json ----
    totals = {k: 0.0 for k in cases}
    em = {k: 0.0 for k in cases}
    rows = list(csv.DictReader((ROOT / "results/daily_summary.csv").open(encoding="utf-8-sig")))
    csv_cases = {row["case"] for row in rows}
    for row in rows:
        if row["case"] in totals:
            totals[row["case"]] += float(row["total_cost"])
            em[row["case"]] += float(row["emergency_kwh"])
    for name in sorted(csv_cases & set(cases)):
        check(abs(totals[name] - cases[name]["total_cost"]) <= TOL,
              f"daily_summary.csv {name}: 费用 {totals[name]:.2f} ≠ {cases[name]['total_cost']:.2f}")
        check(abs(em[name] - cases[name]["emergency_kwh"]) <= TOL,
              f"daily_summary.csv {name}: 紧急电量 {em[name]:.2f} ≠ {cases[name]['emergency_kwh']:.2f}")

    # ---- 6: 问题1 ----
    wb1 = openpyxl.load_workbook(ROOT / "结果/result1.xlsx", data_only=True)
    ws1 = wb1["计划购电量"]
    values = {ws1.cell(r, 1).value: (ws1.cell(r, 2).value or 0.0) for r in range(2, 146)}
    slots, tab_day, tab_cost = parse_purchase_table(ROOT / "tables/q1_purchase.tex")
    day = sum(values.values())
    check(abs(day - tab_day) <= TOL, f"result1: 全天购电量 {day:.2f} ≠ 表 {tab_day:.2f}")
    check(abs(day - summary["q1"]["grid_kwh"]) <= TOL,
          f"result1: 全天购电量 {day:.2f} ≠ summary.json {summary['q1']['grid_kwh']:.2f}")
    check(abs(tab_cost - summary["q1"]["objective"]) <= TOL,
          f"问题1表: 全天费用 {tab_cost:.2f} ≠ summary.json {summary['q1']['objective']:.2f}")
    for label in SLOTS:
        a, b = label.split("--")
        got = values[f"{a}-{b}"]
        check(abs(got - slots[label]) <= TOL, f"result1 {label}: 表 {slots[label]:.2f} 簿 {got:.2f}")
    ws2 = wb1["充放电量"]
    ch = sum((ws2.cell(r, 2).value or 0.0) for r in range(2, 8))
    dis = sum((ws2.cell(r, 3).value or 0.0) for r in range(2, 8))
    check(abs(ch - summary["q1"]["ch_kwh"]) <= TOL, f"result1: 充电合计 {ch:.2f} ≠ {summary['q1']['ch_kwh']:.2f}")
    check(abs(dis - summary["q1"]["dis_kwh"]) <= TOL, f"result1: 放电合计 {dis:.2f} ≠ {summary['q1']['dis_kwh']:.2f}")

    # ---- 7: 摘要与正文文字数值同数据源对账 ----
    # 摘要与正文的关键数字均以字面值排版，此处按数据源重算后逐条比对，
    # 防止正文数字与工作簿、结果数组之间发生漂移。
    diag = json.loads((ROOT / "results/review_diagnostics.json").read_text(encoding="utf-8"))
```

```
    refi = json.loads((ROOT / "results/refinement_metrics.json").read_text(encoding="utf-8"))
    dgn = json.loads((ROOT / "results/diagnostics.json").read_text(encoding="utf-8"))
    tex = (ROOT / "main.tex").read_text(encoding="utf-8")
    body = tex.split("\\section*{附录A", 1)[0]
    abstract = tex.split("\\end{center}", 1)[1].split("\\newpage", 1)[0]
    risk = {r["rho"]: r for r in diag["risk_scan"]}
    q1v = summary["q1"]
    q2c, q3c = cases["q2"], cases["q3"]
    q3d, q2n = cases["q3_daily"], cases["q2_nominal"]
    q42, q43 = cases["q4_2"], cases["q4_3"]
    q42k, q43k = cases["q4_2_known"], cases["q4_3_known"]

    def wan(x: float) -> str:
        return f"{x / 1e4:.2f}"

    saving3 = q3d["total_cost"] - q3c["total_cost"]
    claims = [
        ("摘要-问题1全天购电量", f"{q1v['grid_kwh']:.2f} kWh", "summary.json:q1.grid_kwh"),
        ("摘要-问题1电费", f"{q1v['objective']:.2f}元", "summary.json:q1.objective"),
        ("摘要-问题1无储能对照", f"{q1v['net_cost']:.2f}元", "summary.json:q1.net_cost"),
        ("摘要-问题1降幅", f"{(q1v['net_cost'] - q1v['objective']) / q1v['net_cost'] * 100:.2f}\\%", "q1: (net-objective)/net"),
        ("正文-问题1节省额", f"{q1v['net_cost'] - q1v['objective']:.2f}元", "q1: net-objective"),
        ("摘要-问题2总费用", f"{wan(q2c['total_cost'])}万元", "cases.q2.total_cost"),
        ("摘要-问题2紧急电量", f"{q2c['emergency_kwh'] / 1e4:.2f}万kWh", "cases.q2.emergency_kwh"),
        ("摘要-问题2相对点预测", f"{(q2n['total_cost'] - q2c['total_cost']) / q2n['total_cost'] * 100:.2f}\\%", "cases.q2_nominal"),
        ("摘要-问题2未利用供给", f"{diag['q2_energy']['spill_kwh'] / 1e4:.2f}万kWh", "review_diagnostics:q2_energy.spill_kwh"),
        ("摘要-权重8紧急电量", f"{risk[8]['emergency_kwh'] / 1e4:.2f}万kWh", "review_diagnostics:risk_scan[rho=8]"),
        ("摘要-权重8费用增量", f"{(risk[8]['cost'] - q2c['total_cost']) / 1e4:.2f}万元", "risk_scan[rho=8] - cases.q2"),
        ("正文-权重2总费用", f"{risk[2]['cost'] / 1e4:.2f}万元", "review_diagnostics:risk_scan[rho=2]"),
        ("摘要-问题3零时基准", f"{wan(q3d['total_cost'])}万元", "cases.q3_daily.total_cost"),
        ("摘要-问题3更新后", f"{wan(q3c['total_cost'])}万元", "cases.q3.total_cost"),
        ("摘要-问题3节省额", f"{saving3 / 1e4:.2f}万元", "cases.q3_daily - cases.q3"),
        ("摘要-问题3节省率", f"{saving3 / q3d['total_cost'] * 100:.2f}\\%", "saving3 / q3_daily"),
        ("摘要-问题3基准紧急电量", f"{q3d['emergency_kwh'] / 1e4:.2f}万", "cases.q3_daily.emergency_kwh"),
        ("摘要-问题3紧急电量", f"{q3c['emergency_kwh'] / 1e4:.2f}万kWh", "cases.q3.emergency_kwh"),
        ("摘要-问题4日前", f"{wan(q42['total_cost'])}万元", "cases.q4_2.total_cost"),
        ("摘要-问题4滚动", f"{wan(q43['total_cost'])}万元", "cases.q4_3.total_cost"),
        ("摘要-问题4价格已知日前", f"{wan(q42k['total_cost'])}万元", "cases.q4_2_known.total_cost"),
        ("摘要-问题4价格已知滚动", f"{wan(q43k['total_cost'])}万元", "cases.q4_3_known.total_cost"),
        ("摘要-问题4日前信息价值", f"{(q42['total_cost'] - q42k['total_cost']) / q42['total_cost'] * 100:.2f}\\%", "q4_2 vs q4_2_known"),
        ("摘要-问题4滚动信息价值", f"{(q43['total_cost'] - q43k['total_cost']) / q43['total_cost'] * 100:.2f}\\%", "q4_3 vs q4_3_known"),
        ("正文-问题4日前差额", f"{wan(q42['total_cost'] - q42k['total_cost'])}万元", "q4_2 - q4_2_known"),
        ("正文-问题4滚动差额", f"{wan(q43['total_cost'] - q43k['total_cost'])}万元", "q4_3 - q4_3_known"),
        ("摘要-问题4等效循环", re.compile("由约?{:d}、{:d}次升[至到]约?{:d}、{:d}次".format(
            round(q42["equivalent_cycles"]), round(q43["equivalent_cycles"]),
```

```
            round(q42k["equivalent_cycles"]), round(q43k["equivalent_cycles"]))),
         "cases[*].equivalent_cycles"),
        ("正文-费用选型对照", f"{refi['q2economic_increase']:.0f}元", "refinement_metrics:
    q2economic_increase"),
        ("正文-费用选型对照(价格)", f"{refi['q4_2economic_increase']:.0f}元", "
    refinement_metrics:q4_2economic_increase"),
        ("正文-更新后逐日盈亏", f"{dgn['update_days']['more_expensive']}天更新后更贵，"
                              f"{dgn['update_days']['cheaper']}天更便宜", "diagnostics.json
    :update_days"),
        ("正文-更新试验月份数", f"{len(dgn['update_cost_by_month'])}个月", "
    review_diagnostics:update_cost_by_month"),
    ]
    for label, literal, source in claims:
        where = "摘要" if label.startswith("摘要") else "正文"
        target = abstract if where == "摘要" else body
        found = bool(literal.search(target)) if isinstance(literal, re.Pattern) else
    literal in target
        check(found, f"{label}: {where}未见 {literal!r}（数据源 {source}）")

    only_summary = sorted(set(cases) - csv_cases)
    print(f"核验项：{checks}    失败：{len(failures)}")
    if only_summary:
        print(f"说明：{len(only_summary)} 个敏感性/对照方案只存于 summary.json，未逐日写入
    daily_summary.csv：{only_summary}")
    for f in failures:
        print("  [不一致]", f)
    if not failures:
        print("全部一致：摘要与正文数值、论文指定表、结果工作簿、results/*.json 与 daily_summary.
    csv 互相对账通过。")
    return 1 if failures else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

B.15 紧急购电分布图

```
"""图10：问题2与问题3紧急购电的月内小时三维分布。

只读取 results/ 中已核对的 emergency 轨迹与 summary.json 的报告期日期，按月份与
小时汇总为 MWh 后绘图；不重新求解，不改写任何主结果。运行方式（工程根目录）：

    python code/emergency_surface.py
"""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import colors, font_manager
from mpl_toolkits.mplot3d.art3d import Line3DCollection

import scientific_style as style

ROOT = Path(__file__).resolve().parents[1]
R = ROOT / 'results'
F = ROOT / 'figures'
DT = 1 / 6                      # 每个时段 10 分钟，换算成小时
MONTHS = np.arange(2, 13)       # 报告期为 2—12 月
HOURS = np.arange(24)
CASES = (('q2', '问题2'), ('q3', '问题3'))

```

```
S = json.loads((R / 'summary.json').read_text(encoding='utf-8'))
months = np.array([int(d[5:7]) for d in S['dates']])
TITLE_FONT = 'SimHei'
MESH_COLOR = '#3165D6'      # 网格线固定蓝，与论文其他线图一致
MARKER_CMAP = 'turbo'     # 顶点按高度着色：低值暗蓝、峰值红，便于读出峰值位置


def register_cjk():
    """Windows 上用 SimSun/SimHei；其他平台退回本机可用的中文字体。"""
    global TITLE_FONT
    have = {f.name for f in font_manager.fontManager.ttflist}
    body = next((n for n in ('SimSun', 'AR PL SungtiL GB', 'Noto Serif CJK SC',
                             'WenQuanYi Zen Hei', 'Droid Sans Fallback') if n in have),
    None)
    # Droid Sans Fallback 的中文可显示，但无法被子集化嵌入 PDF，故不作为首选。
    title = next((n for n in ('SimHei', 'AR PL SungtiL GB', 'WenQuanYi Zen Hei',
                              'AR PL UMing CN') if n in have), None)
    if body:
        plt.rcParams['font.family'] = [body, 'DejaVu Sans']
    if title:
        TITLE_FONT = title
    print(f'中文字体：正文 {body}，标题 {TITLE_FONT}')


def monthly_hourly(key):
    """把 (365,144) 的 kW 轨迹按月、小时汇总为 (11,24) 的 MWh。"""
    em = np.load(R / f'{key}.npz')['emergency']
    out = np.zeros((len(MONTHS), len(HOURS)))
    for i, m in enumerate(MONTHS):
        sel = months == m
        if not sel.any():
            continue
        out[i] = em[sel].reshape(-1, len(HOURS), 6).sum(axis=(0, 2)) * DT / 1000
    return out


def add_mesh(ax, z, vmax):
    """网格线固定蓝色；顶点按高度着色，便于直接读出峰值位置。"""
    segments, values = [], []
    for i in range(z.shape[0]):
        for j in range(z.shape[1] - 1):
            segments.append([(HOURS[j], MONTHS[i], z[i, j]),
                             (HOURS[j + 1], MONTHS[i], z[i, j + 1])])
    for i in range(z.shape[0] - 1):
        for j in range(z.shape[1]):
            segments.append([(HOURS[j], MONTHS[i], z[i, j]),
                             (HOURS[j], MONTHS[i + 1], z[i + 1, j])])
    norm = colors.Normalize(0, vmax)
    ax.add_collection3d(Line3DCollection(segments, colors=MESH_COLOR,
                                         linewidths=.5, alpha=.95))
    # 逐格标记，白描边保证与网格线区分。
    xx, yy = np.meshgrid(HOURS, MONTHS)
    ax.scatter(xx.ravel(), yy.ravel(), z.ravel(), c=z.ravel(), cmap=MARKER_CMAP,
               norm=norm, s=9, edgecolors='white', linewidths=.25,
               depthshade=False, zorder=8)


def draw(ax, z, label, vmax, ztop):
    add_mesh(ax, z, vmax)
    ax.set(xlim=(0, 24), ylim=(2, 12), zlim=(0, ztop),
           xticks=[0, 6, 12, 18, 24], yticks=[2, 4, 6, 8, 10, 12],
           zticks=[0, 3, 6, 9])
    ax.set_xlabel('小时区间 / h', fontsize=9, labelpad=4)
```

```
    ax.set_ylabel('月份', fontsize=9, labelpad=6)
    ax.set_zlabel('月内该小时紧急电量 / MWh', fontsize=9, labelpad=5)
    ax.tick_params(labelsize=8, pad=1)
    ax.view_init(elev=22, azim=-57)
    ax.set_proj_type('ortho')
    ax.set_box_aspect((1.45, 1, .72))
    ax.set_title(label, fontproperties=font_manager.FontProperties(family=TITLE_FONT,
    size=10.5), pad=2)
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.pane.set_facecolor((1, 1, 1, 1)); axis.pane.set_edgecolor('#E0E0E0')
        axis._axinfo['grid'].update(color='#E2E4E6', linewidth=.45)
        axis.line.set_color('#929292'); axis.line.set_linewidth(.6)


def main():
    style.configure()
    register_cjk()
    data = {k: monthly_hourly(k) for k, _ in CASES}
    for (k, _), z in zip(CASES, data.values()):
        want = S['cases'][k]['emergency_kwh'] / 1000
        assert abs(z.sum() - want) < 5e-3, (k, z.sum(), want)
        print(f'{k}: 总量 {z.sum():.2f} MWh  峰值 {z.max():.2f} MWh')
    vmax = max(z.max() for z in data.values())
    ztop = np.ceil(vmax)
    fig = plt.figure(figsize=(7.6, 3.4))
    for j, (k, name) in enumerate(CASES):
        ax = fig.add_subplot(1, 2, j + 1, projection='3d')
        z = data[k]
        draw(ax, z, f'({chr(97 + j)}) {name}  |  总量 {z.sum():.2f} MWh', vmax, ztop)
    fig.subplots_adjust(left=.01, right=.99, top=.99, bottom=.02, wspace=.06)
    style.export(fig, F, 'emergency_heatmap')
    print('已写出 figures/emergency_heatmap.pdf 及预览、SVG、600 dpi PNG')


if __name__ == '__main__':
    main()
```

B.16　绘图公共样式

```
"""Origin/MATLAB presentation style. Geometry comes only from stored data.

White canvas, light grid, restrained categorical colours, vector text and marks.
The 3-D seasonal panels connect observed net loads; no fitted plane is invented.
"""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager, colors
from matplotlib.colors import LinearSegmentedColormap
from mpl_toolkits.mplot3d.art3d import Line3DCollection

PALETTE=['#1464A0','#CE4938','#25926B','#7554A3','#C99A26','#2A9EAD','#8C405D']
SURFACE=LinearSegmentedColormap.from_list('scientific_mesh',
 ['#5034A4','#3165D6','#269DD9','#24BBAF','#70CB80','#CCD44D','#F0C63C'])

def configure():
    for name in ('arial.ttf','arialbd.ttf','simsun.ttc','simhei.ttf','times.ttf'):
        p=Path('C:/Windows/Fonts')/name
        if p.exists():font_manager.fontManager.addfont(str(p))
    plt.rcParams.update({
        'font.family':['Arial','SimSun','DejaVu Sans'],'font.size':10,
        'axes.labelsize':10.5,'axes.titlesize':10.5,'axes.titleweight':'normal',
        'axes.linewidth':.85,'axes.edgecolor':'#505050',
```

```
        'xtick.direction':'in','ytick.direction':'in','xtick.major.width':.8,
        'ytick.major.width':.8,'xtick.major.size':3.8,'ytick.major.size':3.8,
        'xtick.labelsize':9,'ytick.labelsize':9,'axes.unicode_minus':False,
        'lines.linewidth':1.5,'lines.markersize':4.5,
        'figure.facecolor':'white','axes.facecolor':'white','savefig.facecolor':'white',
        'legend.fontsize':8.5,'legend.fancybox':False,'legend.framealpha':.97,
        'legend.edgecolor':'#C9C9C9','legend.borderpad':.35,
        'grid.color':'#E4E6E8','grid.linewidth':.5,'grid.alpha':1.,
        'pdf.fonttype':42,'ps.fonttype':42,'svg.fonttype':'none',
        'savefig.bbox':'tight','savefig.pad_inches':.08})

def finish_axes(fig):
    for ax in fig.axes:
        if ax.name=='3d' or not ax.axison:continue
        ax.title.set_fontproperties(font_manager.FontProperties(family='SimHei',size
    =10.5))
        for line in ax.get_xgridlines()+ax.get_ygridlines():
            line.set_color('#E4E6E8');line.set_linewidth(.5);line.set_alpha(1.)
        for spine in ax.spines.values():
            spine.set_linewidth(.85);spine.set_color('#505050')
        for ln in ax.lines:
            if ln.get_marker() not in ('None','none','',None,'|','_'):
                ln.set_markeredgewidth(.8)
                if ln.get_marker() in ('o','s','D','^','v'):ln.set_markerfacecolor('white
    ')
        leg=ax.get_legend()
        if leg:
            leg.get_frame().set_linewidth(.55);leg.get_frame().set_edgecolor('#C9C9C9')

def export(fig,folder,name):
    folder=Path(folder)
    for sub in ('previews','png600','svg'):(folder/sub).mkdir(exist_ok=True)
    # Leave dedicated headroom for legends instead of covering measured peaks.
    if name=='q1_dispatch':
        fig.axes[0].set_ylim(-.4,10.6);fig.axes[1].set_ylim(-5.6,8.4)
        fig.axes[2].set_ylim(.6,12.8);fig.axes[3].set_ylim(.25,1.65)
    elif name=='q4_price':
        lo,hi=fig.axes[0].get_ylim();fig.axes[0].set_ylim(lo,hi+(hi-lo)*.20)
        fig.axes[1].set_ylim(.6,12.8)
    elif name=='q3_typical':
        lo,hi=fig.axes[0].get_ylim();fig.axes[0].set_ylim(lo,hi+(hi-lo)*.16)
    finish_axes(fig)
    fig.savefig(folder/(name+'.pdf'))
    fig.savefig(folder/'svg'/(name+'.svg'))
    fig.savefig(folder/'png600'/(name+'.png'),dpi=600)
    fig.savefig(folder/'previews'/(name+'.png'),dpi=180)
    plt.close(fig)

def seasonal_surface(inputs,months):
    """Four representative months; hourly mesh of actual 10-minute data.

    x = time of day, y = day in month, z = (load-PV)/1000 MW.
    Mark each day's maximum among all 144 actual periods. Same z and colour scale.
    """
    months=np.asarray(months);selected=(3,6,9,12)
    norm=colors.Normalize(-8,8)
    dots=['#54349A','#18834B','#C64132','#9A930D']
    fig=plt.figure(figsize=(7.6,4.95))
    for j,m in enumerate(selected):
        ax=fig.add_subplot(2,2,j+1,projection='3d')
        a=(inputs['load'][months==m]-inputs['pv'][months==m])/1000
        assert a.min()>=-8 and a.max()<=8,(m,a.min(),a.max())
        slots=np.r_[np.arange(0,144,6),143]
```

```
        xx,yy=np.meshgrid(slots/6,np.arange(1,len(a)+1));zz=a[:,slots]
        segs=[];values=[]
        for d in range(len(a)):
            for t in range(len(slots)-1):
                segs.append([(xx[d,t],yy[d,t],zz[d,t]),(xx[d,t+1],yy[d,t+1],zz[d,t+1])])
                values.append((zz[d,t]+zz[d,t+1])/2)
        for d in range(len(a)-1):
            for t in range(len(slots)):
                segs.append([(xx[d,t],yy[d,t],zz[d,t]),(xx[d+1,t],yy[d+1,t],zz[d+1,t])])
                values.append((zz[d,t]+zz[d+1,t])/2)
        collection=Line3DCollection(segs,cmap=SURFACE,norm=norm,linewidths=.43,alpha=.85)
        collection.set_array(np.asarray(values));ax.add_collection3d(collection)
        peak=a.argmax(axis=1);days=np.arange(1,len(a)+1)
        ax.scatter(peak/6,days,a[np.arange(len(a)),peak],c=dots[j],s=8,
                   edgecolors='white',linewidths=.15,depthshade=False,zorder=8)
        ax.set(xlim=(0,24),ylim=(1,31),zlim=(-8,8),xticks=[0,6,12,18,24],
               yticks=[1,15,31],zticks=[-8,-4,0,4,8])
        ax.set_title(f'({chr(97+j)}) {m}月',fontproperties=font_manager.FontProperties(
    family='SimHei',size=11),pad=0)
        ax.set_xlabel('时刻 / h',fontsize=9,labelpad=3)
        ax.set_ylabel('月内日序',fontsize=9,labelpad=7)
        ax.set_zlabel('净负荷 / MW',fontsize=9,labelpad=2)
        ax.tick_params(labelsize=8,pad=0)
        ax.view_init(elev=23,azim=-57);ax.set_proj_type('ortho')
        ax.set_box_aspect((1.4,1,.67))
        for axis in (ax.xaxis,ax.yaxis,ax.zaxis):
            axis.pane.set_facecolor((1,1,1,1));axis.pane.set_edgecolor('#E0E0E0')
            axis._axinfo['grid'].update(color='#E2E4E6',linewidth=.45)
            axis.line.set_color('#929292');axis.line.set_linewidth(.6)
    fig.subplots_adjust(left=0,right=.97,top=.99,bottom=.03,wspace=.07,hspace=.09)
    return fig
```

附录 C　各问指定时段的完整结果

我们把四问在四个指定日期的逐时购电量、4 小时分组储能与紧急事件完整列出，数值与提交的五份规定工作簿逐格一致。为便于逐项核对，我们保留两位小数并分日期分表排布；若需要更高精度，请直接查看我们提交的工作簿。

C.1　问题 2 指定结果

表 24　问题 2：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	680.23	14:00–14:10	0.00
16:00–16:10	489.95	18:00–18:10	701.46	20:00–20:10	0.00
全天常规购电量：68991.98 kWh				全天总费用：41918.85 元	

表 25　问题 2：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8169.92	91.76	4:00–8:00	1955.56	5259.45
8:00–12:00	6279.61	2777.38	12:00–16:00	3657.03	362.33
16:00–20:00	260.66	5765.39	20:00–24:00	526.61	3011.58
0:00 储电量：2058.88				24:00 储电量：1636.78	

表 26　问题 2：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	230.93	18:00–18:10	0.00	20:00–20:10	0.00
全天常规购电量：36523.04 kWh				全天总费用：21502.49 元	

表 27　问题 2：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	990.61	57.57	4:00–8:00	1355.74	1719.95
8:00–12:00	8311.86	0.00	12:00–16:00	0.00	0.00
16:00–20:00	148.50	5265.68	20:00–24:00	1278.87	2485.90
0:00 储电量：3182.63				24:00 储电量：3471.76	

表 28　问题 2：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	524.51	14:00–14:10	0.00
16:00–16:10	526.03	18:00–18:10	762.12	20:00–20:10	0.00
全天常规购电量：68842.58 kWh				全天总费用：49439.26 元	

表 29　问题 2：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9007.23	85.93	4:00–8:00	1885.31	5720.68
8:00–12:00	5978.77	1896.62	12:00–16:00	3538.48	574.24
16:00–20:00	75.44	6048.66	20:00–24:00	232.09	2384.32
0:00 储电量：1246.18				24:00 储电量：1324.59	

表 30　问题 2：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	997.23	14:00–14:10	0.00
16:00–16:10	859.25	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：98355.39 kWh				全天总费用：63456.61 元	

表 31　问题 2：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8514.82	60.47	4:00–8:00	2166.44	1430.21
8:00–12:00	5900.41	7059.00	12:00–16:00	6532.89	2220.11
16:00–20:00	9.18	5612.85	20:00–24:00	538.84	2842.53
0:00 储电量：1963.33				24:00 储电量：1898.36	

表 32　问题 2 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
无	0.00	无	0.00	20:20–21:30	1018.46	无	0.00
–	–	–	–	21:40–22:00	40.39	–	–
–	–	–	–	22:20–22:30	36.02	–	–

C.2　问题 3 指定结果

表 33　问题 3：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	558.17	14:00–14:10	0.00
16:00–16:10	537.66	18:00–18:10	701.05	20:00–20:10	0.00
全天常规购电量：66853.81 kWh				全天总费用：43037.70 元	

表 34　问题 3：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8172.73	91.76	4:00–8:00	1738.24	6042.51
8:00–12:00	5705.47	2664.33	12:00–16:00	5275.67	272.67
16:00–20:00	61.15	5657.81	20:00–24:00	526.61	3011.58
0:00 储电量：2056.35				24:00 储电量：1676.37	

表 35　问题 3：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	197.95	18:00–18:10	0.00	20:00–20:10	0.00
全天常规购电量：34929.74 kWh				全天总费用：20690.52 元	

表 36　问题 3：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	790.41	59.57	4:00–8:00	1002.19	1719.95
8:00–12:00	8774.71	0.00	12:00–16:00	0.00	0.00
16:00–20:00	156.93	5265.68	20:00–24:00	1271.09	2485.90
0:00 储电量：3266.66				24:00 储电量：3472.35	

表 37　问题 3：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	557.27	18:00–18:10	773.62	20:00–20:10	0.00
全天常规购电量：67541.88 kWh				全天总费用：46227.25 元	

表 38　问题 3：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8974.28	84.63	4:00–8:00	1908.11	6121.76
8:00–12:00	6293.08	1896.62	12:00–16:00	3941.91	434.00
16:00–20:00	196.75	5866.62	20:00–24:00	243.34	2981.17
0:00 储电量：1246.18				24:00 储电量：1331.44	

表 39　问题 3：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	946.45	14:00–14:10	0.00
16:00–16:10	873.78	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：97280.92 kWh				全天总费用：63388.89 元	

表 40　问题 3：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8514.23	60.47	4:00–8:00	2166.43	1442.92
8:00–12:00	5634.30	7116.51	12:00–16:00	6925.11	2252.05
16:00–20:00	9.18	5612.85	20:00–24:00	538.84	2842.53
0:00 储电量：1963.87				24:00 储电量：1898.36	

表 41　问题 3 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
9:30–10:00	113.04	无	0.00	20:20–21:10	374.11	无	0.00
–	–	–	–	21:20–21:30	13.08	–	–
–	–	–	–	21:40–22:00	40.81	–	–
–	–	–	–	22:20–22:30	36.02	–	–

C.3　波动电价日前主方案

表 42　问题 4-2：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	680.23	14:00–14:10	0.00
16:00–16:10	489.95	18:00–18:10	700.33	20:00–20:10	0.00
全天常规购电量：69026.66 kWh				全天总费用：43411.08 元	

表 43　问题 4-2：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8877.96	44.72	4:00–8:00	1572.62	5599.32
8:00–12:00	6279.61	2777.38	12:00–16:00	3570.65	366.38
16:00–20:00	223.13	5715.34	20:00–24:00	536.76	3002.67
0:00 储电量：2169.40				24:00 储电量：1673.16	

表 44　问题 4-2：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	230.93	18:00–18:10	484.03	20:00–20:10	0.00
全天常规购电量：36837.19 kWh				全天总费用：18609.89 元	

表 45　问题 4-2：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	762.69	347.46	4:00–8:00	1339.92	1719.95
8:00–12:00	8942.62	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5285.49	20:00–24:00	2025.68	3091.92
0:00 储电量：3156.42				24:00 储电量：3314.88	

表 46　问题 4-2：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	475.98	14:00–14:10	0.00
16:00–16:10	526.03	18:00–18:10	762.12	20:00–20:10	0.00
全天常规购电量：68830.77 kWh				全天总费用：51152.10 元	

表 47　问题 4-2：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8239.20	31.29	4:00–8:00	2571.51	5701.24
8:00–12:00	5978.77	1896.62	12:00–16:00	3475.00	520.38
16:00–20:00	75.44	5901.94	20:00–24:00	275.95	2538.54
0:00 储电量：1246.18				24:00 储电量：1367.11	

表 48　问题 4-2：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	986.10	14:00–14:10	0.00
16:00–16:10	859.25	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：98222.77 kWh				全天总费用：74692.77 元	

表 49　问题 4-2：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9500.24	60.47	4:00–8:00	1156.20	1430.21
8:00–12:00	5796.89	6955.48	12:00–16:00	5902.39	1729.08
16:00–20:00	7.46	5611.45	20:00–24:00	552.13	2842.53
0:00 储电量：1985.68				24:00 储电量：1910.32	

表 50　问题 4-2 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
无	0.00	无	0.00	20:10–21:30	1000.12	无	0.00
–	–	–	–	21:40–22:00	40.39	–	–
–	–	–	–	22:20–22:30	36.02	–	–

C.4　波动电价滚动主方案

表 51　问题 4-3：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	558.17	14:00–14:10	0.00
16:00–16:10	537.66	18:00–18:10	700.12	20:00–20:10	0.00
全天常规购电量：66695.93 kWh				全天总费用：44535.29 元	

表 52　问题 4-3：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8886.68	44.72	4:00–8:00	852.21	6045.86
8:00–12:00	5669.48	2663.32	12:00–16:00	5266.98	272.67
16:00–20:00	153.57	5617.08	20:00–24:00	536.76	3002.67
0:00 储电量：2161.56				24:00 储电量：1783.66	

表 53　问题 4-3：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	197.95	18:00–18:10	479.69	20:00–20:10	0.00
全天常规购电量：35241.80 kWh				全天总费用：18088.21 元	

表 54　问题 4-3：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	759.23	330.01	4:00–8:00	1406.10	1719.95
8:00–12:00	8858.35	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5285.49	20:00–24:00	2017.91	3091.92
0:00 储电量：3156.43				24:00 储电量：3307.88	

表 55　问题 4-3：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	557.27	18:00–18:10	773.62	20:00–20:10	0.00
全天常规购电量：67662.34 kWh				全天总费用：48346.91 元	

表 56　问题 4-3：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8249.85	31.29	4:00–8:00	2519.65	6073.63
8:00–12:00	6293.08	1896.62	12:00–16:00	3929.52	434.00
16:00–20:00	196.75	5730.12	20:00–24:00	282.98	3117.67
0:00 储电量：1246.18				24:00 储电量：1367.11	

表 57　问题 4-3：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	946.45	14:00–14:10	0.00
16:00–16:10	873.78	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：97208.96 kWh				全天总费用：74630.49 元	

表 58　问题 4-3：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9499.65	60.47	4:00–8:00	1156.19	1442.92
8:00–12:00	5592.34	7108.55	12:00–16:00	6622.62	1981.00
16:00–20:00	7.46	5611.45	20:00–24:00	552.13	2842.53
0:00 储电量：1986.21				24:00 储电量：1910.32	

表 59　问题 4-3 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
9:30–10:00	114.06	无	0.00	20:10–21:10	376.23	无	0.00
–	–	–	–	21:20–21:30	13.08	–	–
–	–	–	–	21:40–22:00	40.81	–	–
–	–	–	–	22:20–22:30	36.02	–	–

附录 D　预测、参数与反馈的精确对照数据

我们把预测误差、参数选择、窗口敏感性与反馈对照的精确数值集中列出，供复核我们正文中的结论。表中每个数字都由 `results/`数组重新导出，我们未在排版时手工改动过任何数值。

表 60　6 时与 12 时更新改善对应白天时段的预报误差（误差单位：kW）

目标区间	历史 MAE	0 时预报 MAE	本次预报 MAE	本次 RMSE
0–6 时	3.90	10.56	10.56	43.67
6–12 时	314.90	342.51	184.69	269.63
12–18 时	292.27	426.27	140.38	192.78
18–24 时	2.52	10.62	10.02	42.92

表 61　11 个月均节省费用，5 月的紧急电量仍有增加

月份	费用差/元	紧急电量差/kWh
2	32485.90	1058.54
3	46366.65	3874.99
4	21154.32	766.36
5	7519.83	-494.66
6	23793.18	3725.56
7	72216.31	11825.84
8	28061.38	1834.55
9	37140.92	3346.32
10	37597.10	2206.01
11	20582.33	1856.31
12	40575.05	4071.00

表 62　1 月验证未支持重复电价下的正终端奖励（库存修正费用/元）

γ	重复电价	波动电价
0.0	1339152.99	1499506.57
0.5	1339152.99	1495331.27
1.0	1355273.69	1508634.51
1.5	1355273.69	1508634.51

表 63　打乱时间配对未改变 LP 代理目标，却改变了反馈费用

指标	原始轨迹	逐时打乱	差值
LP 代理目标/元	47479.93	47479.93	-0.00
固定计划实际反馈费/元	47642.07	47840.80	198.74

表 64　本算例的 36 时段保留规则增加了紧急购电与总费

反馈规则	总费用/万元	紧急量/MWh	年底 SOC/kWh
即时放电	1386.39	85.48	1552.80
36 时段保留	1473.88	265.12	1552.80

表 65　1 月库存修正费用选型偏好 3 日均值，误差选型偏好 7 日均值

候选方法	MAE/kW	RMSE/kW	重复价费用/元	波动价费用/元
前一日	119.48	261.96	1339152.99	1499506.57
3 日均值	100.38	214.94	1329665.76	1483643.74
7 日均值	96.18	203.87	1331308.64	1486679.65
指数加权	96.74	204.52	1331466.49	1486855.16
同星期	141.56	285.35	1338800.61	1494250.52

表 66　7 天窗口回测费用较低，尚不能据此认定全年最优窗口

历史窗口/天	紧急电量/kWh	总费用/元	2 月初 SOC/kWh	年底 SOC/kWh
7	30405.57	13486528.22	9175.29	1347.37
14	43270.28	13642862.05	9340.95	1552.80
28	49801.19	13737954.20	9353.45	1667.29

表 67　仅改变计划惩罚权重的连续回测（实际紧急电仍按 5 倍价）

计划权重	总费用/万元	未利用电量/万 kWh	紧急电量/万 kWh
2	1481.29	159.82	43.44
3	1393.80	213.75	18.31
5	1386.39	275.35	8.55
8	1408.12	334.63	4.37

表 68　效率解释改变问题 1 费用，主设定维持两侧各 0.9

效率解释	η_c	η_d	费用/元	较主设定变化
单程均 0.9（主设定）	0.9000	0.9000	35126.95	0.00%
回路 0.9，对称分配	0.9487	0.9487	33801.50	-3.77%
仅充电有损耗	0.9000	1.0000	33416.53	-4.87%
仅放电有损耗	1.0000	0.9000	34168.61	-2.73%

表 69　同一计划下改用当日完全预知执行，现金费用仅再低 6.23 万元，紧急电量反而增加

指标	因果反馈（主结果）	当日完全预知（不可实现上界）
计划购电费/万元	1337.62	1337.62
紧急购电费/元	487678.99	425383.80
现金总费用/万元	1386.39	1380.16
紧急购电量/万 kWh	8.55	12.80
期末储电量/kWh	1552.80	1200.00
相对主结果节省/元	—	62295.19